STAR ROS2 Packages

Generated by Doxygen 1.16.1

Chapter 1

Directory Hierarchy

1.1 Directories

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 5

File Index

5.1 File List

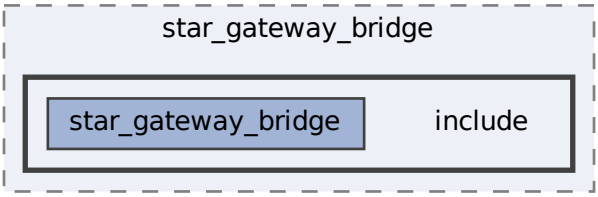
Here is a list of all files with brief descriptions:

Chapter 6

Directory Documentation

6.1 src/star_gateway_bridge/include Directory Reference

Directory dependency graph for include:

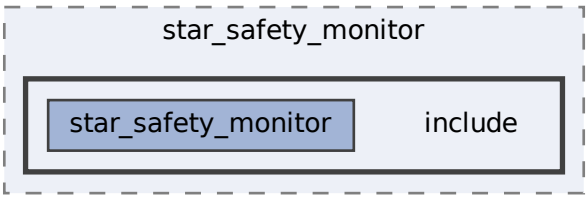


Directories

- directory star_gateway_bridge

6.2 src/star_safety_monitor/include Directory Reference

Directory dependency graph for include:

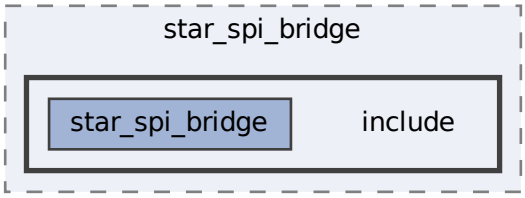


Directories

- directory star_safety_monitor

6.3 src/star_spi_bridge/include Directory Reference

Directory dependency graph for include:



Directories

- directory star_spi_bridge

6.4 src Directory Reference

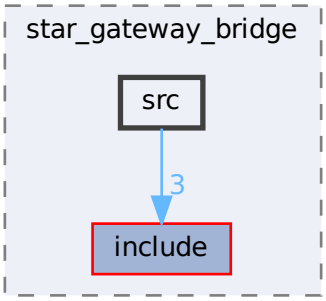
Directory dependency graph for src:

Directories

- directory star_gateway_bridge
- directory star_safety_monitor
- directory star_spi_bridge

6.5 src/star_gateway_bridge/src Directory Reference

Directory dependency graph for src:

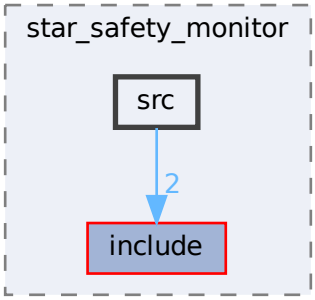


Files

- file main.cpp
- file message_converter.cpp
- file star_gateway_bridge_node.cpp

6.6 src/star_safety_monitor/src Directory Reference

Directory dependency graph for src:

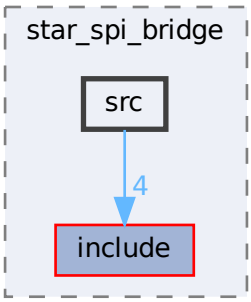


Files

- file safety_monitor.cpp
- file safety_monitor_node.cpp

6.7 src/star_spi_bridge/src Directory Reference

Directory dependency graph for src:

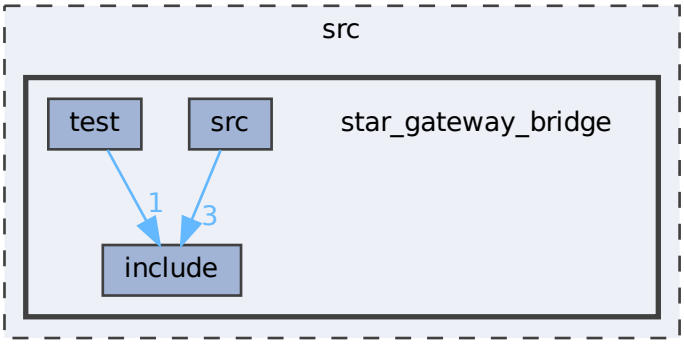


Files

- file main.cpp
- file spi_driver.cpp
- file spi_message_converter.cpp
- file star_spi_driver_node.cpp

6.8 src/star_gateway_bridge Directory Reference

Directory dependency graph for star_gateway_bridge:

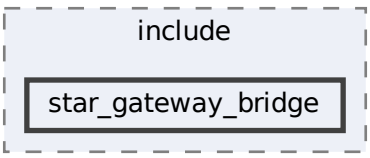


Directories

- directory include
- directory src
- directory test

6.9 src/star_gateway_bridge/include/star_gateway_bridge Directory Reference

Directory dependency graph for star_gateway_bridge:

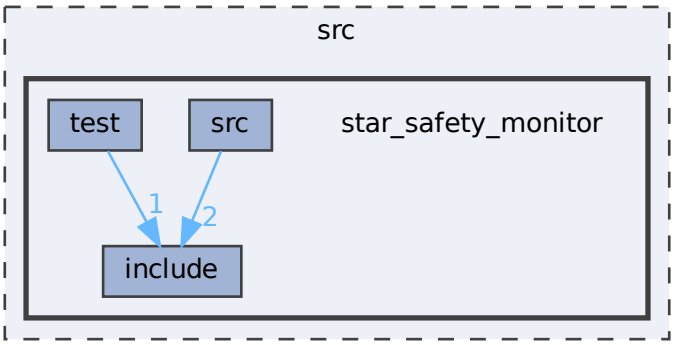


Files

- file message_converter.hpp
- file star_gateway_bridge_node.hpp

6.10 src/star_safety_monitor Directory Reference

Directory dependency graph for star_safety_monitor:



Directories

- directory include
- directory src
- directory test

6.11 src/star_safety_monitor/include/star_safety_monitor Directory Reference

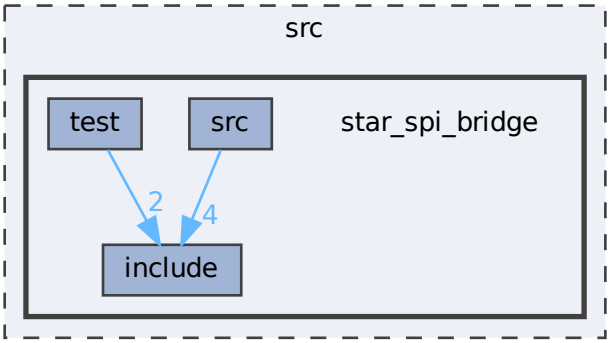
Directory dependency graph for star_safety_monitor:

Files

- file safety_monitor.hpp

6.12 src/star_spi_bridge Directory Reference

Directory dependency graph for star_spi_bridge:

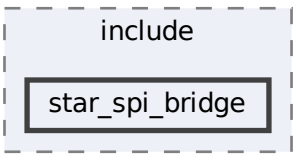


Directories

- directory include
- directory src
- directory test

6.13 src/star_spi_bridge/include/star_spi_bridge Directory Reference

Directory dependency graph for star_spi_bridge:

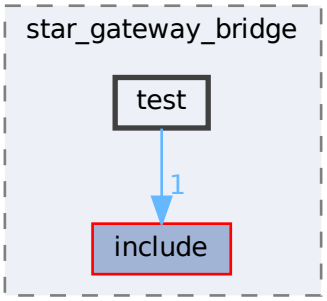


Files

- file spi_driver.hpp
- file spi_message_converter.hpp
- file star_spi_driver_node.hpp

6.14 src/star_gateway_bridge/test Directory Reference

Directory dependency graph for test:

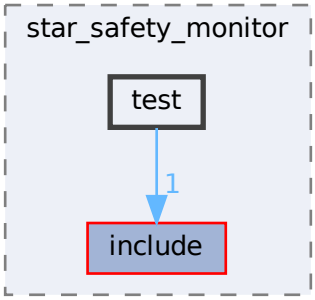


Files

- file test_message_converter.cpp

6.15 src/star_safety_monitor/test Directory Reference

Directory dependency graph for test:

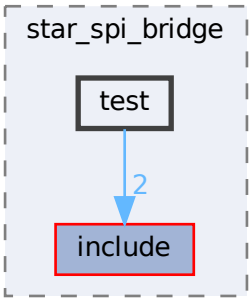


Files

- file test_safety_monitor.cpp

6.16 src/star_spi_bridge/test Directory Reference

Directory dependency graph for test:



Files

- file test_spi_driver.cpp
- file test_spi_message_converter.cpp

Chapter 7

Namespace Documentation

7.1 star Namespace Reference

Classes

- class MessageConverter

  Message converter for ROS2 <-> Protobuf translation.
- class StarGatewayBridgeNode

  ROS2 node that bridges the ROS2 ecosystem with the Go gateway service via gRPC.
- class MessageConverterTest

Functions

- TEST_F (MessageConverterTest, TwistToCommandZeroVelocity)
- TEST_F (MessageConverterTest, TwistToCommandPureTranslation)
- TEST_F (MessageConverterTest, TwistToCommandPureRotation)
- TEST_F (MessageConverterTest, TwistToCommandMixedMotion)
- TEST_F (MessageConverterTest, TwistToCommandNegativeLinear)
- TEST_F (MessageConverterTest, TwistToCommandNegativeAngular)
- TEST_F (MessageConverterTest, CommandToTwistZeroVelocity)
- TEST_F (MessageConverterTest, CommandToTwistPureTranslation)
- TEST_F (MessageConverterTest, CommandToTwistPureRotation)
- TEST_F (MessageConverterTest, CommandToTwistMixedMotion)
- TEST_F (MessageConverterTest, KinematicsRoundtripZero)
- TEST_F (MessageConverterTest, KinematicsRoundtripTranslation)
- TEST_F (MessageConverterTest, KinematicsRoundtripRotation)
- TEST_F (MessageConverterTest, KinematicsRoundtripMixed)
- TEST_F (MessageConverterTest, TwistToCommandNaNLinear)
- TEST_F (MessageConverterTest, TwistToCommandNaNAngular)
- TEST_F (MessageConverterTest, TwistToCommandInfinityLinear)
- TEST_F (MessageConverterTest, TwistToCommandInfinityAngular)
- TEST_F (MessageConverterTest, TwistToCommandNegativeInfinity)
- TEST_F (MessageConverterTest, TwistToCommandZeroWheelBase)
- TEST_F (MessageConverterTest, TwistToCommandNegativeWheelBase)
- TEST_F (MessageConverterTest, CommandToTwistNaNMotor0)
- TEST_F (MessageConverterTest, CommandToTwistNaNMotor1)
- TEST_F (MessageConverterTest, CommandToTwistInfinityMotor0)

- TEST_F (MessageConverterTest, PidConfigToGainsNominal)
- TEST_F (MessageConverterTest, PidConfigToGainsZeroGains)
- TEST_F (MessageConverterTest, PidConfigToGainsNaN)
- TEST_F (MessageConverterTest, PidConfigToGainsInfinity)
- TEST_F (MessageConverterTest, PidConfigToGainsNegativeGains)
- TEST_F (MessageConverterTest, PidConfigToGainsLargeValues)
- TEST_F (MessageConverterTest, TwistToCommandMaxVelocity)
- TEST_F (MessageConverterTest, TwistToCommandVerySmallWheelBase)
- TEST_F (MessageConverterTest, TwistToCommandVeryLargeWheelBase)

7.1.1 Function Documentation

7.1.1.1 TEST_F() [1/33]

```
star::TEST_F (
             MessageConverterTest ,
             CommandToTwistInfinityMotor0 )
```

Definition at line 358 of file test_message_converter.cpp.

7.1.1.2 TEST_F() [2/33]

```
star::TEST_F (
             MessageConverterTest ,
             CommandToTwistMixedMotion )
```

Definition at line 176 of file test_message_converter.cpp.

7.1.1.3 TEST_F() [3/33]

```
star::TEST_F (
             MessageConverterTest ,
             CommandToTwistNaNMotor0 )
```

Definition at line 338 of file test_message_converter.cpp.

7.1.1.4 TEST_F() [4/33]

```
star::TEST_F (
             MessageConverterTest ,
             CommandToTwistNaNMotor1 )
```

Definition at line 348 of file test_message_converter.cpp.

7.1.1.5 TEST_F() [5/33]

```
star::TEST_F (
             MessageConverterTest ,
             CommandToTwistPureRotation )
```

Definition at line 155 of file test_message_converter.cpp.

7.1.1.6 TEST_F() [6/33]

```
star::TEST_F (
             MessageConverterTest ,
             CommandToTwistPureTranslation )
```

Definition at line 138 of file test_message_converter.cpp.

7.1.1.7 TEST_F() [7/33]

```
star::TEST_F (
             MessageConverterTest ,
             CommandToTwistZeroVelocity )
```

Definition at line 125 of file test_message_converter.cpp.

7.1.1.8 TEST_F() [8/33]

```
star::TEST_F (
             MessageConverterTest ,
             KinematicsRoundtripMixed )
```

Definition at line 248 of file test_message_converter.cpp.

7.1.1.9 TEST_F() [9/33]

```
star::TEST_F (
             MessageConverterTest ,
             KinematicsRoundtripRotation )
```

Definition at line 232 of file test_message_converter.cpp.

7.1.1.10 TEST_F() [10/33]

```
star::TEST_F (
             MessageConverterTest ,
             KinematicsRoundtripTranslation )
```

Definition at line 216 of file test_message_converter.cpp.

7.1.1.11 TEST_F() [11/33]

```
star::TEST_F (
             MessageConverterTest ,
             KinematicsRoundtripZero )
```

Definition at line 200 of file test_message_converter.cpp.

7.1.1.12 TEST_F() [12/33]

```
star::TEST_F (
             MessageConverterTest ,
             PidConfigToGainsInfinity )
```

Definition at line 413 of file test_message_converter.cpp.

7.1.1.13 TEST_F() [13/33]

```
star::TEST_F (
             MessageConverterTest ,
             PidConfigToGainsLargeValues )
```

Definition at line 441 of file test_message_converter.cpp.

7.1.1.14 TEST_F() [14/33]

```
star::TEST_F (
             MessageConverterTest ,
             PidConfigToGainsNaN )
```

Definition at line 402 of file test_message_converter.cpp.

7.1.1.15 TEST_F() [15/33]

```
star::TEST_F (
             MessageConverterTest ,
             PidConfigToGainsNegativeGains )
```

Definition at line 424 of file test_message_converter.cpp.

7.1.1.16 TEST_F() [16/33]

```
star::TEST_F (
             MessageConverterTest ,
             PidConfigToGainsNominal )
```

Definition at line 372 of file test_message_converter.cpp.

7.1.1.17 TEST_F() [17/33]

```
star::TEST_F (
             MessageConverterTest ,
             PidConfigToGainsZeroGains )
```

Definition at line 387 of file test_message_converter.cpp.

7.1.1.18 TEST_F() [18/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandInfinityAngular )
```

Definition at line 298 of file test_message_converter.cpp.

7.1.1.19 TEST_F() [19/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandInfinityLinear )
```

Definition at line 288 of file test_message_converter.cpp.

7.1.1.20 TEST_F() [20/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandMaxVelocity )
```

Definition at line 460 of file test_message_converter.cpp.

7.1.1.21 TEST_F() [21/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandMixedMotion )
```

Definition at line 72 of file test_message_converter.cpp.

7.1.1.22 TEST_F() [22/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandNaNAngular )
```

Definition at line 278 of file test_message_converter.cpp.

7.1.1.23 TEST_F() [23/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandNaNLinear )
```

Definition at line 268 of file test_message_converter.cpp.

7.1.1.24 TEST_F() [24/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandNegativeAngular )
```

Definition at line 105 of file test_message_converter.cpp.

7.1.1.25 TEST_F() [25/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandNegativeInfinity )
```

Definition at line 308 of file test_message_converter.cpp.

7.1.1.26 TEST_F() [26/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandNegativeLinear )
```

Definition at line 91 of file test_message_converter.cpp.

7.1.1.27 TEST_F() [27/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandNegativeWheelBase )
```

Definition at line 328 of file test_message_converter.cpp.

7.1.1.28 TEST_F() [28/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandPureRotation )
```

Definition at line 56 of file test_message_converter.cpp.

7.1.1.29 TEST_F() [29/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandPureTranslation )
```

Definition at line 42 of file test_message_converter.cpp.

7.1.1.30 TEST_F() [30/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandVeryLargeWheelBase )
```

Definition at line 492 of file test_message_converter.cpp.

7.1.1.31 TEST_F() [31/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandVerySmallWheelBase )
```

Definition at line 477 of file test_message_converter.cpp.

7.1.1.32 TEST_F() [32/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandZeroVelocity )
```

Definition at line 29 of file test_message_converter.cpp.

7.1.1.33 TEST_F() [33/33]

```
star::TEST_F (
             MessageConverterTest ,
             TwistToCommandZeroWheelBase )
```

Definition at line 318 of file test_message_converter.cpp.

7.2 star_safety_monitor Namespace Reference

Classes

- class SafetyMonitor

Enumerations

- enum class SeverityLevel { OK = 0 , WARN = 1 , ERROR = 2 , STALE = 3 }

7.2.1 Enumeration Type Documentation

7.2.1.1 SeverityLevel

```
enum class star_safety_monitor::SeverityLevel  [strong]
```

Enumerator

OK	

WARN	
ERROR	
STALE	

Definition at line 41 of file safety_monitor.hpp.

7.3 star_spi_bridge Namespace Reference

Classes

- class SpiDriver
  SPI driver for framed communication with the RX72N peripheral MCU.
- class SpiMessageConverter
- class StarSpiDriverNode

Enumerations

- enum class FrameType : uint8_t { VelocityCommand = 0x01 , TelemetryData = 0x02 }
  Frame type identifiers for SPI protocol messages.

Variables

- static constexpr int kLogThrottleMs = 1000
- static constexpr double kImuOrientationVar = 0.01
- static constexpr double kImuAngularVelocityVar = 0.001
- static constexpr double kImuLinearAccelVar = 0.01

7.3.1 Enumeration Type Documentation

7.3.1.1 FrameType

```
enum class star_spi_bridge::FrameType : uint8_t [strong]
```

Frame type identifiers for SPI protocol messages.

Each frame carries a one-byte type field at offset 6 in the header. Values match the RX72N firmware rx_frame_↩ type_t enum so both sides agree on the semantics of every message.

See also

SpiDriver::encode_frame Frame construction

SpiDriver::decode_frame Frame parsing

Since

Version 1.0.0

Enumerator

VelocityCommand	Motor velocity setpoint (RPi5 -> RX72N)

TelemetryData	Sensor telemetry payload (RX72N -> RPi5)

Definition at line 27 of file spi_driver.hpp.

7.3.2 Variable Documentation

7.3.2.1 kImuAngularVelocityVar

```
double star_spi_bridge::kImuAngularVelocityVar = 0.001  [static], [constexpr]
```

Definition at line 18 of file star_spi_driver_node.cpp.

7.3.2.2 kImuLinearAccelVar

```
double star_spi_bridge::kImuLinearAccelVar = 0.01  [static], [constexpr]
```

Definition at line 19 of file star_spi_driver_node.cpp.

7.3.2.3 kImuOrientationVar

```
double star_spi_bridge::kImuOrientationVar = 0.01  [static], [constexpr]
```

Definition at line 17 of file star_spi_driver_node.cpp.

7.3.2.4 kLogThrottleMs

```
int star_spi_bridge::kLogThrottleMs = 1000  [static], [constexpr]
```

Definition at line 13 of file star_spi_driver_node.cpp.

Chapter 8

Class Documentation

8.1 star::MessageConverter Class Reference

Message converter for ROS2 <-> Protobuf translation.

`#include <message_converter.hpp>`

Static Public Member Functions

- static bool twist_to_velocity_command (const geometry_msgs::msg::Twist &twist, star::v1::VelocityCommand &command, double wheel_base=0.150, uint32_t sequence=0)

  Convert ROS2 Twist message to Protobuf VelocityCommand.
- static bool string_to_system_status (const std_msgs::msg::String &status_msg, star::v1::SystemStatus &system_status)

  Convert ROS2 String message to Protobuf SystemStatus.
- static bool velocity_command_to_twist (const star::v1::VelocityCommand &command, geometry_msgs::msg::Twist &twist, double wheel_base=0.150)

  Convert Protobuf VelocityCommand to ROS2 Twist message.
- static bool pid_config_to_gains (const star::v1::PidConfig &pid_config, double &kp, double &ki, double &kd)

  Convert Protobuf PidConfig to individual gains (for ROS2 service).
- static bool is_valid_double (double value)

  Validate double value for NaN and infinity.
- static double clamp (double value, double min, double max)

  Clamp value to range [min, max].
- static int64_t ros_time_to_us (const rclcpp::Time &time)

  Convert ROS2 timestamp to microseconds since epoch.

8.1.1 Detailed Description

Message converter for ROS2 <-> Protobuf translation.

Provides stateless conversion functions with input validation and unit conversions. All conversions validate for NaN/infinity and clamp values to safe ranges.

Definition at line 30 of file message_converter.hpp.

8.1.2 Member Function Documentation

8.1.2.1 clamp()

```
double star::MessageConverter::clamp (
            double value,
            double min,
            double max)  [inline], [static]
```

Clamp value to range [min, max].

Parameters

value	Value to clamp
min	Minimum allowed value
max	Maximum allowed value

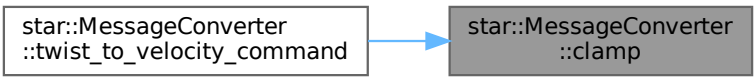
Returns

Clamped value

Definition at line 143 of file message_converter.hpp.

Referenced by twist_to_velocity_command().

Here is the caller graph for this function:



8.1.2.2 is_valid_double()

```
bool star::MessageConverter::is_valid_double (
            double value)  [inline], [static]
```

Validate double value for NaN and infinity.

Parameters

value	Value to validate

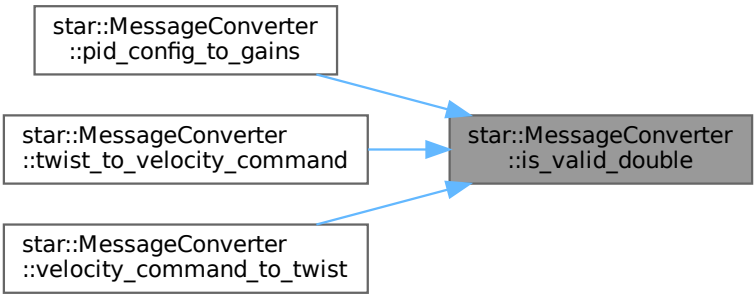
Returns

true if value is finite (not NaN, not infinity)

Definition at line 134 of file message_converter.hpp.

Referenced by pid_config_to_gains(), twist_to_velocity_command(), and velocity_command_to_twist().

Here is the caller graph for this function:



8.1.2.3 pid_config_to_gains()

```
bool star::MessageConverter::pid_config_to_gains (
            const star::v1::PidConfig & pid_config,
            double & kp,
            double & ki,
            double & kd)  [static]
```

Convert Protobuf PidConfig to individual gains (for ROS2 service).

Extracts PID gains from STAR PidConfig protobuf. No unit conversion needed (all values are dimensionless gains).

Parameters

pid_config	Input PidConfig protobuf
kp	Output proportional gain
ki	Output integral gain
kd	Output derivative gain

Returns

true if conversion successful, false if input validation failed

Definition at line 157 of file message_converter.cpp.

References is_valid_double().

Here is the call graph for this function:

8.1.2.4 ros_time_to_us()

```
int64_t star::MessageConverter::ros_time_to_us (
            const rclcpp::Time & time)  [static]
```

Convert ROS2 timestamp to microseconds since epoch.

Parameters

time	ROS2 time

Returns

Microseconds since epoch

Definition at line 179 of file message_converter.cpp.

8.1.2.5 string_to_system_status()

```
bool star::MessageConverter::string_to_system_status (
            const std_msgs::msg::String & status_msg,
            star::v1::SystemStatus & system_status)  [static]
```

Convert ROS2 String message to Protobuf SystemStatus.

Parses JSON-formatted robot status string into SystemStatus protobuf. Expected format: {"mode": "MANUAL", "esp32": true, "lidar": true, ...}

Parameters

status_msg	ROS2 String message (JSON format)

system_status	Output SystemStatus protobuf

Returns

true if conversion successful, false if parse failed

Definition at line 73 of file message_converter.cpp.

8.1.2.6 twist_to_velocity_command()

```
bool star::MessageConverter::twist_to_velocity_command (
            const geometry_msgs::msg::Twist & twist,
            star::v1::VelocityCommand & command,
            double wheel_base = 0.150,
            uint32_t sequence = 0)  [static]
```

Convert ROS2 Twist message to Protobuf VelocityCommand.

Maps differential drive velocities from ROS2 standard Twist to STAR VelocityCommand. Uses differential drive kinematics: (linear, angular) -> (motor_0, motor_1) wheel velocities. Note: motor_0 = left wheel, motor_1 = right wheel for differential drive.

Conversions:

- Linear velocity (m/s) -> motor_0/motor_1 wheel velocities (m/s)
- Angular velocity (rad/s) -> Wheel velocity differential (m/s)
- Timestamp -> Microseconds since epoch

Safety features:

- NaN/infinity validation (returns false on invalid input)
- Velocity clamping to +/-2.0 m/s (VelocityCommand valid range)

Parameters

twist	ROS2 Twist message (differential drive command)
command	Output VelocityCommand protobuf
wheel_base	Distance between wheels in meters (default: 0.150m)
sequence	Optional sequence number for command ordering

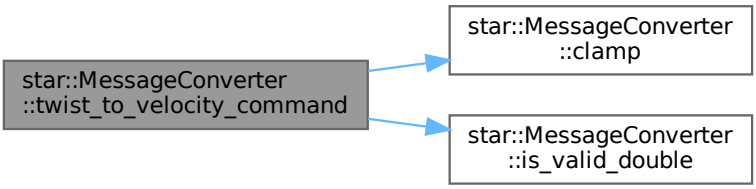
Returns

true if conversion successful, false if input validation failed

Definition at line 19 of file message_converter.cpp.

References clamp(), and is_valid_double().

Here is the call graph for this function:



8.1.2.7 velocity_command_to_twist()

```
bool star::MessageConverter::velocity_command_to_twist (
            const star::v1::VelocityCommand & command,
            geometry_msgs::msg::Twist & twist,
            double wheel_base = 0.150)  [static]
```

Convert Protobuf VelocityCommand to ROS2 Twist message.

Maps STAR VelocityCommand (motor_0/motor_1 wheel velocities) to ROS2 Twist (linear/angular velocities) using differential drive kinematics. Note: motor_0 = left wheel, motor_1 = right wheel for differential drive.

Conversions:

- motor_0/motor_1 wheel velocities (m/s) -> Linear velocity (m/s)
- Wheel velocity differential (m/s) -> Angular velocity (rad/s)

Kinematics:

- linear.x = (motor_0 + motor_1) / 2
- angular.z = (motor_1 - motor_0) / wheel_base

Parameters

command	Input VelocityCommand protobuf
twist	Output ROS2 Twist message

wheel_base	Distance between wheels in meters (default: 0.150m)

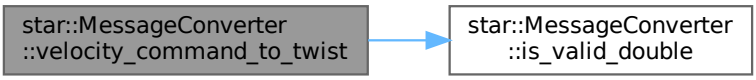
Returns

true if conversion successful, false if input validation failed

Definition at line 112 of file message_converter.cpp.

References is_valid_double().

Here is the call graph for this function:

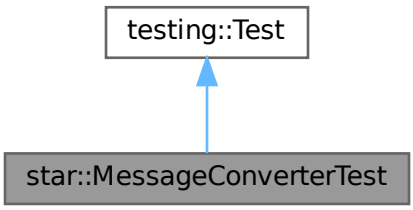


The documentation for this class was generated from the following files:

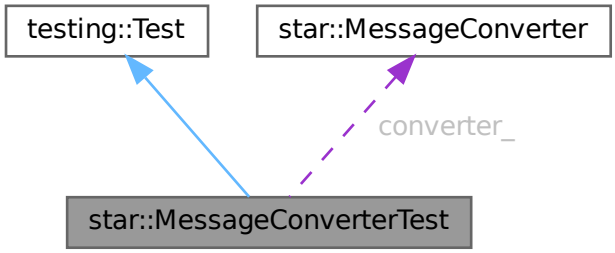
- src/star_gateway_bridge/include/star_gateway_bridge/message_converter.hpp
- src/star_gateway_bridge/src/message_converter.cpp

8.2 star::MessageConverterTest Class Reference

Inheritance diagram for star::MessageConverterTest:



Collaboration diagram for star::MessageConverterTest:



Protected Attributes

- MessageConverter converter_

Static Protected Attributes

- static constexpr double TOLERANCE = 1e-6
- static constexpr double WHEEL_BASE_M = 0.150

8.2.1 Detailed Description

Definition at line 17 of file test_message_converter.cpp.

8.2.2 Member Data Documentation

8.2.2.1 converter_

```
MessageConverter star::MessageConverterTest::converter_  [protected]
```

Definition at line 20 of file test_message_converter.cpp.

8.2.2.2 TOLERANCE

```
double star::MessageConverterTest::TOLERANCE = 1e-6  [static], [constexpr], [protected]
```

Definition at line 21 of file test_message_converter.cpp.

8.2.2.3 WHEEL_BASE_M

```
double star::MessageConverterTest::WHEEL_BASE_M = 0.150  [static], [constexpr], [protected]
```

Definition at line 22 of file test_message_converter.cpp.

The documentation for this class was generated from the following file:

- src/star_gateway_bridge/test/test_message_converter.cpp

8.3 SpiMessageConverter::Parameters Struct Reference

```
#include <spi_message_converter.hpp>
```

Public Attributes

- double wheel_base
- double wheel_radius
- int32_t ticks_per_rev

8.3.1 Detailed Description

Definition at line 18 of file spi_message_converter.hpp.

8.3.2 Member Data Documentation

8.3.2.1 ticks_per_rev

```
int32_t star_spi_bridge::SpiMessageConverter::Parameters::ticks_per_rev
```

Definition at line 22 of file spi_message_converter.hpp.

8.3.2.2 wheel_base

```
double star_spi_bridge::SpiMessageConverter::Parameters::wheel_base
```

Definition at line 20 of file spi_message_converter.hpp.

8.3.2.3 wheel_radius

```
double star_spi_bridge::SpiMessageConverter::Parameters::wheel_radius
```

Definition at line 21 of file spi_message_converter.hpp.

The documentation for this struct was generated from the following file:

- src/star_spi_bridge/include/star_spi_bridge/spi_message_converter.hpp

8.4 star_spi_bridge::SpiMessageConverter::Parameters Struct Reference

`#include <spi_message_converter.hpp>`

Public Attributes

- double wheel_base
- double wheel_radius
- int32_t ticks_per_rev

8.4.1 Detailed Description

Definition at line 18 of file spi_message_converter.hpp.

8.4.2 Member Data Documentation

8.4.2.1 ticks_per_rev

`int32_t star_spi_bridge::SpiMessageConverter::Parameters::ticks_per_rev`

Definition at line 22 of file spi_message_converter.hpp.

Referenced by star_spi_bridge::StarSpiDriverNode::on_configure().

8.4.2.2 wheel_base

`double star_spi_bridge::SpiMessageConverter::Parameters::wheel_base`

Definition at line 20 of file spi_message_converter.hpp.

Referenced by star_spi_bridge::StarSpiDriverNode::on_configure().

8.4.2.3 wheel_radius

`double star_spi_bridge::SpiMessageConverter::Parameters::wheel_radius`

Definition at line 21 of file spi_message_converter.hpp.

Referenced by star_spi_bridge::StarSpiDriverNode::on_configure().

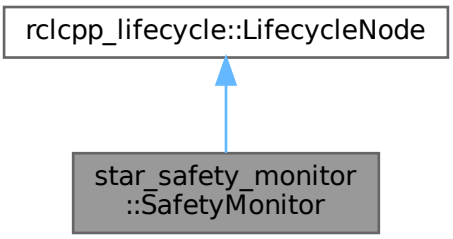
The documentation for this struct was generated from the following file:

- src/star_spi_bridge/include/star_spi_bridge/spi_message_converter.hpp

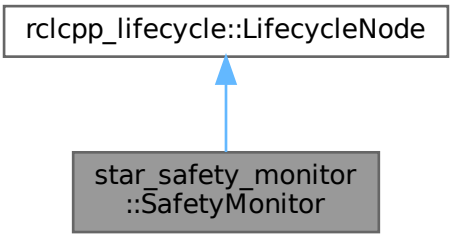
8.5 star_safety_monitor::SafetyMonitor Class Reference

`#include <safety_monitor.hpp>`

Inheritance diagram for star_safety_monitor::SafetyMonitor:



Collaboration diagram for star_safety_monitor::SafetyMonitor:



Public Member Functions

- SafetyMonitor (const rclcpp::NodeOptions &options=rclcpp::NodeOptions())
- ~SafetyMonitor ()
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn on_configure (const rclcpp_↩ lifecycle::State &previous_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn on_activate (const rclcpp_↩ lifecycle::State &previous_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn on_deactivate (const rclcpp_↩ lifecycle::State &previous_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn on_cleanup (const rclcpp_↩ lifecycle::State &previous_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn on_shutdown (const rclcpp_↩ lifecycle::State &previous_state) override

8.5.1 Detailed Description

Definition at line 43 of file safety_monitor.hpp.

8.5.2 Constructor & Destructor Documentation

8.5.2.1 SafetyMonitor()

```
star_safety_monitor::SafetyMonitor::SafetyMonitor (
            const rclcpp::NodeOptions & options = rclcpp::NodeOptions())  [explicit]
```

Definition at line 30 of file safety_monitor.cpp.

8.5.2.2 ~SafetyMonitor()

```
star_safety_monitor::SafetyMonitor::~SafetyMonitor ()
```

Definition at line 36 of file safety_monitor.cpp.

8.5.3 Member Function Documentation

8.5.3.1 on_activate()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_safety_monitor::↩
SafetyMonitor::on_activate (
            const rclcpp_lifecycle::State & previous_state)  [override]
```

Definition at line 114 of file safety_monitor.cpp.

8.5.3.2 on_cleanup()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_safety_monitor::↩
SafetyMonitor::on_cleanup (
            const rclcpp_lifecycle::State & previous_state)  [override]
```

Definition at line 165 of file safety_monitor.cpp.

8.5.3.3 on_configure()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_safety_monitor::↩
SafetyMonitor::on_configure (
            const rclcpp_lifecycle::State & previous_state)  [override]
```

Definition at line 42 of file safety_monitor.cpp.

References star_safety_monitor::OK.

8.5.3.4 on_deactivate()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_safety_monitor::↩
SafetyMonitor::on_deactivate (
            const rclcpp_lifecycle::State & previous_state)  [override]
```

Definition at line 140 of file safety_monitor.cpp.

8.5.3.5 on_shutdown()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_safety_monitor::↩
SafetyMonitor::on_shutdown (
            const rclcpp_lifecycle::State & previous_state)  [override]
```

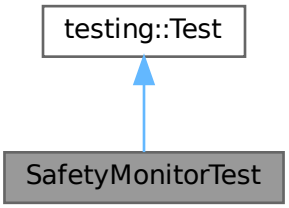
Definition at line 190 of file safety_monitor.cpp.

The documentation for this class was generated from the following files:

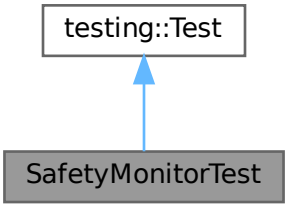
- src/star_safety_monitor/include/star_safety_monitor/safety_monitor.hpp
- src/star_safety_monitor/src/safety_monitor.cpp

8.6 SafetyMonitorTest Class Reference

Inheritance diagram for SafetyMonitorTest:



Collaboration diagram for SafetyMonitorTest:



Protected Member Functions

- void SetUp () override
- void TearDown () override

8.6.1 Detailed Description

Definition at line 39 of file test_safety_monitor.cpp.

8.6.2 Member Function Documentation

8.6.2.1 SetUp()

```
void SafetyMonitorTest::SetUp ()  [inline], [override], [protected]
```

Definition at line 42 of file test_safety_monitor.cpp.

8.6.2.2 TearDown()

```
void SafetyMonitorTest::TearDown ()  [inline], [override], [protected]
```

Definition at line 49 of file test_safety_monitor.cpp.

The documentation for this class was generated from the following file:

- src/star_safety_monitor/test/test_safety_monitor.cpp

8.7 spi_ioc_transfer Struct Reference

Public Attributes

- uint64_t tx_buf_
- uint64_t rx_buf_
- uint32_t len_
- uint32_t speed_hz_
- uint16_t delay_usecs_
- uint8_t bits_per_word_
- uint8_t cs_change_
- uint32_t pad_

8.7.1 Detailed Description

Definition at line 28 of file spi_driver.cpp.

8.7.2 Member Data Documentation

8.7.2.1 bits_per_word_

```
uint8_t spi_ioc_transfer::bits_per_word_
```

Definition at line 35 of file spi_driver.cpp.

8.7.2.2 cs_change_

```
uint8_t spi_ioc_transfer::cs_change_
```

Definition at line 36 of file spi_driver.cpp.

8.7.2.3 delay_usecs_

```
uint16_t spi_ioc_transfer::delay_usecs_
```

Definition at line 34 of file spi_driver.cpp.

8.7.2.4 len_

```
uint32_t spi_ioc_transfer::len_
```

Definition at line 32 of file spi_driver.cpp.

8.7.2.5 pad_

```
uint32_t spi_ioc_transfer::pad_
```

Definition at line 37 of file spi_driver.cpp.

8.7.2.6 rx_buf_

```
uint64_t spi_ioc_transfer::rx_buf_
```

Definition at line 31 of file spi_driver.cpp.

8.7.2.7 speed_hz_

```
uint32_t spi_ioc_transfer::speed_hz_
```

Definition at line 33 of file spi_driver.cpp.

8.7.2.8 tx_buf_

```
uint64_t spi_ioc_transfer::tx_buf_
```

Definition at line 30 of file spi_driver.cpp.

The documentation for this struct was generated from the following file:

- src/star_spi_bridge/src/spi_driver.cpp

8.8 SpiDriver Class Reference

SPI driver for framed communication with the RX72N peripheral MCU.

```
#include <spi_driver.hpp>
```

Public Member Functions

- SpiDriver (const std::string &device_path, uint32_t speed_hz=10000000)
  Construct a new SpiDriver.
- virtual ~SpiDriver ()
  Destructor – calls close_device() if the device is still open.
- bool initialize ()
  Open and configure the SPI device.
- void close_device ()
  Close the SPI device file descriptor.
- bool transfer (const std::vector< uint8_t > &tx_data, std::vector< uint8_t > &rx_data)
  Perform a full-duplex SPI transfer.

Static Public Member Functions

- static void encode_frame (uint16_t seq, FrameType type, uint8_t flags, const std::vector< uint8_t > &payload, std::vector< uint8_t > &out_frame)
  Encode an application payload into the STAR wire format.
- static bool decode_frame (const std::vector< uint8_t > &frame, uint16_t &seq, FrameType &type, uint8_t &flags, std::vector< uint8_t > &payload)
  Decode a STAR wire frame into its constituent fields.
- static uint32_t calculate_crc32 (const std::vector< uint8_t > &data)
  Compute the IEEE 802.3 CRC-32 of the given byte vector.

Static Public Attributes

- static constexpr size_t k_header_size = 8
  SYNC + SEQ + LEN + TYPE + FLAGS = 8 bytes.
- static constexpr size_t k_crc_size = 4
  CRC-32 trailer length in bytes.
- static constexpr size_t k_max_payload_size = 1024
  Maximum application payload per frame (bytes).
- static constexpr uint16_t k_sync_word = 0x55AA
  SYNC word transmitted in every frame header.

8.8.1 Detailed Description

SPI driver for framed communication with the RX72N peripheral MCU.

Implements the STAR binary framing protocol over Linux spidev. The wire format is:

SYNC(2, LE) + SEQ(2, LE) + LEN(2, LE) + TYPE(1) + FLAGS(1)

- PAYLOAD(0-1024) + CRC-32 (IEEE 802.3, 4 B, LE)

The Raspberry Pi 5 acts as SPI controller at 10 MHz, Mode 0 (CPOL=0, CPHA=0).

Thread Safety

- CRC-32 table initialisation is thread-safe (`std::call_once`).
- `encode_frame` / `decode_frame` / `calculate_crc32` are stateless and safe to call from any thread.
- `initialize`, `close_device`, and `transfer` share `spi_fd_` and must not be called concurrently on the same instance.

Invariant

After a successful `initialize()`, `spi_fd_ >= 0`.

See also

star_spi_bridge::FrameType Supported frame types

Since

Version 1.0.0

Definition at line 58 of file spi_driver.hpp.

8.8.2 Constructor & Destructor Documentation

8.8.2.1 SpiDriver()

```
star_spi_bridge::SpiDriver::SpiDriver (
            const std::string & device_path,
            uint32_t speed_hz = 10000000)  [explicit]
```

Construct a new SpiDriver.

Parameters

in	*device_path*	Path to the Linux spidev node (e.g. "/dev/spidev0.0").

in	*speed_hz*	SPI clock frequency in Hz (default 10 MHz).

Precondition

device_path is a non-empty, null-terminated string.

Postcondition

CRC-32 lookup table is initialised (thread-safe, once).

Definition at line 102 of file spi_driver.cpp.

8.8.2.2 ~SpiDriver()

```
star_spi_bridge::SpiDriver::~SpiDriver () [virtual]
```

Destructor – calls close_device() if the device is still open.

Definition at line 108 of file spi_driver.cpp.

8.8.3 Member Function Documentation

8.8.3.1 calculate_crc32()

```
uint32_t star_spi_bridge::SpiDriver::calculate_crc32 (
            const std::vector< uint8_t > & data) [static]
```

Compute the IEEE 802.3 CRC-32 of the given byte vector.

Parameters

in	*data*	Input bytes.

Returns

CRC-32 value.

Precondition

CRC-32 table has been initialised (guaranteed by the constructor).

Postcondition

Return value matches `crc32(0, data, len)` from zlib.

Note

Public for unit-test access.

Definition at line 311 of file spi_driver.cpp.

Referenced by TEST_F().

Here is the caller graph for this function:

8.8.3.2 close_device()

```
void star_spi_bridge::SpiDriver::close_device ()
```

Close the SPI device file descriptor.

Postcondition

`spi_fd_ == -1`.

Definition at line 153 of file spi_driver.cpp.

8.8.3.3 decode_frame()

```
bool star_spi_bridge::SpiDriver::decode_frame (
            const std::vector< uint8_t > & frame,
            uint16_t & seq,
            FrameType & type,
            uint8_t & flags,
            std::vector< uint8_t > & payload)  [static]
```

Decode a STAR wire frame into its constituent fields.

Validates SYNC word, frame length, and CRC-32. On success the sequence number, type, flags, and payload are written to the output parameters.

Parameters

in	*frame*	Raw wire bytes to parse.
out	*seq*	Decoded sequence number.

out	*type*	Decoded frame type.
out	*flags*	Decoded flags byte.
out	*payload*	Extracted payload bytes.

Returns

true Frame valid (SYNC, length, and CRC all match).

false Frame invalid or corrupted.

Precondition

`frame.size() >= k_header_size + k_crc_size` (12 bytes minimum).

Postcondition

On success, all output parameters are populated.

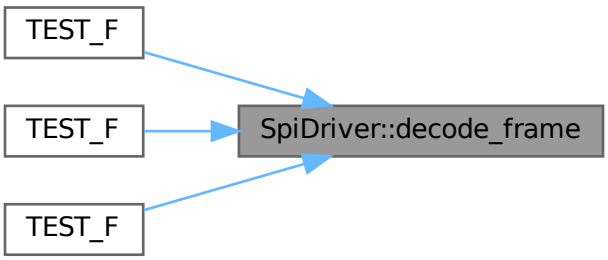
See also

encode_frame Inverse operation.

Definition at line 243 of file spi_driver.cpp.

Referenced by TEST_F(), TEST_F(), and TEST_F().

Here is the caller graph for this function:



8.8.3.4 encode_frame()

```
void star_spi_bridge::SpiDriver::encode_frame (
            uint16_t seq,
            FrameType type,
            uint8_t flags,
            const std::vector< uint8_t > & payload,
            std::vector< uint8_t > & out_frame)  [static]
```

Encode an application payload into the STAR wire format.

Builds the header (little-endian SYNC, SEQ, LEN, TYPE, FLAGS), appends the payload, then computes and appends the CRC-32. The output buffer is cleared and filled in place for reuse.

Parameters

in	*seq*	Sequence number (0-65535).

in	*type*	Frame type (see FrameType enum).
in	*flags*	Control flags byte.
in	*payload*	Application data (0-k_max_payload_size bytes).
out	*out_frame*	Encoded wire bytes (header + payload + CRC).

Precondition

`payload.size() <= k_max_payload_size`.

Postcondition

`out_frame.size() == k_header_size + payload.size() + k_crc_size`.

Exceptions

std::invalid_argument	If `payload.size() > k_max_payload_size`.

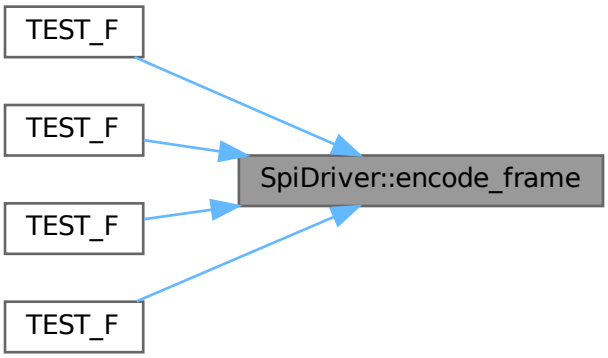
See also

decode_frame Inverse operation.

Definition at line 193 of file spi_driver.cpp.

Referenced by TEST_F(), TEST_F(), TEST_F(), and TEST_F().

Here is the caller graph for this function:



8.8.3.5 initialize()

```
bool star_spi_bridge::SpiDriver::initialize ()
```

Open and configure the SPI device.

Opens the spidev file descriptor, sets Mode 0, 8 bits per word, and the requested clock speed via ioctl calls.

Returns

true Device opened and configured successfully.
false Open or ioctl failed (error logged via RCLCPP_ERROR).

Precondition

`device_path_` refers to a valid spidev node.

Postcondition

On success, `spi_fd_ >= 0` and the device is ready for `transfer`.

Note

Only supported on Linux; returns false on other platforms.

Definition at line 113 of file spi_driver.cpp.

8.8.3.6 transfer()

```
bool star_spi_bridge::SpiDriver::transfer (
            const std::vector< uint8_t > & tx_data,
            std::vector< uint8_t > & rx_data)
```

Perform a full-duplex SPI transfer.

Parameters

in	*tx_data*	Data to transmit.
out	*rx_data*	Buffer filled with received data (resized to match tx_data).

Returns

true Transfer completed successfully.
false Device not open or ioctl failed.

Precondition

`initialize()` returned true.

Postcondition

`rx_data.size() == tx_data.size()`.

Note

Only supported on Linux; returns false on other platforms.

Definition at line 161 of file spi_driver.cpp.

8.8.4 Member Data Documentation

8.8.4.1 k_crc_size

```
size_t star_spi_bridge::SpiDriver::k_crc_size = 4  [static], [constexpr]
```

CRC-32 trailer length in bytes.

Definition at line 65 of file spi_driver.hpp.

8.8.4.2 k_header_size

```
size_t star_spi_bridge::SpiDriver::k_header_size = 8  [static], [constexpr]
```

SYNC + SEQ + LEN + TYPE + FLAGS = 8 bytes.

Definition at line 62 of file spi_driver.hpp.

8.8.4.3 k_max_payload_size

```
size_t star_spi_bridge::SpiDriver::k_max_payload_size = 1024  [static], [constexpr]
```

Maximum application payload per frame (bytes).

Definition at line 68 of file spi_driver.hpp.

Referenced by TEST_F().

8.8.4.4 k_sync_word

```
uint16_t star_spi_bridge::SpiDriver::k_sync_word = 0x55AA  [static], [constexpr]
```

SYNC word transmitted in every frame header.

Definition at line 71 of file spi_driver.hpp.

The documentation for this class was generated from the following files:

- src/star_spi_bridge/include/star_spi_bridge/spi_driver.hpp
- src/star_spi_bridge/src/spi_driver.cpp

8.9 star_spi_bridge::SpiDriver Class Reference

SPI driver for framed communication with the RX72N peripheral MCU.

```
#include <spi_driver.hpp>
```

Public Member Functions

- SpiDriver (const std::string &device_path, uint32_t speed_hz=10000000)

  Construct a new SpiDriver.
- virtual ~SpiDriver ()

  Destructor – calls close_device() if the device is still open.
- bool initialize ()

  Open and configure the SPI device.
- void close_device ()

  Close the SPI device file descriptor.
- bool transfer (const std::vector< uint8_t > &tx_data, std::vector< uint8_t > &rx_data)

  Perform a full-duplex SPI transfer.

Static Public Member Functions

- static void encode_frame (uint16_t seq, FrameType type, uint8_t flags, const std::vector< uint8_t > &payload, std::vector< uint8_t > &out_frame)

  Encode an application payload into the STAR wire format.
- static bool decode_frame (const std::vector< uint8_t > &frame, uint16_t &seq, FrameType &type, uint8_t &flags, std::vector< uint8_t > &payload)

  Decode a STAR wire frame into its constituent fields.
- static uint32_t calculate_crc32 (const std::vector< uint8_t > &data)

  Compute the IEEE 802.3 CRC-32 of the given byte vector.

Static Public Attributes

- static constexpr size_t k_header_size = 8

  SYNC + SEQ + LEN + TYPE + FLAGS = 8 bytes.
- static constexpr size_t k_crc_size = 4

  CRC-32 trailer length in bytes.
- static constexpr size_t k_max_payload_size = 1024

  Maximum application payload per frame (bytes).
- static constexpr uint16_t k_sync_word = 0x55AA

  SYNC word transmitted in every frame header.

8.9.1 Detailed Description

SPI driver for framed communication with the RX72N peripheral MCU.

Implements the STAR binary framing protocol over Linux spidev. The wire format is:

SYNC(2, LE) + SEQ(2, LE) + LEN(2, LE) + TYPE(1) + FLAGS(1)

- PAYLOAD(0-1024) + CRC-32 (IEEE 802.3, 4 B, LE)

The Raspberry Pi 5 acts as SPI controller at 10 MHz, Mode 0 (CPOL=0, CPHA=0).

Thread Safety

- CRC-32 table initialisation is thread-safe (`std::call_once`).
- `encode_frame` / `decode_frame` / `calculate_crc32` are stateless and safe to call from any thread.
- `initialize`, `close_device`, and `transfer` share `spi_fd_` and must not be called concurrently on the same instance.

Invariant

After a successful `initialize()`, `spi_fd_ >= 0`.

See also

star_spi_bridge::FrameType Supported frame types

Since

Version 1.0.0

Definition at line 58 of file spi_driver.hpp.

8.9.2 Constructor & Destructor Documentation

8.9.2.1 SpiDriver()

```
star_spi_bridge::SpiDriver::SpiDriver (
            const std::string & device_path,
            uint32_t speed_hz = 10000000)  [explicit]
```

Construct a new SpiDriver.

Parameters

in	*device_path*	Path to the Linux spidev node (e.g. "/dev/spidev0.0").
in	*speed_hz*	SPI clock frequency in Hz (default 10 MHz).

Precondition

device_path is a non-empty, null-terminated string.

Postcondition

CRC-32 lookup table is initialised (thread-safe, once).

Definition at line 102 of file spi_driver.cpp.

8.9.2.2 ~SpiDriver()

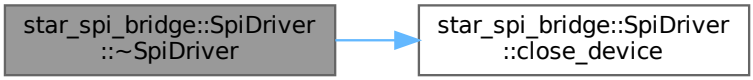
```
star_spi_bridge::SpiDriver::~SpiDriver ()  [virtual]
```

Destructor – calls close_device() if the device is still open.

Definition at line 108 of file spi_driver.cpp.

References close_device().

Here is the call graph for this function:



8.9.3 Member Function Documentation

8.9.3.1 calculate_crc32()

```
uint32_t star_spi_bridge::SpiDriver::calculate_crc32 (
            const std::vector< uint8_t > & data)  [static]
```

Compute the IEEE 802.3 CRC-32 of the given byte vector.

Parameters

in	*data*	Input bytes.

Returns

CRC-32 value.

Precondition

CRC-32 table has been initialised (guaranteed by the constructor).

Postcondition

Return value matches `crc32(0, data, len)` from zlib.

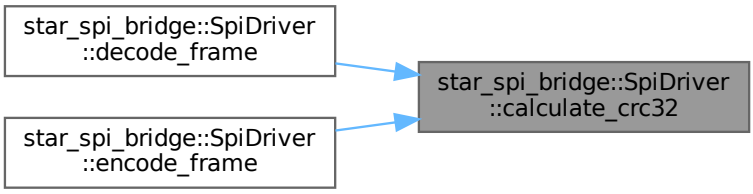
Note

Public for unit-test access.

Definition at line 311 of file spi_driver.cpp.

Referenced by decode_frame(), and encode_frame().

Here is the caller graph for this function:



8.9.3.2 close_device()

```
void star_spi_bridge::SpiDriver::close_device ()
```

Close the SPI device file descriptor.

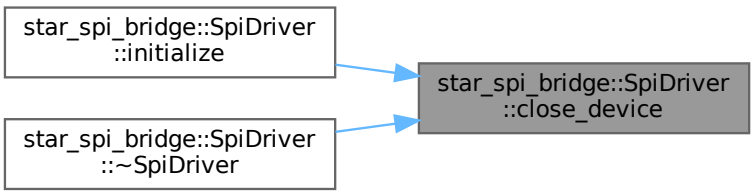
Postcondition

`spi_fd_ == -1`.

Definition at line 153 of file spi_driver.cpp.

Referenced by initialize(), and ~SpiDriver().

Here is the caller graph for this function:



8.9.3.3 decode_frame()

```
bool star_spi_bridge::SpiDriver::decode_frame (
            const std::vector< uint8_t > & frame,
            uint16_t & seq,
            FrameType & type,
            uint8_t & flags,
            std::vector< uint8_t > & payload)  [static]
```

Decode a STAR wire frame into its constituent fields.

Validates SYNC word, frame length, and CRC-32. On success the sequence number, type, flags, and payload are written to the output parameters.

Parameters

in	*frame*	Raw wire bytes to parse.
out	*seq*	Decoded sequence number.
out	*type*	Decoded frame type.
out	*flags*	Decoded flags byte.
out	*payload*	Extracted payload bytes.

Returns

true Frame valid (SYNC, length, and CRC all match).

false Frame invalid or corrupted.

Precondition

`frame.size() >= k_header_size + k_crc_size` (12 bytes minimum).

Postcondition

On success, all output parameters are populated.

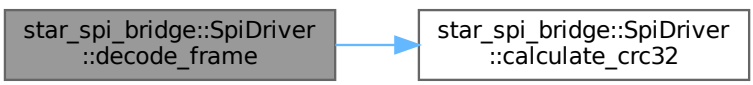
See also

encode_frame Inverse operation.

Definition at line 243 of file spi_driver.cpp.

References calculate_crc32(), k_crc_size, k_header_size, and k_sync_word.

Here is the call graph for this function:



8.9.3.4 encode_frame()

```
void star_spi_bridge::SpiDriver::encode_frame (
            uint16_t seq,
            FrameType type,
            uint8_t flags,
            const std::vector< uint8_t > & payload,
            std::vector< uint8_t > & out_frame)  [static]
```

Encode an application payload into the STAR wire format.

Builds the header (little-endian SYNC, SEQ, LEN, TYPE, FLAGS), appends the payload, then computes and appends the CRC-32. The output buffer is cleared and filled in place for reuse.

Parameters

in	*seq*	Sequence number (0-65535).
in	*type*	Frame type (see FrameType enum).
in	*flags*	Control flags byte.
in	*payload*	Application data (0-k_max_payload_size bytes).
out	*out_frame*	Encoded wire bytes (header + payload + CRC).

Precondition

`payload.size() <= k_max_payload_size`.

Postcondition

`out_frame.size() == k_header_size + payload.size() + k_crc_size`.

Exceptions

std::invalid_argument	If `payload.size() > k_max_payload_size`.

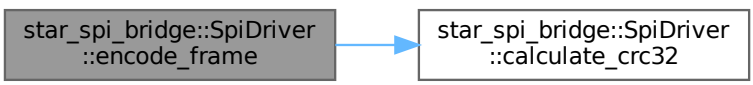
See also

decode_frame Inverse operation.

Definition at line 193 of file spi_driver.cpp.

References calculate_crc32(), k_crc_size, k_header_size, k_max_payload_size, and k_sync_word.

Here is the call graph for this function:



8.9.3.5 initialize()

```
bool star_spi_bridge::SpiDriver::initialize ()
```

Open and configure the SPI device.

Opens the spidev file descriptor, sets Mode 0, 8 bits per word, and the requested clock speed via ioctl calls.

Returns

true Device opened and configured successfully.

false Open or ioctl failed (error logged via RCLCPP_ERROR).

Precondition

`device_path_` refers to a valid spidev node.

Postcondition

On success, `spi_fd_ >= 0` and the device is ready for `transfer`.

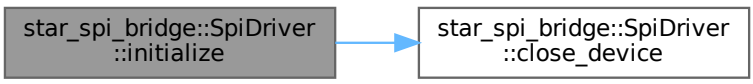
Note

Only supported on Linux; returns false on other platforms.

Definition at line 113 of file spi_driver.cpp.

References close_device(), SPI_IOC_WR_BITS_PER_WORD, SPI_IOC_WR_MAX_SPEED_HZ, SPI_IOC_WR_MODE, and SPI_MODE_0.

Here is the call graph for this function:



8.9.3.6 transfer()

```
bool star_spi_bridge::SpiDriver::transfer (
            const std::vector< uint8_t > & tx_data,
            std::vector< uint8_t > & rx_data)
```

Perform a full-duplex SPI transfer.

Parameters

in	*tx_data*	Data to transmit.

out	*rx_data*	Buffer filled with received data (resized to match tx_data).

Returns

true Transfer completed successfully.

false Device not open or ioctl failed.

Precondition

`initialize()` returned true.

Postcondition

`rx_data.size() == tx_data.size()`.

Note

Only supported on Linux; returns false on other platforms.

Definition at line 161 of file spi_driver.cpp.

References SPI_IOC_MESSAGE.

8.9.4 Member Data Documentation

8.9.4.1 k_crc_size

```
size_t star_spi_bridge::SpiDriver::k_crc_size = 4  [static], [constexpr]
```

CRC-32 trailer length in bytes.

Definition at line 65 of file spi_driver.hpp.

Referenced by decode_frame(), and encode_frame().

8.9.4.2 k_header_size

```
size_t star_spi_bridge::SpiDriver::k_header_size = 8  [static], [constexpr]
```

SYNC + SEQ + LEN + TYPE + FLAGS = 8 bytes.

Definition at line 62 of file spi_driver.hpp.

Referenced by decode_frame(), and encode_frame().

8.9.4.3 k_max_payload_size

```
size_t star_spi_bridge::SpiDriver::k_max_payload_size = 1024  [static], [constexpr]
```

Maximum application payload per frame (bytes).

Definition at line 68 of file spi_driver.hpp.

Referenced by encode_frame().

8.9.4.4 k_sync_word

```
uint16_t star_spi_bridge::SpiDriver::k_sync_word = 0x55AA  [static], [constexpr]
```

SYNC word transmitted in every frame header.

Definition at line 71 of file spi_driver.hpp.

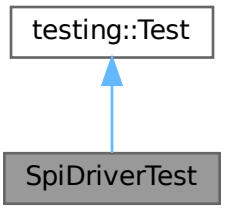
Referenced by decode_frame(), and encode_frame().

The documentation for this class was generated from the following files:

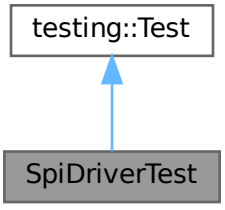
- src/star_spi_bridge/include/star_spi_bridge/spi_driver.hpp
- src/star_spi_bridge/src/spi_driver.cpp

8.10 SpiDriverTest Class Reference

Inheritance diagram for SpiDriverTest:



Collaboration diagram for SpiDriverTest:



Protected Member Functions

- void SetUp () override

8.10.1 Detailed Description

Definition at line 10 of file test_spi_driver.cpp.

8.10.2 Member Function Documentation

8.10.2.1 SetUp()

```
void SpiDriverTest::SetUp ()  [inline], [override], [protected]
```

Definition at line 13 of file test_spi_driver.cpp.

The documentation for this class was generated from the following file:

- src/star_spi_bridge/test/test_spi_driver.cpp

8.11 SpiMessageConverter Class Reference

```
#include <spi_message_converter.hpp>
```

Classes

- struct Parameters

Public Member Functions

- SpiMessageConverter (const Parameters &params)
- bool twist_to_velocity_command (const geometry_msgs::msg::Twist &twist, star::v1::VelocityCommand &command)
- void telemetry_to_odometry (const star::v1::TelemetryData &telemetry, nav_msgs::msg::Odometry &odom)
- void telemetry_to_joint_state (const star::v1::TelemetryData &telemetry, sensor_msgs::msg::JointState &joint_state)

8.11.1 Detailed Description

Definition at line 16 of file spi_message_converter.hpp.

8.11.2 Constructor & Destructor Documentation

8.11.2.1 SpiMessageConverter()

```
star_spi_bridge::SpiMessageConverter::SpiMessageConverter (
            const Parameters & params)  [explicit]
```

Definition at line 14 of file spi_message_converter.cpp.

8.11.3 Member Function Documentation

8.11.3.1 telemetry_to_joint_state()

```
void star_spi_bridge::SpiMessageConverter::telemetry_to_joint_state (
            const star::v1::TelemetryData & telemetry,
            sensor_msgs::msg::JointState & joint_state)
```

Definition at line 158 of file spi_message_converter.cpp.

8.11.3.2 telemetry_to_odometry()

```
void star_spi_bridge::SpiMessageConverter::telemetry_to_odometry (
            const star::v1::TelemetryData & telemetry,
            nav_msgs::msg::Odometry & odom)
```

Definition at line 49 of file spi_message_converter.cpp.

8.11.3.3 twist_to_velocity_command()

```
bool star_spi_bridge::SpiMessageConverter::twist_to_velocity_command (
            const geometry_msgs::msg::Twist & twist,
            star::v1::VelocityCommand & command)
```

Definition at line 17 of file spi_message_converter.cpp.

The documentation for this class was generated from the following files:

- src/star_spi_bridge/include/star_spi_bridge/spi_message_converter.hpp
- src/star_spi_bridge/src/spi_message_converter.cpp

8.12 star_spi_bridge::SpiMessageConverter Class Reference

```
#include <spi_message_converter.hpp>
```

Classes

- struct Parameters

Public Member Functions

- SpiMessageConverter (const Parameters &params)
- bool twist_to_velocity_command (const geometry_msgs::msg::Twist &twist, star::v1::VelocityCommand &command)
- void telemetry_to_odometry (const star::v1::TelemetryData &telemetry, nav_msgs::msg::Odometry &odom)
- void telemetry_to_joint_state (const star::v1::TelemetryData &telemetry, sensor_msgs::msg::JointState &joint_state)

8.12.1 Detailed Description

Definition at line 16 of file spi_message_converter.hpp.

8.12.2 Constructor & Destructor Documentation

8.12.2.1 SpiMessageConverter()

```
star_spi_bridge::SpiMessageConverter::SpiMessageConverter (
            const Parameters & params)  [explicit]
```

Definition at line 14 of file spi_message_converter.cpp.

8.12.3 Member Function Documentation

8.12.3.1 telemetry_to_joint_state()

```
void star_spi_bridge::SpiMessageConverter::telemetry_to_joint_state (
            const star::v1::TelemetryData & telemetry,
            sensor_msgs::msg::JointState & joint_state)
```

Definition at line 158 of file spi_message_converter.cpp.

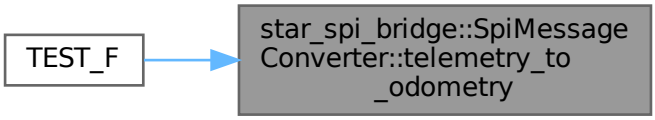
8.12.3.2 telemetry_to_odometry()

```
void star_spi_bridge::SpiMessageConverter::telemetry_to_odometry (
            const star::v1::TelemetryData & telemetry,
            nav_msgs::msg::Odometry & odom)
```

Definition at line 49 of file spi_message_converter.cpp.

Referenced by TEST_F().

Here is the caller graph for this function:



8.12.3.3 twist_to_velocity_command()

```
bool star_spi_bridge::SpiMessageConverter::twist_to_velocity_command (
            const geometry_msgs::msg::Twist & twist,
            star::v1::VelocityCommand & command)
```

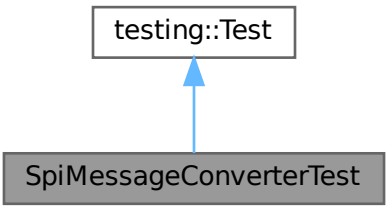
Definition at line 17 of file spi_message_converter.cpp.

The documentation for this class was generated from the following files:

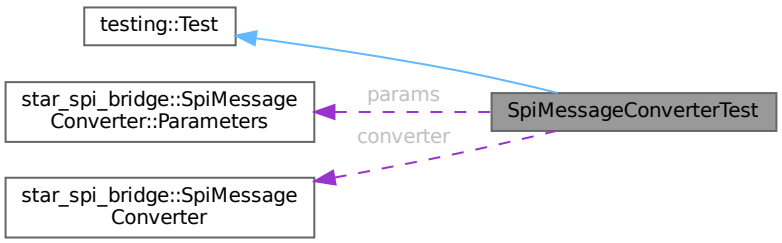
- src/star_spi_bridge/include/star_spi_bridge/spi_message_converter.hpp
- src/star_spi_bridge/src/spi_message_converter.cpp

8.13 SpiMessageConverterTest Class Reference

Inheritance diagram for SpiMessageConverterTest:



Collaboration diagram for SpiMessageConverterTest:



Protected Attributes

- SpiMessageConverter::Parameters params {0.150, 0.0325, 11599}
- SpiMessageConverter converter {params}

8.13.1 Detailed Description

Definition at line 11 of file test_spi_message_converter.cpp.

8.13.2 Member Data Documentation

8.13.2.1 converter

```
SpiMessageConverter SpiMessageConverterTest::converter {params}  [protected]
```

Definition at line 15 of file test_spi_message_converter.cpp.

8.13.2.2 params

```
SpiMessageConverter::Parameters SpiMessageConverterTest::params {0.150, 0.0325, 11599}  [protected]
```

Definition at line 14 of file test_spi_message_converter.cpp.

The documentation for this class was generated from the following file:

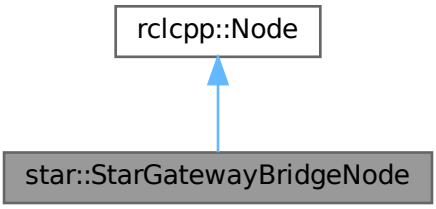
- src/star_spi_bridge/test/test_spi_message_converter.cpp

8.14 star::StarGatewayBridgeNode Class Reference

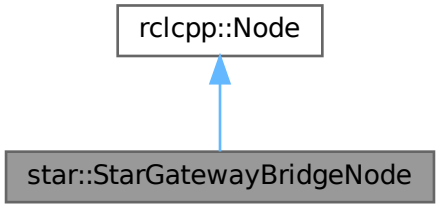
ROS2 node that bridges the ROS2 ecosystem with the Go gateway service via gRPC.

```
#include <star_gateway_bridge_node.hpp>
```

Inheritance diagram for star::StarGatewayBridgeNode:



Collaboration diagram for star::StarGatewayBridgeNode:



Public Member Functions

- StarGatewayBridgeNode (const rclcpp::NodeOptions &options=rclcpp::NodeOptions())

  Construct StarGatewayBridgeNode with ROS2 and gRPC initialization.
- ~StarGatewayBridgeNode ()

  Destructor - sends stop command and shuts down gracefully.

8.14.1 Detailed Description

ROS2 node that bridges the ROS2 ecosystem with the Go gateway service via gRPC.

Architecture: ROS2 Topics/Services <-> StarGatewayBridgeNode <-> gRPC <-> Go Gateway <-> UI

Responsibilities:

1. Subscribe to /robot_status ROS2 topic
2. Forward critical telemetry to Gateway via gRPC for UI display
3. Poll Gateway for teleop commands from UI
4. Publish teleop commands to /teleop/cmd_vel ROS2 topic
5. Expose /set_pid_gains ROS2 service for PID tuning from UI

Safety Features:

- Teleop command staleness check (500ms timeout -> zero velocity)
- Connection watchdog (5s interval, automatic reconnection)
- Non-blocking gRPC calls (100ms deadline)
- Input validation (NaN, infinity checks via MessageConverter)
- Graceful shutdown (stop command on node destruction)

gRPC Service Definition (to be implemented in Phase 4): service GatewayService { rpc ForwardTelemetry(↩ TelemetryRequest) returns (TelemetryResponse); rpc GetTeleopCommand(TeleopRequest) returns (Teleop↩ Response); rpc SetPIDGains(PIDGainsRequest) returns (PIDGainsResponse); }

Definition at line 63 of file star_gateway_bridge_node.hpp.

8.14.2 Constructor & Destructor Documentation

8.14.2.1 StarGatewayBridgeNode()

```
star::StarGatewayBridgeNode::StarGatewayBridgeNode (
            const rclcpp::NodeOptions & options = rclcpp::NodeOptions())  [explicit]
```

Construct StarGatewayBridgeNode with ROS2 and gRPC initialization.

Declares parameters:

- gateway_address: Gateway gRPC server address (default: "localhost:50051")
- telemetry_rate_hz: Telemetry forwarding rate (default: 10.0 Hz)
- teleop_rate_hz: Teleop polling rate (default: 50.0 Hz)
- watchdog_timeout_sec: Connection health check interval (default: 5.0s)
- teleop_timeout_ms: Teleop command staleness timeout (default: 500ms)
- grpc_deadline_ms: gRPC call deadline (default: 100ms)
- wheel_base: Distance between wheels in meters (default: 0.150m)

Parameters

options	ROS2 node options for component configuration

Definition at line 18 of file star_gateway_bridge_node.cpp.

8.14.2.2 ~StarGatewayBridgeNode()

```
star::StarGatewayBridgeNode::~StarGatewayBridgeNode ()
```

Destructor - sends stop command and shuts down gracefully.

Definition at line 66 of file star_gateway_bridge_node.cpp.

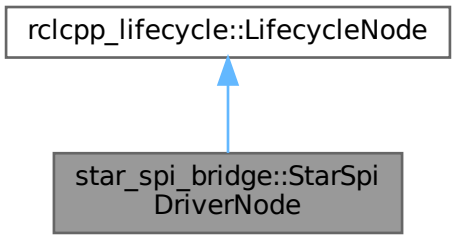
The documentation for this class was generated from the following files:

- src/star_gateway_bridge/include/star_gateway_bridge/star_gateway_bridge_node.hpp
- src/star_gateway_bridge/src/star_gateway_bridge_node.cpp

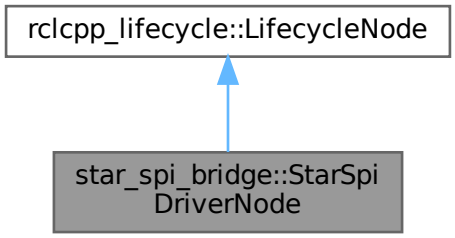
8.15 star_spi_bridge::StarSpiDriverNode Class Reference

`#include <star_spi_driver_node.hpp>`

Inheritance diagram for star_spi_bridge::StarSpiDriverNode:



Collaboration diagram for star_spi_bridge::StarSpiDriverNode:



Public Member Functions

- StarSpiDriverNode (const rclcpp::NodeOptions &options=rclcpp::NodeOptions())
- ~StarSpiDriverNode () override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn on_configure (const rclcpp_↩ lifecycle::State &prev_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn on_activate (const rclcpp_↩ lifecycle::State &prev_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn on_deactivate (const rclcpp_↩ lifecycle::State &prev_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn on_cleanup (const rclcpp_↩ lifecycle::State &prev_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn on_shutdown (const rclcpp_↩ lifecycle::State &prev_state) override

8.15.1 Detailed Description

Definition at line 22 of file star_spi_driver_node.hpp.

8.15.2 Constructor & Destructor Documentation

8.15.2.1 StarSpiDriverNode()

```
star_spi_bridge::StarSpiDriverNode::StarSpiDriverNode (
            const rclcpp::NodeOptions & options = rclcpp::NodeOptions())  [explicit]
```

Definition at line 21 of file star_spi_driver_node.cpp.

8.15.2.2 ~StarSpiDriverNode()

```
star_spi_bridge::StarSpiDriverNode::~StarSpiDriverNode ()  [override]
```

Definition at line 33 of file star_spi_driver_node.cpp.

8.15.3 Member Function Documentation

8.15.3.1 on_activate()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_spi_bridge::↩
StarSpiDriverNode::on_activate (
            const rclcpp_lifecycle::State & prev_state)  [override]
```

Definition at line 86 of file star_spi_driver_node.cpp.

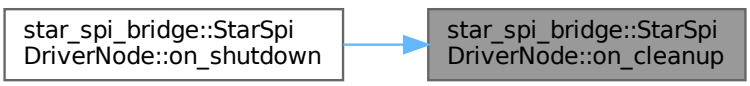
8.15.3.2 on_cleanup()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_spi_bridge::↩
StarSpiDriverNode::on_cleanup (
            const rclcpp_lifecycle::State & prev_state)  [override]
```

Definition at line 129 of file star_spi_driver_node.cpp.

Referenced by on_shutdown().

Here is the caller graph for this function:



8.15.3.3 on_configure()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_spi_bridge::↩
StarSpiDriverNode::on_configure (
            const rclcpp_lifecycle::State & prev_state)  [override]
```

Definition at line 42 of file star_spi_driver_node.cpp.

References star_spi_bridge::SpiMessageConverter::Parameters::ticks_per_rev, star_spi_bridge::SpiMessageConverter::Parameters:
and star_spi_bridge::SpiMessageConverter::Parameters::wheel_radius.

8.15.3.4 on_deactivate()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_spi_bridge::↩
StarSpiDriverNode::on_deactivate (
            const rclcpp_lifecycle::State & prev_state)  [override]
```

Definition at line 103 of file star_spi_driver_node.cpp.

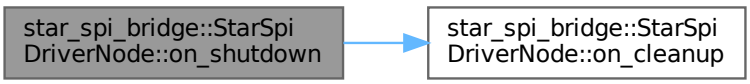
8.15.3.5 on_shutdown()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_spi_bridge::↩
StarSpiDriverNode::on_shutdown (
            const rclcpp_lifecycle::State & prev_state)  [override]
```

Definition at line 147 of file star_spi_driver_node.cpp.

References on_cleanup().

Here is the call graph for this function:



The documentation for this class was generated from the following files:

- src/star_spi_bridge/include/star_spi_bridge/star_spi_driver_node.hpp
- src/star_spi_bridge/src/star_spi_driver_node.cpp

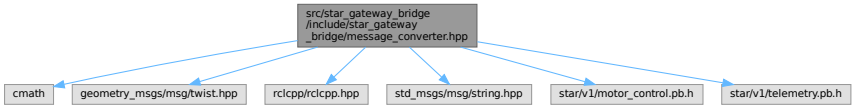
Chapter 9

File Documentation

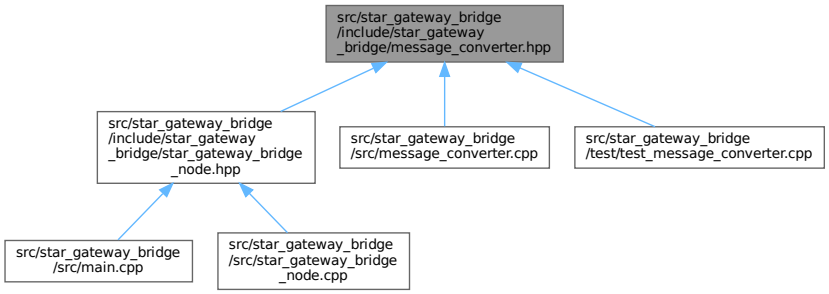
9.1 src/star_gateway_bridge/include/star_gateway_bridge/message_converter.hpp File Reference

```
#include <cmath>
#include <geometry_msgs/msg/twist.hpp>
#include <rclcpp/rclcpp.hpp>
#include <std_msgs/msg/string.hpp>
#include "star/v1/motor_control.pb.h"
#include "star/v1/telemetry.pb.h"
```

Include dependency graph for message_converter.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class star::MessageConverter

  Message converter for ROS2 <-> Protobuf translation.

Namespaces

- namespace star

9.2 message_converter.hpp

Go to the documentation of this file.

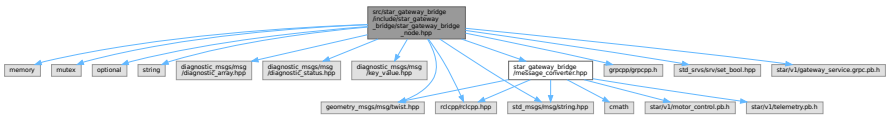
```
00001 // message_converter.hpp - ROS2 <-> Protobuf Message Converter
00002 // Bidirectional conversion between ROS2 standard messages and STAR Protocol
00003 // Buffers.
00004 //
00005 // STAR Project - Texas A&M University
00006 // January 2026
00007
00008 #ifndef STAR_GATEWAY_BRIDGE__MESSAGE_CONVERTER_HPP_
00009 #define STAR_GATEWAY_BRIDGE__MESSAGE_CONVERTER_HPP_
00010
00011 #include <cmath>
00012
00013 #include <geometry_msgs/msg/twist.hpp>
00014 #include <rclcpp/rclcpp.hpp>
00015 #include <std_msgs/msg/string.hpp>
00016
00017 #include "star/v1/motor_control.pb.h"
00018 #include "star/v1/telemetry.pb.h"
00019
00020 namespace star
00021 {
00022
00030 class MessageConverter {
00031 public:
00032   // ============================================================================
00033   // ROS2 -> Protobuf Conversions
00034   // ============================================================================
00035
00059   static bool twist_to_velocity_command(
00060     const geometry_msgs::msg::Twist & twist,
00061     star::v1::VelocityCommand & command,
00062     double wheel_base = 0.150,
00063     uint32_t sequence = 0);
00064
00075   static bool string_to_system_status(
00076     const std_msgs::msg::String & status_msg,
00077     star::v1::SystemStatus & system_status);
00078
00079   // ============================================================================
00080   // Protobuf -> ROS2 Conversions
00081   // ============================================================================
00082
00103   static bool
00104   velocity_command_to_twist(
00105     const star::v1::VelocityCommand & command,
00106     geometry_msgs::msg::Twist & twist,
00107     double wheel_base = 0.150);
00108
00121   static bool pid_config_to_gains(
00122     const star::v1::PidConfig & pid_config,
00123     double & kp, double & ki, double & kd);
00124
00125   // ============================================================================
00126   // Validation Helpers
00127   // ============================================================================
00128
00134   static bool is_valid_double(double value) {return std::isfinite(value);}
00135
00143   static double clamp(double value, double min, double max)
00144   {
00145     return std::max(min, std::min(max, value));
00146   }
00147
00153   static int64_t ros_time_to_us(const rclcpp::Time & time);
00154
00155 private:
00156   // Differential drive kinematics constants
00157   static constexpr double k_max_velocity_mps =
00158     2.0;   // VelocityCommand valid range
00159   static constexpr double k_max_angular_vel =
00160     4.0;   // Maximum angular velocity (rad/s)
00161
00162 };
```

```
00163
00164 } // namespace star
00165
00166 #endif // STAR_GATEWAY_BRIDGE__MESSAGE_CONVERTER_HPP_
```

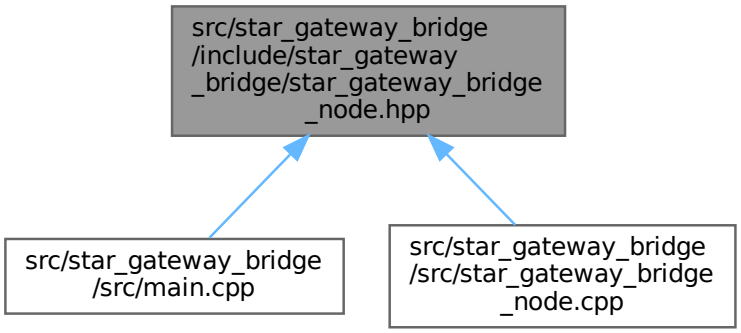
9.3 src/star_gateway_bridge/include/star_gateway_bridge/star_gateway_bridge_node.hpp File Reference

```
#include <memory>
#include <mutex>
#include <optional>
#include <string>
#include <diagnostic_msgs/msg/diagnostic_array.hpp>
#include <diagnostic_msgs/msg/diagnostic_status.hpp>
#include <diagnostic_msgs/msg/key_value.hpp>
#include <geometry_msgs/msg/twist.hpp>
#include <grpcpp/grpcpp.h>
#include <rclcpp/rclcpp.hpp>
#include <std_msgs/msg/string.hpp>
#include <std_srvs/srv/set_bool.hpp>
#include "star/v1/gateway_service.grpc.pb.h"
#include "star_gateway_bridge/message_converter.hpp"
```

Include dependency graph for star_gateway_bridge_node.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class star::StarGatewayBridgeNode

  ROS2 node that bridges the ROS2 ecosystem with the Go gateway service via gRPC.

Namespaces

- namespace star

9.4 star_gateway_bridge_node.hpp

Go to the documentation of this file.

```
00001 // star_gateway_bridge_node.hpp - ROS2 Gateway Bridge Node
00002 // Bridges ROS2 ecosystem with Go gateway service via gRPC.
00003 //
00004 // This node handles bidirectional communication:
00005 // - ROS2 -> Gateway: Forward telemetry (robot status) for UI display
00006 // - Gateway -> ROS2: Poll teleop commands and PID gain updates from UI
00007 //
00008 // STAR Project - Texas A&M University
00009 // Copyright 2026 STAR Project
00010 // January 2026
00011
00012 #ifndef STAR_GATEWAY_BRIDGE__STAR_GATEWAY_BRIDGE_NODE_HPP_
00013 #define STAR_GATEWAY_BRIDGE__STAR_GATEWAY_BRIDGE_NODE_HPP_
00014
00015 #include <memory>
00016 #include <mutex>
00017 #include <optional>
00018 #include <string>
00019
00020 #include <diagnostic_msgs/msg/diagnostic_array.hpp>
00021 #include <diagnostic_msgs/msg/diagnostic_status.hpp>
00022 #include <diagnostic_msgs/msg/key_value.hpp>
00023 #include <geometry_msgs/msg/twist.hpp>
00024 #include <grpcpp/grpcpp.h> // NOLINT(build/include_order)
00025 #include <rclcpp/rclcpp.hpp>
00026 #include <std_msgs/msg/string.hpp>
00027 #include <std_srvs/srv/set_bool.hpp>
00028
00029 #include "star/v1/gateway_service.grpc.pb.h"
00030 #include "star_gateway_bridge/message_converter.hpp"
00031
00032 namespace star
00033 {
00034
00063 class StarGatewayBridgeNode : public rclcpp::Node {
00064 public:
00079   explicit StarGatewayBridgeNode(
00080     const rclcpp::NodeOptions & options = rclcpp::NodeOptions());
00081
00085   ~StarGatewayBridgeNode();
00086
00087 private:
00088   // ============================================================================
00089   // Initialization
00090   // ============================================================================
00091
00100   bool initialize_grpc_client();
00101
00106   void initialize_ros_interfaces();
00107
00108   // ============================================================================
00109   // ROS2 Callbacks
00110   // ============================================================================
00111
00120   void robot_status_callback(const std_msgs::msg::String::SharedPtr msg);
00121
00131   void set_pid_gains_callback(
00132     const std::shared_ptr<std_srvs::srv::SetBool::Request> request,
00133     std::shared_ptr<std_srvs::srv::SetBool::Response> response);
00134
00135   // ============================================================================
00136   // Timer Callbacks
00137   // ============================================================================
00138
00145   void telemetry_forward_timer_callback();
00146
00154   void teleop_poll_timer_callback();
00155
00161   void connection_watchdog_callback();
00162
00163   // ============================================================================
00164   // gRPC Helpers
00165   // ============================================================================
```

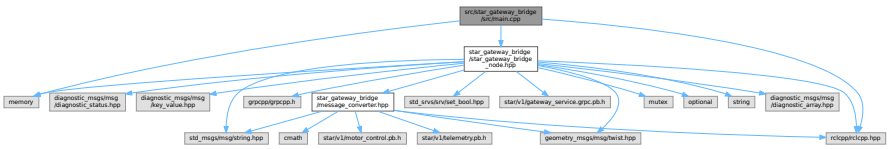
```
00166
00175   bool reconnect_grpc_client();
00176
00181   bool is_grpc_connected() const;
00182
00183   // ========================================================================
00184   // Member Variables
00185   // ========================================================================
00186
00187   // ROS2 Publishers
00188   rclcpp::Publisher<geometry_msgs::msg::Twist>::SharedPtr teleop_cmd_vel_pub_;
00189
00190   // ROS2 Subscribers
00191   rclcpp::Subscription<std_msgs::msg::String>::SharedPtr robot_status_sub_;
00192
00193   // ROS2 Services
00194   rclcpp::Service<std_srvs::srv::SetBool>::SharedPtr set_pid_gains_service_;
00195
00196   // ROS2 Timers
00197   rclcpp::TimerBase::SharedPtr telemetry_timer_;
00198   rclcpp::TimerBase::SharedPtr teleop_timer_;
00199   rclcpp::TimerBase::SharedPtr watchdog_timer_;
00200
00201   // gRPC Client
00202   std::shared_ptr<grpc::Channel> grpc_channel_;
00203   std::unique_ptr<star::v1::GatewayService::Stub> grpc_stub_;
00204
00205   // Cached telemetry data (protected by mutexes)
00206   std::mutex robot_status_mutex_;
00207   std::optional<std_msgs::msg::String> cached_robot_status_;
00208
00209
00210   // Parameters (cached for performance)
00211   std::string gateway_address_;
00212   double telemetry_rate_hz_;
00213   double teleop_rate_hz_;
00214   double watchdog_timeout_sec_;
00215   int teleop_timeout_ms_;
00216   int grpc_deadline_ms_;
00217   double wheel_base_;
00218
00219   // Connection state
00220   bool grpc_connected_;
00221   int reconnect_attempts_;
00222   static constexpr int k_max_reconnect_attempts = 10;
00223   static constexpr int k_reconnect_backoff_ms_base = 100;
00224
00225   // Message converter (stateless utility)
00226   MessageConverter converter_;
00227
00228   // Frame drop detection for teleop commands and telemetry
00229   rclcpp::Publisher<diagnostic_msgs::msg::DiagnosticArray>::SharedPtr
00230     diagnostics_pub_;
00231   rclcpp::TimerBase::SharedPtr diagnostics_timer_;
00232
00233   uint32_t last_teleop_sequence_{0};
00234   uint64_t total_teleop_frames_{0};
00235   uint64_t teleop_frames_dropped_{0};
00236   bool first_teleop_frame_{true};
00237
00238   uint32_t last_telemetry_sequence_{0};
00239   uint64_t total_telemetry_frames_{0};
00240   uint64_t telemetry_frames_dropped_{0};
00241   bool first_telemetry_frame_{true};
00242
00243   // Methods
00244   void publish_diagnostics();
00245   void check_teleop_sequence_continuity(uint32_t current_sequence);
00246   void check_telemetry_sequence_continuity(uint32_t current_sequence);
00247 };
00248
00249 } // namespace star
00250
00251 #endif // STAR_GATEWAY_BRIDGE__STAR_GATEWAY_BRIDGE_NODE_HPP_
```

9.5 src/star_gateway_bridge/src/main.cpp File Reference

```
#include <memory>
#include "rclcpp/rclcpp.hpp"
```

```
#include "star_gateway_bridge/star_gateway_bridge_node.hpp"
```

Include dependency graph for main.cpp:



Functions

- int main (int argc, char **argv)

  Main entry point for star_gateway_bridge standalone executable.

9.5.1 Function Documentation

9.5.1.1 main()

```
int main (
            int argc,
            char ** argv)
```

Main entry point for star_gateway_bridge standalone executable.

Usage: ros2 run star_gateway_bridge star_gateway_bridge_main

Parameters can be set via command line: ros2 run star_gateway_bridge star_gateway_bridge_main \ –ros-args -p gateway_address:=192.168.1.100:50051 \ -p telemetry_rate_hz:=20.0

Definition at line 23 of file main.cpp.

9.6 main.cpp

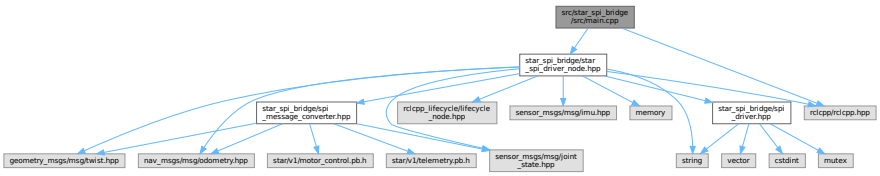
Go to the documentation of this file.

```
00001 // main.cpp - STAR Gateway Bridge Standalone Executable
00002 // Entry point for running star_gateway_bridge node as standalone process.
00003 //
00004 // STAR Project - Texas A&M University
00005 // January 2026
00006
00007 #include <memory>
00008
00009 #include "rclcpp/rclcpp.hpp"
00010 #include "star_gateway_bridge/star_gateway_bridge_node.hpp"
00011
00023 int main(int argc, char **argv)
00024 {
00025   // Initialize ROS2
00026   rclcpp::init(argc, argv);
00027
00028   // Create and spin node
00029   auto node = std::make_shared<star::StarGatewayBridgeNode>();
00030
00031   RCLCPP_INFO(node->get_logger(), "STAR Gateway Bridge node running");
00032   RCLCPP_INFO(node->get_logger(), "Press Ctrl+C to exit");
00033
00034   // Spin until shutdown
00035   rclcpp::spin(node);
00036
00037   // Cleanup
00038   rclcpp::shutdown();
00039
00040   RCLCPP_INFO(node->get_logger(), "STAR Gateway Bridge node exited cleanly");
00041
00042   return 0;
00043 }
```

9.7 src/star_spi_bridge/src/main.cpp File Reference

```
#include "star_spi_bridge/star_spi_driver_node.hpp"
#include <rclcpp/rclcpp.hpp>
```

Include dependency graph for main.cpp:



Functions

- int main (int argc, char **argv)

9.7.1 Function Documentation

9.7.1.1 main()

```
int main (
            int argc,
            char ** argv)
```

Definition at line 7 of file main.cpp.

9.8 main.cpp

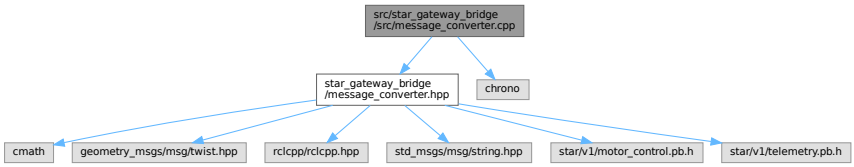
Go to the documentation of this file.

```
00001 // Copyright 2026 Locked Inc.
00002
00003 #include "star_spi_bridge/star_spi_driver_node.hpp"
00004
00005 #include <rclcpp/rclcpp.hpp>
00006
00007 int main(int argc, char **argv)
00008 {
00009   rclcpp::init(argc, argv);
00010
00011   rclcpp::executors::SingleThreadedExecutor executor;
00012   auto node = std::make_shared<star_spi_bridge::StarSpiDriverNode>();
00013
00014   executor.add_node(node->get_node_base_interface());
00015   executor.spin();
00016
00017   rclcpp::shutdown();
00018   return 0;
00019 }
```

9.9 src/star_gateway_bridge/src/message_converter.cpp File Reference

```
#include "star_gateway_bridge/message_converter.hpp"
#include <chrono>
```

Include dependency graph for message_converter.cpp:



Namespaces

- namespace star

9.10 message_converter.cpp

Go to the documentation of this file.

```
00001 // message_converter.cpp - ROS2 <-> Protobuf Message Converter Implementation
00002 // Bidirectional conversion between ROS2 standard messages and STAR Protocol
00003 // Buffers.
00004 //
00005 // STAR Project - Texas A&M University
00006 // Copyright 2026 STAR Project
00007 // January 2026
00008
00009 #include "star_gateway_bridge/message_converter.hpp"
00010
00011 #include <chrono>
00012 namespace star
00013 {
00014
00015 // ============================================================================
00016 // ROS2 -> Protobuf Conversions
00017 // ============================================================================
00018
00019 bool MessageConverter::twist_to_velocity_command(
00020   const geometry_msgs::msg::Twist & twist, star::v1::VelocityCommand & command,
00021   double wheel_base, uint32_t sequence)
00022 {
00023   // Validate inputs for NaN/infinity
00024   if (!is_valid_double(twist.linear.x) || !is_valid_double(twist.angular.z)) {
00025     RCLCPP_WARN(rclcpp::get_logger("message_converter"),
00026                 "Invalid Twist: NaN/infinity in linear.x or angular.z");
00027     return false;
00028   }
00029
00030   if (!is_valid_double(wheel_base) || wheel_base <= 0.0) {
00031     RCLCPP_ERROR(rclcpp::get_logger("message_converter"),
00032                  "Invalid wheel_base: must be positive and finite");
00033     return false;
00034   }
00035
00036   // Clamp input velocities to safe ranges
00037   double linear =
00038     clamp(twist.linear.x, -k_max_velocity_mps, k_max_velocity_mps);
00039   double angular =
00040     clamp(twist.angular.z, -k_max_angular_vel, k_max_angular_vel);
00041
00042   // Differential drive kinematics: (linear, angular) -> (left, right)
00043   // left_vel = linear - (angular * wheel_base / 2)
00044   // right_vel = linear + (angular * wheel_base / 2)
00045   double half_base = wheel_base / 2.0;
00046   double left_velocity = linear - (angular * half_base);
```

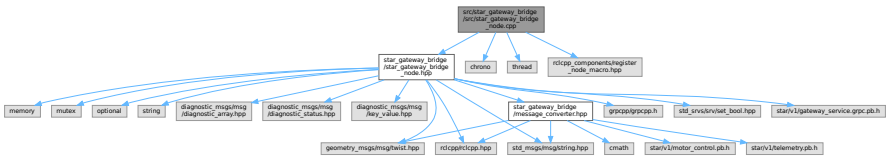
```
00047   double right_velocity = linear + (angular * half_base);
00048
00049   // Clamp wheel velocities to VelocityCommand valid range [-2.0, 2.0] m/s
00050   left_velocity = clamp(left_velocity, -k_max_velocity_mps, k_max_velocity_mps);
00051   right_velocity =
00052     clamp(right_velocity, -k_max_velocity_mps, k_max_velocity_mps);
00053
00054   // Populate protobuf message
00055   // Note: front_left/back_left = left side, front_right/back_right = right side
00056   // for differential drive
00057   command.set_front_left_velocity_mps(left_velocity);
00058   command.set_back_left_velocity_mps(left_velocity);
00059   command.set_front_right_velocity_mps(right_velocity);
00060   command.set_back_right_velocity_mps(right_velocity);
00061   command.set_sequence(sequence);
00062
00063   // Timestamp in microseconds since epoch
00064   auto now = std::chrono::system_clock::now();
00065   auto us = std::chrono::duration_cast<std::chrono::microseconds>(
00066                 now.time_since_epoch())
00067     .count();
00068   command.set_timestamp_us(us);
00069
00070   return true;
00071 }
00072
00073 bool MessageConverter::string_to_system_status(
00074   const std_msgs::msg::String & status_msg,
00075   star::v1::SystemStatus & system_status)
00076 {
00077   // For now, implement basic string parsing
00078   // In production, use a JSON library (e.g., nlohmann/json) for robust parsing
00079   // This is a placeholder implementation
00080
00081   const std::string & data = status_msg.data;
00082
00083   // Simple keyword-based parsing (not robust, but sufficient for MVP)
00084   if (data.find("MANUAL") != std::string::npos) {
00085     system_status.set_mode(star::v1::ROBOT_MODE_MANUAL);
00086   } else if (data.find("AUTONOMOUS") != std::string::npos) {
00087     system_status.set_mode(star::v1::ROBOT_MODE_AUTONOMOUS);
00088   } else if (data.find("MAPPING") != std::string::npos) {
00089     system_status.set_mode(star::v1::ROBOT_MODE_MAPPING);
00090   } else if (data.find("EMERGENCY_STOP") != std::string::npos) {
00091     system_status.set_mode(star::v1::ROBOT_MODE_EMERGENCY_STOP);
00092   } else {
00093     system_status.set_mode(star::v1::ROBOT_MODE_IDLE);
00094   }
00095
00096   // Connection status (default to CONNECTED if we're receiving messages)
00097   system_status.set_connection_status(star::v1::CONNECTION_STATUS_CONNECTED);
00098
00099   // TODO(star): Parse additional fields from JSON when available
00100   // For now, assume all subsystems are connected
00101   system_status.set_rx72n_connected(true);
00102   system_status.set_lidar_connected(true);
00103   system_status.set_ros_connected(true);
00104
00105   return true;
00106 }
00107
00108 // ============================================================================
00109 // Protobuf -> ROS2 Conversions
00110 // ============================================================================
00111
00112 bool MessageConverter::velocity_command_to_twist(
00113   const star::v1::VelocityCommand & command, geometry_msgs::msg::Twist & twist,
00114   double wheel_base)
00115 {
00116   // Validate protobuf inputs
00117   // Note: front_left/back_left = left side, front_right/back_right = right side
00118   // for differential drive Average left/right side velocities for differential
00119   // drive
00120   double left_vel =
00121     (command.front_left_velocity_mps() + command.back_left_velocity_mps()) /
00122     2.0;
00123   double right_vel =
00124     (command.front_right_velocity_mps() + command.back_right_velocity_mps()) /
00125     2.0;
00126
00127   if (!is_valid_double(left_vel) || !is_valid_double(right_vel)) {
00128     RCLCPP_WARN(rclcpp::get_logger("message_converter"),
00129                 "Invalid VelocityCommand: NaN/infinity in wheel velocities");
00130     return false;
00131   }
00132
00133   if (!is_valid_double(wheel_base) || wheel_base <= 0.0) {
```

```
00134     RCLCPP_ERROR(rclcpp::get_logger("message_converter"),
00135                  "Invalid wheel_base: must be positive and finite");
00136     return false;
00137   }
00138
00139   // Differential drive inverse kinematics: (left, right) -> (linear, angular)
00140   // linear = (left + right) / 2
00141   // angular = (right - left) / wheel_base
00142   double linear_x = (left_vel + right_vel) / 2.0;
00143   double angular_z = (right_vel - left_vel) / wheel_base;
00144
00145   // Populate ROS2 Twist message
00146   twist.linear.x = linear_x;
00147   twist.linear.y = 0.0;
00148   twist.linear.z = 0.0;
00149
00150   twist.angular.x = 0.0;
00151   twist.angular.y = 0.0;
00152   twist.angular.z = angular_z;
00153
00154   return true;
00155 }
00156
00157 bool MessageConverter::pid_config_to_gains(
00158   const star::v1::PidConfig & pid_config, double & kp, double & ki, double & kd)
00159 {
00160   // Extract gains from protobuf (no unit conversion needed)
00161   kp = pid_config.kp();
00162   ki = pid_config.ki();
00163   kd = pid_config.kd();
00164
00165   // Validate gains are finite
00166   if (!is_valid_double(kp) || !is_valid_double(ki) || !is_valid_double(kd)) {
00167     RCLCPP_WARN(rclcpp::get_logger("message_converter"),
00168                 "Invalid PidConfig: NaN/infinity in gains");
00169     return false;
00170   }
00171
00172   return true;
00173 }
00174
00175 // ================================================================================
00176 // Utility Functions
00177 // ================================================================================
00178
00179 int64_t MessageConverter::ros_time_to_us(const rclcpp::Time & time)
00180 {
00181   return time.nanoseconds() / 1000;
00182 }
00183
00184 } // namespace star
```

9.11 src/star_gateway_bridge/src/star_gateway_bridge_node.cpp File Reference

```
#include "star_gateway_bridge/star_gateway_bridge_node.hpp"
#include <chrono>
#include <thread>
#include "rclcpp_components/register_node_macro.hpp"
```

Include dependency graph for star_gateway_bridge_node.cpp:



Namespaces

- namespace star

9.12 star_gateway_bridge_node.cpp

Go to the documentation of this file.

```
00001 // star_gateway_bridge_node.cpp - ROS2 Gateway Bridge Node Implementation
00002 // Bridges ROS2 ecosystem with Go gateway service via gRPC.
00003 //
00004 // STAR Project - Texas A&M University
00005 // Copyright 2026 STAR Project
00006 // January 2026
00007
00008 #include "star_gateway_bridge/star_gateway_bridge_node.hpp"
00009
00010 #include <chrono>
00011 #include <thread>
00012
00013 using namespace std::chrono_literals;
00014
00015 namespace star
00016 {
00017
00018 StarGatewayBridgeNode::StarGatewayBridgeNode(const rclcpp::NodeOptions & options)
00019 : Node("star_gateway_bridge", options), grpc_connected_(false),
00020   reconnect_attempts_(0)
00021 {
00022   RCLCPP_INFO(this->get_logger(), "Initializing STAR Gateway Bridge Node");
00023
00024   // Declare parameters with defaults
00025   this->declare_parameter("gateway_address", "localhost:50051");
00026   this->declare_parameter("telemetry_rate_hz", 10.0);
00027   this->declare_parameter("teleop_rate_hz", 50.0);
00028   this->declare_parameter("watchdog_timeout_sec", 5.0);
00029   this->declare_parameter("teleop_timeout_ms", 500);
00030   this->declare_parameter("grpc_deadline_ms", 100);
00031   this->declare_parameter("wheel_base", 0.150); // 150mm track width
00032
00033   // Cache parameters
00034   gateway_address_ = this->get_parameter("gateway_address").as_string();
00035   telemetry_rate_hz_ = this->get_parameter("telemetry_rate_hz").as_double();
00036   teleop_rate_hz_ = this->get_parameter("teleop_rate_hz").as_double();
00037   watchdog_timeout_sec_ =
00038     this->get_parameter("watchdog_timeout_sec").as_double();
00039   teleop_timeout_ms_ = this->get_parameter("teleop_timeout_ms").as_int();
00040   grpc_deadline_ms_ = this->get_parameter("grpc_deadline_ms").as_int();
00041   wheel_base_ = this->get_parameter("wheel_base").as_double();
00042
00043   RCLCPP_INFO(
00044       this->get_logger(),
00045       "Configuration: gateway=%s, telemetry_rate=%.1fHz, teleop_rate=%.1fHz, "
00046       "watchdog=%.1fs, teleop_timeout=%dms, grpc_deadline=%dms, "
00047       "wheel_base=%.3fm",
00048       gateway_address_.c_str(), telemetry_rate_hz_, teleop_rate_hz_,
00049       watchdog_timeout_sec_, teleop_timeout_ms_, grpc_deadline_ms_,
00050       wheel_base_);
00051
00052   // Initialize gRPC client
00053   if (!initialize_grpc_client()) {
00054     RCLCPP_WARN(this->get_logger(),
00055                 "Failed to connect to Gateway at %s - will retry in background",
00056                 gateway_address_.c_str());
00057   }
00058
00059   // Initialize ROS2 interfaces
00060   initialize_ros_interfaces();
00061
00062   RCLCPP_INFO(this->get_logger(),
00063               "STAR Gateway Bridge Node initialized successfully");
00064 }
00065
00066 StarGatewayBridgeNode::~StarGatewayBridgeNode()
00067 {
00068   RCLCPP_INFO(this->get_logger(), "Shutting down STAR Gateway Bridge Node");
00069
00070   // Send stop command before shutdown
00071   auto zero_twist = geometry_msgs::msg::Twist();
00072   if (teleop_cmd_vel_pub_) {
00073     teleop_cmd_vel_pub_->publish(zero_twist);
00074     RCLCPP_INFO(this->get_logger(), "Sent stop command on shutdown");
00075   }
00076
00077   // Close gRPC channel gracefully
00078   if (grpc_channel_) {
00079     RCLCPP_INFO(this->get_logger(), "Closing gRPC channel");
00080   }
00081 }
00082
```

```
00083 // ============================================================================
00084 // Initialization
00085 // ============================================================================
00086
00087 bool StarGatewayBridgeNode::initialize_grpc_client()
00088 {
00089   RCLCPP_INFO(this->get_logger(), "Connecting to Gateway gRPC server at %s",
00090               gateway_address_.c_str());
00091
00092   // Create gRPC channel with keepalive settings
00093   grpc::ChannelArguments args;
00094   args.SetInt(GRPC_ARG_KEEPALIVE_TIME_MS, 10000);   // 10s keepalive ping
00095   args.SetInt(GRPC_ARG_KEEPALIVE_TIMEOUT_MS, 5000); // 5s keepalive timeout
00096   args.SetInt(GRPC_ARG_KEEPALIVE_PERMIT_WITHOUT_CALLS,
00097               1); // Allow keepalive without calls
00098
00099   grpc_channel_ = grpc::CreateCustomChannel(
00100       gateway_address_, grpc::InsecureChannelCredentials(), args);
00101
00102   if (!grpc_channel_) {
00103     RCLCPP_ERROR(this->get_logger(), "Failed to create gRPC channel");
00104     return false;
00105   }
00106
00107   // Wait for channel to be ready (with timeout)
00108   auto deadline = std::chrono::system_clock::now() +
00109     std::chrono::milliseconds(grpc_deadline_ms_);
00110
00111   if (!grpc_channel_->WaitForConnected(deadline)) {
00112     RCLCPP_WARN(this->get_logger(), "gRPC channel not ready within %dms",
00113                 grpc_deadline_ms_);
00114     grpc_connected_ = false;
00115     return false;
00116   }
00117
00118   // Create gRPC stub
00119   grpc_stub_ = star::v1::GatewayService::NewStub(grpc_channel_);
00120
00121   RCLCPP_INFO(this->get_logger(),
00122               "Successfully connected to Gateway gRPC server");
00123   grpc_connected_ = true;
00124   reconnect_attempts_ = 0;
00125
00126   return true;
00127 }
00128
00129 void StarGatewayBridgeNode::initialize_ros_interfaces()
00130 {
00131   RCLCPP_INFO(this->get_logger(), "Initializing ROS2 interfaces");
00132
00133   // Publishers
00134   teleop_cmd_vel_pub_ =
00135     this->create_publisher<geometry_msgs::msg::Twist>("/teleop/cmd_vel", 10);
00136
00137   // Subscribers
00138   robot_status_sub_ = this->create_subscription<std_msgs::msg::String>(
00139       "/robot_status", 10,
00140       std::bind(&StarGatewayBridgeNode::robot_status_callback, this,
00141                 std::placeholders::_1));
00142
00143   // Services
00144   set_pid_gains_service_ = this->create_service<std_srvs::srv::SetBool>(
00145       "/set_pid_gains",
00146       std::bind(&StarGatewayBridgeNode::set_pid_gains_callback, this,
00147                 std::placeholders::_1, std::placeholders::_2));
00148
00149   // Timers
00150   auto telemetry_period_ms = static_cast<int>(1000.0 / telemetry_rate_hz_);
00151   telemetry_timer_ = this->create_wall_timer(
00152       std::chrono::milliseconds(telemetry_period_ms),
00153       std::bind(&StarGatewayBridgeNode::telemetry_forward_timer_callback,
00154                 this));
00155
00156   auto teleop_period_ms = static_cast<int>(1000.0 / teleop_rate_hz_);
00157   teleop_timer_ = this->create_wall_timer(
00158       std::chrono::milliseconds(teleop_period_ms),
00159       std::bind(&StarGatewayBridgeNode::teleop_poll_timer_callback, this));
00160
00161   auto watchdog_period_ms = static_cast<int>(watchdog_timeout_sec_ * 1000.0);
00162   watchdog_timer_ = this->create_wall_timer(
00163       std::chrono::milliseconds(watchdog_period_ms),
00164       std::bind(&StarGatewayBridgeNode::connection_watchdog_callback, this));
00165
00166   // Create diagnostics publisher
00167   diagnostics_pub_ =
00168     this->create_publisher<diagnostic_msgs::msg::DiagnosticArray>(
00169           "/diagnostics", 10);
```

```
00170
00171   // Create diagnostics timer (1 Hz for human readability)
00172   diagnostics_timer_ = this->create_wall_timer(
00173       std::chrono::seconds(1),
00174       std::bind(&StarGatewayBridgeNode::publish_diagnostics, this));
00175
00176   RCLCPP_INFO(
00177       this->get_logger(),
00178       "ROS2 interfaces initialized: telemetry=%dms, teleop=%dms, watchdog=%dms",
00179       telemetry_period_ms, teleop_period_ms, watchdog_period_ms);
00180   RCLCPP_INFO(this->get_logger(), "Diagnostics publisher initialized");
00181 }
00182
00183 // ============================================================================
00184 // ROS2 Callbacks
00185 // ============================================================================
00186
00187 void StarGatewayBridgeNode::robot_status_callback(
00188   const std_msgs::msg::String::SharedPtr msg)
00189 {
00190   // Use try_lock to avoid blocking callback (non-blocking pattern)
00191   if (robot_status_mutex_.try_lock()) {
00192     cached_robot_status_ = *msg;
00193     robot_status_mutex_.unlock();
00194   }
00195   // If try_lock fails, skip this update (cache will use previous value)
00196 }
00197
00198 void StarGatewayBridgeNode::set_pid_gains_callback(
00199   const std::shared_ptr<std_srvs::srv::SetBool::Request> request,
00200   std::shared_ptr<std_srvs::srv::SetBool::Response> response)
00201 {
00202   // TODO(star): Phase 4: Implement PID gains service after defining custom
00203   // service type. For now, placeholder implementation.
00204   (void)request; // Unused parameter (placeholder service)
00205
00206   RCLCPP_INFO(this->get_logger(), "set_pid_gains service called (placeholder)");
00207
00208   // TODO(star): Parse PID gains from request, forward to Gateway via gRPC
00209   // TODO(star): Gateway will forward to motor controller via SPI bridge
00210
00211   response->success = false;
00212   response->message = "PID gains service not yet implemented (Phase 4)";
00213 }
00214
00215 // ============================================================================
00216 // Timer Callbacks
00217 // ============================================================================
00218
00219 void StarGatewayBridgeNode::telemetry_forward_timer_callback()
00220 {
00221   if (!grpc_connected_) {
00222     RCLCPP_DEBUG_THROTTLE(this->get_logger(), *this->get_clock(), 5000,
00223                           "Skipping telemetry forward - gRPC not connected");
00224     return;
00225   }
00226
00227   // Get cached telemetry (non-blocking)
00228   std::optional<std_msgs::msg::String> robot_status;
00229
00230   if (robot_status_mutex_.try_lock()) {
00231     robot_status = cached_robot_status_;
00232     robot_status_mutex_.unlock();
00233   }
00234
00235   // Forward telemetry to Gateway via gRPC
00236   if (grpc_stub_ && robot_status.has_value()) {
00237     grpc::ClientContext context;
00238     context.set_deadline(std::chrono::system_clock::now() +
00239                          std::chrono::milliseconds(grpc_deadline_ms_));
00240
00241     star::v1::ForwardTelemetryRequest request;
00242
00243     // Set request header
00244     auto *header = request.mutable_header();
00245     header->set_request_id(
00246         "telemetry_" +
00247         std::to_string(
00248             std::chrono::system_clock::now().time_since_epoch().count()));
00249
00250     if (robot_status.has_value()) {
00251       converter_.string_to_system_status(*robot_status,
00252                                          *request.mutable_system_status());
00253     }
00254
00255     star::v1::ForwardTelemetryResponse response;
00256     grpc::Status status =
```

```
00257       grpc_stub_->ForwardTelemetry(&context, request, &response);
00258
00259     if (!status.ok()) {
00260       RCLCPP_WARN_THROTTLE(this->get_logger(), *this->get_clock(), 5000,
00261                            "ForwardTelemetry gRPC failed: %s",
00262                            status.error_message().c_str());
00263       grpc_connected_ = false;
00264     } else {
00265       // Successful transmission - increment frame counter
00266       total_telemetry_frames_++;
00267     }
00268   }
00269
00270   RCLCPP_DEBUG_THROTTLE(this->get_logger(), *this->get_clock(), 10000,
00271                         "Telemetry forward: robot_status=%s",
00272                         robot_status.has_value() ? "cached" : "none");
00273 }
00274
00275 void StarGatewayBridgeNode::teleop_poll_timer_callback()
00276 {
00277   if (!grpc_connected_ || !grpc_stub_) {
00278     // Publish zero velocity when not connected (safety feature)
00279     auto zero_twist = geometry_msgs::msg::Twist();
00280     teleop_cmd_vel_pub_->publish(zero_twist);
00281     return;
00282   }
00283
00284   // Poll Gateway for latest teleop command via gRPC
00285   grpc::ClientContext context;
00286   context.set_deadline(std::chrono::system_clock::now() +
00287                        std::chrono::milliseconds(grpc_deadline_ms_));
00288
00289   star::v1::GetTeleopCommandRequest request;
00290
00291   // Set request header
00292   auto *header = request.mutable_header();
00293   header->set_request_id(
00294       "teleop_" +
00295       std::to_string(
00296           std::chrono::system_clock::now().time_since_epoch().count()));
00297
00298   star::v1::GetTeleopCommandResponse response;
00299
00300   grpc::Status status =
00301     grpc_stub_->GetTeleopCommand(&context, request, &response);
00302
00303   if (!status.ok()) {
00304     RCLCPP_WARN_THROTTLE(this->get_logger(), *this->get_clock(), 5000,
00305                          "GetTeleopCommand gRPC failed: %s",
00306                          status.error_message().c_str());
00307     grpc_connected_ = false;
00308     auto zero_twist = geometry_msgs::msg::Twist();
00309     teleop_cmd_vel_pub_->publish(zero_twist);
00310     return;
00311   }
00312
00313   // Check if command is available
00314   if (!response.command_available()) {
00315     RCLCPP_DEBUG_THROTTLE(this->get_logger(), *this->get_clock(), 10000,
00316                           "No teleop command available from Gateway");
00317     auto zero_twist = geometry_msgs::msg::Twist();
00318     teleop_cmd_vel_pub_->publish(zero_twist);
00319     return;
00320   }
00321
00322   // Check command staleness (safety feature)
00323   auto now_us = std::chrono::duration_cast<std::chrono::microseconds>(
00324                     std::chrono::system_clock::now().time_since_epoch())
00325     .count();
00326   auto cmd_age_us = now_us - response.command().timestamp_us();
00327   auto cmd_age_ms = cmd_age_us / 1000;
00328
00329   if (cmd_age_ms > teleop_timeout_ms_) {
00330     RCLCPP_WARN_THROTTLE(
00331         this->get_logger(), *this->get_clock(), 1000,
00332         "Teleop command stale (%ldms > %dms) - sending zero velocity",
00333         cmd_age_ms, teleop_timeout_ms_);
00334     auto zero_twist = geometry_msgs::msg::Twist();
00335     teleop_cmd_vel_pub_->publish(zero_twist);
00336     return;
00337   }
00338
00339   // Convert and publish fresh command
00340   geometry_msgs::msg::Twist twist;
00341   if (converter_.velocity_command_to_twist(response.command(), twist,
00342                                            wheel_base_))
00343   {
```

```
00344     // Check sequence continuity for frame drop detection
00345     uint32_t current_seq = response.command().sequence();
00346     check_teleop_sequence_continuity(current_seq);
00347
00348     teleop_cmd_vel_pub_->publish(twist);
00349   } else {
00350     RCLCPP_WARN(this->get_logger(),
00351                 "Failed to convert VelocityCommand to Twist");
00352     auto zero_twist = geometry_msgs::msg::Twist();
00353     teleop_cmd_vel_pub_->publish(zero_twist);
00354   }
00355 }
00356
00357 void StarGatewayBridgeNode::connection_watchdog_callback()
00358 {
00359   if (!is_grpc_connected()) {
00360     RCLCPP_WARN_THROTTLE(
00361         this->get_logger(), *this->get_clock(), 10000,
00362         "Gateway gRPC connection unhealthy - attempting reconnection");
00363     grpc_connected_ = false;
00364     reconnect_grpc_client();
00365   }
00366 }
00367
00368 // ============================================================================
00369 // gRPC Helpers
00370 // ============================================================================
00371
00372 bool StarGatewayBridgeNode::reconnect_grpc_client()
00373 {
00374   if (reconnect_attempts_ >= k_max_reconnect_attempts) {
00375     RCLCPP_ERROR_THROTTLE(this->get_logger(), *this->get_clock(), 30000,
00376                           "Max reconnection attempts (%d) reached - giving up",
00377                           k_max_reconnect_attempts);
00378     return false;
00379   }
00380
00381   reconnect_attempts_++;
00382
00383   // Exponential backoff
00384   int backoff_ms =
00385     k_reconnect_backoff_ms_base * (1 << std::min(reconnect_attempts_, 5));
00386
00387   RCLCPP_INFO(this->get_logger(), "Reconnection attempt %d/%d (backoff: %dms)",
00388               reconnect_attempts_, k_max_reconnect_attempts, backoff_ms);
00389
00390   std::this_thread::sleep_for(std::chrono::milliseconds(backoff_ms));
00391
00392   return initialize_grpc_client();
00393 }
00394
00395 bool StarGatewayBridgeNode::is_grpc_connected() const
00396 {
00397   if (!grpc_channel_) {
00398     return false;
00399   }
00400
00401   // false = don't try to connect
00402   auto state = grpc_channel_->GetState(false);
00403   return state == GRPC_CHANNEL_READY;
00404 }
00405
00406 void StarGatewayBridgeNode::check_teleop_sequence_continuity(
00407   uint32_t current_sequence)
00408 {
00409   if (first_teleop_frame_) {
00410     last_teleop_sequence_ = current_sequence;
00411     first_teleop_frame_ = false;
00412     RCLCPP_INFO(this->get_logger(), "First teleop frame received, sequence=%u",
00413                 current_sequence);
00414   } else {
00415     uint32_t expected_seq = last_teleop_sequence_ + 1;
00416
00417     // Detect sequence gap (accounting for uint32 wraparound)
00418     if (current_sequence != expected_seq) {
00419       uint32_t gap = current_sequence - expected_seq;
00420
00421       // Sanity check: ignore huge gaps (likely system reset or wraparound)
00422       if (gap < 10000) {
00423         teleop_frames_dropped_ += gap;
00424         RCLCPP_WARN(this->get_logger(),
00425                     "Teleop frame drop: expected seq %u, got %u (gap=%u, "
00426                     "total_dropped=%lu)",
00427                     expected_seq, current_sequence, gap,
00428                     teleop_frames_dropped_);
00429       } else {
00430         RCLCPP_WARN(this->get_logger(),
```

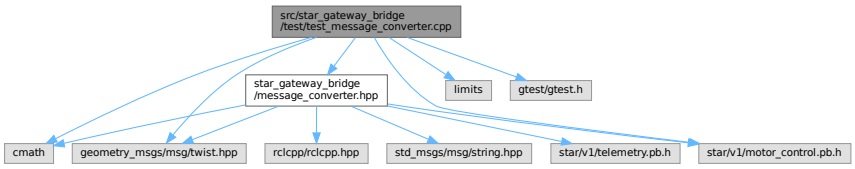
```
00431                     "Teleop sequence reset detected: %u -> %u (ignoring as "
00432                     "system restart)",
00433                     last_teleop_sequence_, current_sequence);
00434       }
00435     }
00436
00437     last_teleop_sequence_ = current_sequence;
00438   }
00439
00440   total_teleop_frames_++;
00441 }
00442
00443 void StarGatewayBridgeNode::check_telemetry_sequence_continuity(
00444   uint32_t current_sequence)
00445 {
00446   if (first_telemetry_frame_) {
00447     last_telemetry_sequence_ = current_sequence;
00448     first_telemetry_frame_ = false;
00449     RCLCPP_INFO(this->get_logger(),
00450                 "First telemetry frame received, sequence=%u",
00451                 current_sequence);
00452   } else {
00453     uint32_t expected_seq = last_telemetry_sequence_ + 1;
00454
00455     // Detect sequence gap (accounting for uint32 wraparound)
00456     if (current_sequence != expected_seq) {
00457       uint32_t gap = current_sequence - expected_seq;
00458
00459       // Sanity check: ignore huge gaps (likely system reset or wraparound)
00460       if (gap < 10000) {
00461         telemetry_frames_dropped_ += gap;
00462         RCLCPP_WARN(this->get_logger(),
00463                     "Telemetry frame drop: expected seq %u, got %u (gap=%u, "
00464                     "total_dropped=%lu)",
00465                     expected_seq, current_sequence, gap,
00466                     telemetry_frames_dropped_);
00467       } else {
00468         RCLCPP_WARN(this->get_logger(),
00469                     "Telemetry sequence reset detected: %u -> %u (ignoring as "
00470                     "system restart)",
00471                     last_telemetry_sequence_, current_sequence);
00472       }
00473     }
00474
00475     last_telemetry_sequence_ = current_sequence;
00476   }
00477
00478   total_telemetry_frames_++;
00479 }
00480
00481 void StarGatewayBridgeNode::publish_diagnostics()
00482 {
00483   auto diag_array = diagnostic_msgs::msg::DiagnosticArray();
00484   diag_array.header.stamp = this->now();
00485
00486   diagnostic_msgs::msg::DiagnosticStatus status;
00487   status.name = "star_gateway_bridge/teleop_command_drops";
00488   status.hardware_id = "gateway_bridge";
00489
00490   // Determine severity level
00491   if (total_teleop_frames_ == 0) {
00492     status.level = diagnostic_msgs::msg::DiagnosticStatus::STALE;
00493     status.message = "No teleop commands received yet";
00494   } else if (teleop_frames_dropped_ == 0) {
00495     status.level = diagnostic_msgs::msg::DiagnosticStatus::OK;
00496     status.message = "No teleop frame drops detected";
00497   } else {
00498     double drop_rate = (teleop_frames_dropped_ * 100.0) / total_teleop_frames_;
00499
00500     if (drop_rate < 1.0) {
00501       status.level = diagnostic_msgs::msg::DiagnosticStatus::WARN;
00502       status.message =
00503         "Minor teleop drops (" + std::to_string(drop_rate) + "%)";
00504     } else {
00505       status.level = diagnostic_msgs::msg::DiagnosticStatus::ERROR;
00506       status.message =
00507         "Critical teleop loss (" + std::to_string(drop_rate) + "%)";
00508     }
00509   }
00510
00511   // Add detailed metrics as key-value pairs
00512   diagnostic_msgs::msg::KeyValue kv;
00513
00514   kv.key = "total_frames";
00515   kv.value = std::to_string(total_teleop_frames_);
00516   status.values.push_back(kv);
00517
```

```
00518   kv.key = "dropped_frames";
00519   kv.value = std::to_string(teleop_frames_dropped_);
00520   status.values.push_back(kv);
00521
00522   double drop_rate =
00523     total_teleop_frames_ > 0 ?
00524     (teleop_frames_dropped_ * 100.0) / total_teleop_frames_ :
00525     0.0;
00526   kv.key = "drop_rate_percent";
00527   kv.value = std::to_string(drop_rate);
00528   status.values.push_back(kv);
00529
00530   kv.key = "last_sequence";
00531   kv.value = std::to_string(last_teleop_sequence_);
00532   status.values.push_back(kv);
00533
00534   diag_array.status.push_back(status);
00535
00536   // Add telemetry diagnostics
00537   diagnostic_msgs::msg::DiagnosticStatus telemetry_status;
00538   telemetry_status.name = "star_gateway_bridge/telemetry_frame_drops";
00539   telemetry_status.hardware_id = "gateway_bridge";
00540
00541   if (total_telemetry_frames_ == 0) {
00542     telemetry_status.level = diagnostic_msgs::msg::DiagnosticStatus::STALE;
00543     telemetry_status.message = "No telemetry frames received";
00544   } else if (telemetry_frames_dropped_ == 0) {
00545     telemetry_status.level = diagnostic_msgs::msg::DiagnosticStatus::OK;
00546     telemetry_status.message = "No telemetry drops";
00547   } else {
00548     double telemetry_drop_rate =
00549       (telemetry_frames_dropped_ * 100.0) / total_telemetry_frames_;
00550
00551     if (telemetry_drop_rate < 5.0) {
00552       telemetry_status.level = diagnostic_msgs::msg::DiagnosticStatus::WARN;
00553       telemetry_status.message = "Minor telemetry drops (" +
00554         std::to_string(telemetry_drop_rate) + "%)";
00555     } else {
00556       telemetry_status.level = diagnostic_msgs::msg::DiagnosticStatus::ERROR;
00557       telemetry_status.message = "Critical telemetry loss (" +
00558         std::to_string(telemetry_drop_rate) + "%)";
00559     }
00560   }
00561
00562   // Add telemetry metrics
00563   diagnostic_msgs::msg::KeyValue telemetry_kv;
00564
00565   telemetry_kv.key = "total_frames";
00566   telemetry_kv.value = std::to_string(total_telemetry_frames_);
00567   telemetry_status.values.push_back(telemetry_kv);
00568
00569   telemetry_kv.key = "dropped_frames";
00570   telemetry_kv.value = std::to_string(telemetry_frames_dropped_);
00571   telemetry_status.values.push_back(telemetry_kv);
00572
00573   double telemetry_drop_rate =
00574     total_telemetry_frames_ > 0 ?
00575     (telemetry_frames_dropped_ * 100.0) / total_telemetry_frames_ :
00576     0.0;
00577   telemetry_kv.key = "drop_rate_percent";
00578   telemetry_kv.value = std::to_string(telemetry_drop_rate);
00579   telemetry_status.values.push_back(telemetry_kv);
00580
00581   telemetry_kv.key = "last_sequence";
00582   telemetry_kv.value = std::to_string(last_telemetry_sequence_);
00583   telemetry_status.values.push_back(telemetry_kv);
00584
00585   diag_array.status.push_back(telemetry_status);
00586
00587   // Publish combined diagnostics
00588   diagnostics_pub_->publish(diag_array);
00589 }
00590
00591 } // namespace star
00592
00593 // Component registration for composable node
00594 #include "rclcpp_components/register_node_macro.hpp"
00595 RCLCPP_COMPONENTS_REGISTER_NODE(star::StarGatewayBridgeNode)
```

9.13 src/star_gateway_bridge/test/test_message_converter.cpp File Reference

```
#include "star_gateway_bridge/message_converter.hpp"
#include <cmath>
#include <limits>
#include <geometry_msgs/msg/twist.hpp>
#include <gtest/gtest.h>
#include "star/v1/motor_control.pb.h"
```

Include dependency graph for test_message_converter.cpp:



Classes

- class star::MessageConverterTest

Namespaces

- namespace star

Functions

- star::TEST_F (MessageConverterTest, TwistToCommandZeroVelocity)
- star::TEST_F (MessageConverterTest, TwistToCommandPureTranslation)
- star::TEST_F (MessageConverterTest, TwistToCommandPureRotation)
- star::TEST_F (MessageConverterTest, TwistToCommandMixedMotion)
- star::TEST_F (MessageConverterTest, TwistToCommandNegativeLinear)
- star::TEST_F (MessageConverterTest, TwistToCommandNegativeAngular)
- star::TEST_F (MessageConverterTest, CommandToTwistZeroVelocity)
- star::TEST_F (MessageConverterTest, CommandToTwistPureTranslation)
- star::TEST_F (MessageConverterTest, CommandToTwistPureRotation)
- star::TEST_F (MessageConverterTest, CommandToTwistMixedMotion)
- star::TEST_F (MessageConverterTest, KinematicsRoundtripZero)
- star::TEST_F (MessageConverterTest, KinematicsRoundtripTranslation)
- star::TEST_F (MessageConverterTest, KinematicsRoundtripRotation)
- star::TEST_F (MessageConverterTest, KinematicsRoundtripMixed)
- star::TEST_F (MessageConverterTest, TwistToCommandNaNLinear)
- star::TEST_F (MessageConverterTest, TwistToCommandNaNAngular)
- star::TEST_F (MessageConverterTest, TwistToCommandInfinityLinear)
- star::TEST_F (MessageConverterTest, TwistToCommandInfinityAngular)
- star::TEST_F (MessageConverterTest, TwistToCommandNegativeInfinity)
- star::TEST_F (MessageConverterTest, TwistToCommandZeroWheelBase)
- star::TEST_F (MessageConverterTest, TwistToCommandNegativeWheelBase)

- star::TEST_F (MessageConverterTest, CommandToTwistNaNMotor0)
- star::TEST_F (MessageConverterTest, CommandToTwistNaNMotor1)
- star::TEST_F (MessageConverterTest, CommandToTwistInfinityMotor0)
- star::TEST_F (MessageConverterTest, PidConfigToGainsNominal)
- star::TEST_F (MessageConverterTest, PidConfigToGainsZeroGains)
- star::TEST_F (MessageConverterTest, PidConfigToGainsNaN)
- star::TEST_F (MessageConverterTest, PidConfigToGainsInfinity)
- star::TEST_F (MessageConverterTest, PidConfigToGainsNegativeGains)
- star::TEST_F (MessageConverterTest, PidConfigToGainsLargeValues)
- star::TEST_F (MessageConverterTest, TwistToCommandMaxVelocity)
- star::TEST_F (MessageConverterTest, TwistToCommandVerySmallWheelBase)
- star::TEST_F (MessageConverterTest, TwistToCommandVeryLargeWheelBase)
- int main (int argc, char **argv)

9.13.1 Function Documentation

9.13.1.1 main()

```
int main (
            int argc,
            char ** argv)
```

Definition at line 508 of file test_message_converter.cpp.

9.14 test_message_converter.cpp

Go to the documentation of this file.

```
00001 // Copyright (c) 2026 STAR Project
00002 // Licensed under MIT
00003
00004 #include "star_gateway_bridge/message_converter.hpp"
00005
00006 #include <cmath>  // NOLINT(build/include_order)
00007 #include <limits>  // NOLINT(build/include_order)
00008
00009 #include <geometry_msgs/msg/twist.hpp>  // NOLINT(build/include_order)
00010 #include <gtest/gtest.h>  // NOLINT(build/include_order)
00011 #include "star/v1/motor_control.pb.h"
00012
00013 namespace star
00014 {
00015
00016 // Test fixture for MessageConverter tests
00017 class MessageConverterTest : public ::testing::Test
00018 {
00019 protected:
00020   MessageConverter converter_;
00021   static constexpr double TOLERANCE = 1e-6;
00022   static constexpr double WHEEL_BASE_M = 0.150;  // 150mm wheel base
00023 };
00024
00025 // ============================================================================
00026 // Forward Kinematics Tests (Twist -> VelocityCommand)
00027 // ============================================================================
00028
00029 TEST_F(MessageConverterTest, TwistToCommandZeroVelocity)
00030 {
00031   geometry_msgs::msg::Twist twist;
00032   twist.linear.x = 0.0;
00033   twist.angular.z = 0.0;
00034
00035   star::v1::VelocityCommand command;
00036   ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00037
```

```
00038   EXPECT_NEAR(command.front_left_velocity_mps(), 0.0, TOLERANCE);
00039   EXPECT_NEAR(command.front_right_velocity_mps(), 0.0, TOLERANCE);
00040 }
00041
00042 TEST_F(MessageConverterTest, TwistToCommandPureTranslation)
00043 {
00044   geometry_msgs::msg::Twist twist;
00045   twist.linear.x = 1.0;  // 1 m/s forward
00046   twist.angular.z = 0.0;
00047
00048   star::v1::VelocityCommand command;
00049   ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00050
00051   // Both wheels should move at same speed
00052   EXPECT_NEAR(command.front_left_velocity_mps(), 1.0, TOLERANCE);
00053   EXPECT_NEAR(command.front_right_velocity_mps(), 1.0, TOLERANCE);
00054 }
00055
00056 TEST_F(MessageConverterTest, TwistToCommandPureRotation)
00057 {
00058   geometry_msgs::msg::Twist twist;
00059   twist.linear.x = 0.0;
00060   twist.angular.z = 1.0;  // 1 rad/s counter-clockwise
00061
00062   star::v1::VelocityCommand command;
00063   ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00064
00065   // Differential rotation: v_left = -omega*L/2, v_right = +omega*L/2
00066   double expected_velocity = twist.angular.z * WHEEL_BASE_M / 2.0;  // omega * L/2 = 0.075 m/s
00067
00068   EXPECT_NEAR(command.front_left_velocity_mps(), -expected_velocity, TOLERANCE);
00069   EXPECT_NEAR(command.front_right_velocity_mps(), expected_velocity, TOLERANCE);
00070 }
00071
00072 TEST_F(MessageConverterTest, TwistToCommandMixedMotion)
00073 {
00074   geometry_msgs::msg::Twist twist;
00075   twist.linear.x = 1.0;   // 1 m/s forward
00076   twist.angular.z = 2.0;  // 2 rad/s counter-clockwise
00077
00078   star::v1::VelocityCommand command;
00079   ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00080
00081   // v_left = v - omega*L/2
00082   // v_right = v + omega*L/2
00083   double rotational_component = 2.0 * WHEEL_BASE_M / 2.0;  // 0.150 m/s
00084   double expected_left = 1.0 - rotational_component;       // 0.850 m/s
00085   double expected_right = 1.0 + rotational_component;      // 1.150 m/s
00086
00087   EXPECT_NEAR(command.front_left_velocity_mps(), expected_left, TOLERANCE);
00088   EXPECT_NEAR(command.front_right_velocity_mps(), expected_right, TOLERANCE);
00089 }
00090
00091 TEST_F(MessageConverterTest, TwistToCommandNegativeLinear)
00092 {
00093   geometry_msgs::msg::Twist twist;
00094   twist.linear.x = -0.5;  // 0.5 m/s backward
00095   twist.angular.z = 0.0;
00096
00097   star::v1::VelocityCommand command;
00098   ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00099
00100   // Both wheels should be negative (backward)
00101   EXPECT_NEAR(command.front_left_velocity_mps(), -0.5, TOLERANCE);
00102   EXPECT_NEAR(command.front_right_velocity_mps(), -0.5, TOLERANCE);
00103 }
00104
00105 TEST_F(MessageConverterTest, TwistToCommandNegativeAngular)
00106 {
00107   geometry_msgs::msg::Twist twist;
00108   twist.linear.x = 0.0;
00109   twist.angular.z = -1.0;  // 1 rad/s clockwise
00110
00111   star::v1::VelocityCommand command;
00112   ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00113
00114   // Clockwise rotation: left positive, right negative
00115   double expected_velocity = std::abs(twist.angular.z) * WHEEL_BASE_M / 2.0;
00116
00117   EXPECT_NEAR(command.front_left_velocity_mps(), expected_velocity, TOLERANCE);
00118   EXPECT_NEAR(command.front_right_velocity_mps(), -expected_velocity, TOLERANCE);
00119 }
00120
00121 // ============================================================================
00122 // Inverse Kinematics Tests (VelocityCommand -> Twist)
00123 // ============================================================================
00124
```

```
00125 TEST_F(MessageConverterTest, CommandToTwistZeroVelocity)
00126 {
00127   star::v1::VelocityCommand command;
00128   command.set_front_left_velocity_mps(0.0);
00129   command.set_front_right_velocity_mps(0.0);
00130
00131   geometry_msgs::msg::Twist twist;
00132   ASSERT_TRUE(converter_.velocity_command_to_twist(command, twist, WHEEL_BASE_M));
00133
00134   EXPECT_NEAR(twist.linear.x, 0.0, TOLERANCE);
00135   EXPECT_NEAR(twist.angular.z, 0.0, TOLERANCE);
00136 }
00137
00138 TEST_F(MessageConverterTest, CommandToTwistPureTranslation)
00139 {
00140   star::v1::VelocityCommand command;
00141   command.set_front_left_velocity_mps(1.0);
00142   command.set_front_right_velocity_mps(1.0);
00143   // Also set back wheels for completeness
00144   command.set_back_left_velocity_mps(1.0);
00145   command.set_back_right_velocity_mps(1.0);
00146
00147   geometry_msgs::msg::Twist twist;
00148   ASSERT_TRUE(converter_.velocity_command_to_twist(command, twist, WHEEL_BASE_M));
00149
00150   // v = (v_left + v_right) / 2
00151   EXPECT_NEAR(twist.linear.x, 1.0, TOLERANCE);
00152   EXPECT_NEAR(twist.angular.z, 0.0, TOLERANCE);
00153 }
00154
00155 TEST_F(MessageConverterTest, CommandToTwistPureRotation)
00156 {
00157   // For pure rotation: v_left = -v_right
00158   double wheel_velocity = 0.075;  // Arbitrary value
00159
00160   star::v1::VelocityCommand command;
00161   command.set_front_left_velocity_mps(-wheel_velocity);
00162   command.set_front_right_velocity_mps(wheel_velocity);
00163   command.set_back_left_velocity_mps(-wheel_velocity);
00164   command.set_back_right_velocity_mps(wheel_velocity);
00165
00166   geometry_msgs::msg::Twist twist;
00167   ASSERT_TRUE(converter_.velocity_command_to_twist(command, twist, WHEEL_BASE_M));
00168
00169   // omega = (v_right - v_left) / L
00170   double expected_angular = (wheel_velocity - (-wheel_velocity)) / WHEEL_BASE_M;
00171
00172   EXPECT_NEAR(twist.linear.x, 0.0, TOLERANCE);
00173   EXPECT_NEAR(twist.angular.z, expected_angular, TOLERANCE);
00174 }
00175
00176 TEST_F(MessageConverterTest, CommandToTwistMixedMotion)
00177 {
00178   star::v1::VelocityCommand command;
00179   command.set_front_left_velocity_mps(0.5);
00180   command.set_front_right_velocity_mps(1.5);
00181   command.set_back_left_velocity_mps(0.5);
00182   command.set_back_right_velocity_mps(1.5);
00183
00184   geometry_msgs::msg::Twist twist;
00185   ASSERT_TRUE(converter_.velocity_command_to_twist(command, twist, WHEEL_BASE_M));
00186
00187   // v = (v_left + v_right) / 2 = (0.5 + 1.5) / 2 = 1.0
00188   // omega = (v_right - v_left) / L = (1.5 - 0.5) / 0.150 = 6.667
00189   double expected_linear = (0.5 + 1.5) / 2.0;
00190   double expected_angular = (1.5 - 0.5) / WHEEL_BASE_M;
00191
00192   EXPECT_NEAR(twist.linear.x, expected_linear, TOLERANCE);
00193   EXPECT_NEAR(twist.angular.z, expected_angular, TOLERANCE);
00194 }
00195
00196 // ============================================================================
00197 // Kinematics Roundtrip Tests (Twist -> Command -> Twist)
00198 // ============================================================================
00199
00200 TEST_F(MessageConverterTest, KinematicsRoundtripZero)
00201 {
00202   geometry_msgs::msg::Twist original_twist;
00203   original_twist.linear.x = 0.0;
00204   original_twist.angular.z = 0.0;
00205
00206   star::v1::VelocityCommand command;
00207   ASSERT_TRUE(converter_.twist_to_velocity_command(original_twist, command, WHEEL_BASE_M));
00208
00209   geometry_msgs::msg::Twist reconstructed_twist;
00210   ASSERT_TRUE(converter_.velocity_command_to_twist(command, reconstructed_twist, WHEEL_BASE_M));
00211
```

```
00212   EXPECT_NEAR(reconstructed_twist.linear.x, original_twist.linear.x, TOLERANCE);
00213   EXPECT_NEAR(reconstructed_twist.angular.z, original_twist.angular.z, TOLERANCE);
00214 }
00215
00216 TEST_F(MessageConverterTest, KinematicsRoundtripTranslation)
00217 {
00218   geometry_msgs::msg::Twist original_twist;
00219   original_twist.linear.x = 1.23;
00220   original_twist.angular.z = 0.0;
00221
00222   star::v1::VelocityCommand command;
00223   ASSERT_TRUE(converter_.twist_to_velocity_command(original_twist, command, WHEEL_BASE_M));
00224
00225   geometry_msgs::msg::Twist reconstructed_twist;
00226   ASSERT_TRUE(converter_.velocity_command_to_twist(command, reconstructed_twist, WHEEL_BASE_M));
00227
00228   EXPECT_NEAR(reconstructed_twist.linear.x, original_twist.linear.x, TOLERANCE);
00229   EXPECT_NEAR(reconstructed_twist.angular.z, original_twist.angular.z, TOLERANCE);
00230 }
00231
00232 TEST_F(MessageConverterTest, KinematicsRoundtripRotation)
00233 {
00234   geometry_msgs::msg::Twist original_twist;
00235   original_twist.linear.x = 0.0;
00236   original_twist.angular.z = 2.5;
00237
00238   star::v1::VelocityCommand command;
00239   ASSERT_TRUE(converter_.twist_to_velocity_command(original_twist, command, WHEEL_BASE_M));
00240
00241   geometry_msgs::msg::Twist reconstructed_twist;
00242   ASSERT_TRUE(converter_.velocity_command_to_twist(command, reconstructed_twist, WHEEL_BASE_M));
00243
00244   EXPECT_NEAR(reconstructed_twist.linear.x, original_twist.linear.x, TOLERANCE);
00245   EXPECT_NEAR(reconstructed_twist.angular.z, original_twist.angular.z, TOLERANCE);
00246 }
00247
00248 TEST_F(MessageConverterTest, KinematicsRoundtripMixed)
00249 {
00250   geometry_msgs::msg::Twist original_twist;
00251   original_twist.linear.x = 0.75;
00252   original_twist.angular.z = -1.5;
00253
00254   star::v1::VelocityCommand command;
00255   ASSERT_TRUE(converter_.twist_to_velocity_command(original_twist, command, WHEEL_BASE_M));
00256
00257   geometry_msgs::msg::Twist reconstructed_twist;
00258   ASSERT_TRUE(converter_.velocity_command_to_twist(command, reconstructed_twist, WHEEL_BASE_M));
00259
00260   EXPECT_NEAR(reconstructed_twist.linear.x, original_twist.linear.x, TOLERANCE);
00261   EXPECT_NEAR(reconstructed_twist.angular.z, original_twist.angular.z, TOLERANCE);
00262 }
00263
00264 // ============================================================================
00265 // Validation Tests (NaN, Infinity, Invalid Values)
00266 // ============================================================================
00267
00268 TEST_F(MessageConverterTest, TwistToCommandNaNLinear)
00269 {
00270   geometry_msgs::msg::Twist twist;
00271   twist.linear.x = std::numeric_limits<double>::quiet_NaN();
00272   twist.angular.z = 0.0;
00273
00274   star::v1::VelocityCommand command;
00275   ASSERT_FALSE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00276 }
00277
00278 TEST_F(MessageConverterTest, TwistToCommandNaNAngular)
00279 {
00280   geometry_msgs::msg::Twist twist;
00281   twist.linear.x = 0.0;
00282   twist.angular.z = std::numeric_limits<double>::quiet_NaN();
00283
00284   star::v1::VelocityCommand command;
00285   ASSERT_FALSE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00286 }
00287
00288 TEST_F(MessageConverterTest, TwistToCommandInfinityLinear)
00289 {
00290   geometry_msgs::msg::Twist twist;
00291   twist.linear.x = std::numeric_limits<double>::infinity();
00292   twist.angular.z = 0.0;
00293
00294   star::v1::VelocityCommand command;
00295   ASSERT_FALSE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00296 }
00297
00298 TEST_F(MessageConverterTest, TwistToCommandInfinityAngular)
```

```
00299 {
00300   geometry_msgs::msg::Twist twist;
00301   twist.linear.x = 0.0;
00302   twist.angular.z = std::numeric_limits<double>::infinity();
00303
00304   star::v1::VelocityCommand command;
00305   ASSERT_FALSE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00306 }
00307
00308 TEST_F(MessageConverterTest, TwistToCommandNegativeInfinity)
00309 {
00310   geometry_msgs::msg::Twist twist;
00311   twist.linear.x = -std::numeric_limits<double>::infinity();
00312   twist.angular.z = 0.0;
00313
00314   star::v1::VelocityCommand command;
00315   ASSERT_FALSE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00316 }
00317
00318 TEST_F(MessageConverterTest, TwistToCommandZeroWheelBase)
00319 {
00320   geometry_msgs::msg::Twist twist;
00321   twist.linear.x = 1.0;
00322   twist.angular.z = 0.0;
00323
00324   star::v1::VelocityCommand command;
00325   ASSERT_FALSE(converter_.twist_to_velocity_command(twist, command, 0.0));
00326 }
00327
00328 TEST_F(MessageConverterTest, TwistToCommandNegativeWheelBase)
00329 {
00330   geometry_msgs::msg::Twist twist;
00331   twist.linear.x = 1.0;
00332   twist.angular.z = 0.0;
00333
00334   star::v1::VelocityCommand command;
00335   ASSERT_FALSE(converter_.twist_to_velocity_command(twist, command, -0.150));
00336 }
00337
00338 TEST_F(MessageConverterTest, CommandToTwistNaNMotor0)
00339 {
00340   star::v1::VelocityCommand command;
00341   command.set_front_left_velocity_mps(std::numeric_limits<double>::quiet_NaN());
00342   command.set_front_right_velocity_mps(0.0);
00343
00344   geometry_msgs::msg::Twist twist;
00345   ASSERT_FALSE(converter_.velocity_command_to_twist(command, twist, WHEEL_BASE_M));
00346 }
00347
00348 TEST_F(MessageConverterTest, CommandToTwistNaNMotor1)
00349 {
00350   star::v1::VelocityCommand command;
00351   command.set_front_left_velocity_mps(0.0);
00352   command.set_front_right_velocity_mps(std::numeric_limits<double>::quiet_NaN());
00353
00354   geometry_msgs::msg::Twist twist;
00355   ASSERT_FALSE(converter_.velocity_command_to_twist(command, twist, WHEEL_BASE_M));
00356 }
00357
00358 TEST_F(MessageConverterTest, CommandToTwistInfinityMotor0)
00359 {
00360   star::v1::VelocityCommand command;
00361   command.set_front_left_velocity_mps(std::numeric_limits<double>::infinity());
00362   command.set_front_right_velocity_mps(0.0);
00363
00364   geometry_msgs::msg::Twist twist;
00365   ASSERT_FALSE(converter_.velocity_command_to_twist(command, twist, WHEEL_BASE_M));
00366 }
00367
00368 // ============================================================================
00369 // PID Configuration Tests (Proto -> ROS2 direction)
00370 // ============================================================================
00371
00372 TEST_F(MessageConverterTest, PidConfigToGainsNominal)
00373 {
00374   star::v1::PidConfig proto_config;
00375   proto_config.set_kp(1.5);
00376   proto_config.set_ki(0.3);
00377   proto_config.set_kd(0.05);
00378
00379   double kp, ki, kd;
00380   ASSERT_TRUE(converter_.pid_config_to_gains(proto_config, kp, ki, kd));
00381
00382   EXPECT_NEAR(kp, 1.5, TOLERANCE);
00383   EXPECT_NEAR(ki, 0.3, TOLERANCE);
00384   EXPECT_NEAR(kd, 0.05, TOLERANCE);
00385 }
```

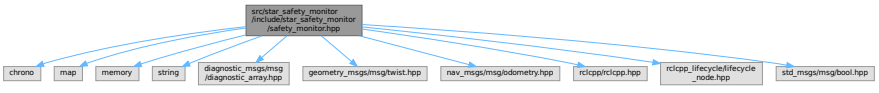
```
00386
00387 TEST_F(MessageConverterTest, PidConfigToGainsZeroGains)
00388 {
00389   star::v1::PidConfig proto_config;
00390   proto_config.set_kp(0.0);
00391   proto_config.set_ki(0.0);
00392   proto_config.set_kd(0.0);
00393
00394   double kp, ki, kd;
00395   ASSERT_TRUE(converter_.pid_config_to_gains(proto_config, kp, ki, kd));
00396
00397   EXPECT_NEAR(kp, 0.0, TOLERANCE);
00398   EXPECT_NEAR(ki, 0.0, TOLERANCE);
00399   EXPECT_NEAR(kd, 0.0, TOLERANCE);
00400 }
00401
00402 TEST_F(MessageConverterTest, PidConfigToGainsNaN)
00403 {
00404   star::v1::PidConfig proto_config;
00405   proto_config.set_kp(std::numeric_limits<double>::quiet_NaN());
00406   proto_config.set_ki(0.3);
00407   proto_config.set_kd(0.05);
00408
00409   double kp, ki, kd;
00410   ASSERT_FALSE(converter_.pid_config_to_gains(proto_config, kp, ki, kd));
00411 }
00412
00413 TEST_F(MessageConverterTest, PidConfigToGainsInfinity)
00414 {
00415   star::v1::PidConfig proto_config;
00416   proto_config.set_kp(1.5);
00417   proto_config.set_ki(std::numeric_limits<double>::infinity());
00418   proto_config.set_kd(0.05);
00419
00420   double kp, ki, kd;
00421   ASSERT_FALSE(converter_.pid_config_to_gains(proto_config, kp, ki, kd));
00422 }
00423
00424 TEST_F(MessageConverterTest, PidConfigToGainsNegativeGains)
00425 {
00426   // Negative PID gains are mathematically valid (though unusual)
00427   star::v1::PidConfig proto_config;
00428   proto_config.set_kp(-1.5);
00429   proto_config.set_ki(-0.3);
00430   proto_config.set_kd(-0.05);
00431
00432   double kp, ki, kd;
00433   // Implementation only checks for NaN/infinity, not negative values
00434   ASSERT_TRUE(converter_.pid_config_to_gains(proto_config, kp, ki, kd));
00435
00436   EXPECT_NEAR(kp, -1.5, TOLERANCE);
00437   EXPECT_NEAR(ki, -0.3, TOLERANCE);
00438   EXPECT_NEAR(kd, -0.05, TOLERANCE);
00439 }
00440
00441 TEST_F(MessageConverterTest, PidConfigToGainsLargeValues)
00442 {
00443   star::v1::PidConfig proto_config;
00444   proto_config.set_kp(1000.0);
00445   proto_config.set_ki(500.0);
00446   proto_config.set_kd(100.0);
00447
00448   double kp, ki, kd;
00449   ASSERT_TRUE(converter_.pid_config_to_gains(proto_config, kp, ki, kd));
00450
00451   EXPECT_NEAR(kp, 1000.0, TOLERANCE);
00452   EXPECT_NEAR(ki, 500.0, TOLERANCE);
00453   EXPECT_NEAR(kd, 100.0, TOLERANCE);
00454 }
00455
00456 // ============================================================================
00457 // Edge Case Tests
00458 // ============================================================================
00459
00460 TEST_F(MessageConverterTest, TwistToCommandMaxVelocity)
00461 {
00462   // Test with high velocities (near physical limits)
00463   geometry_msgs::msg::Twist twist;
00464   twist.linear.x = 10.0;   // 10 m/s (unrealistic but valid)
00465   twist.angular.z = 20.0;  // 20 rad/s
00466
00467   star::v1::VelocityCommand command;
00468   ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00469
00470   // Should not crash or produce NaN
00471   EXPECT_FALSE(std::isnan(command.front_left_velocity_mps()));
00472   EXPECT_FALSE(std::isnan(command.front_right_velocity_mps()));
```

```
00473   EXPECT_FALSE(std::isinf(command.front_left_velocity_mps()));
00474   EXPECT_FALSE(std::isinf(command.front_right_velocity_mps()));
00475 }
00476
00477 TEST_F(MessageConverterTest, TwistToCommandVerySmallWheelBase)
00478 {
00479   geometry_msgs::msg::Twist twist;
00480   twist.linear.x = 1.0;
00481   twist.angular.z = 1.0;
00482
00483   star::v1::VelocityCommand command;
00484   // Very small but positive wheel base (1mm)
00485   ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, 0.001));
00486
00487   // Should produce large angular velocities but not overflow
00488   EXPECT_FALSE(std::isinf(command.front_left_velocity_mps()));
00489   EXPECT_FALSE(std::isinf(command.front_right_velocity_mps()));
00490 }
00491
00492 TEST_F(MessageConverterTest, TwistToCommandVeryLargeWheelBase)
00493 {
00494   geometry_msgs::msg::Twist twist;
00495   twist.linear.x = 1.0;
00496   twist.angular.z = 1.0;
00497
00498   star::v1::VelocityCommand command;
00499   // Large wheel base (10m)
00500   ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, 10.0));
00501
00502   EXPECT_FALSE(std::isnan(command.front_left_velocity_mps()));
00503   EXPECT_FALSE(std::isnan(command.front_right_velocity_mps()));
00504 }
00505
00506 }  // namespace star
00507
00508 int main(int argc, char **argv)
00509 {
00510   testing::InitGoogleTest(&argc, argv);
00511   return RUN_ALL_TESTS();
00512 }
```

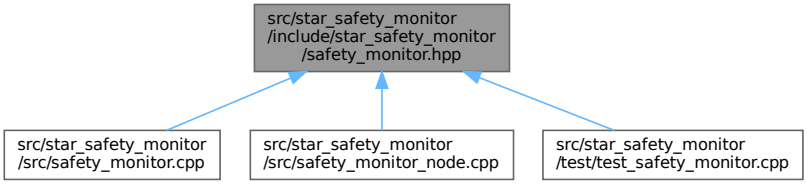
9.15 src/star_safety_monitor/include/star_safety_monitor/safety_monitor.hpp File Reference

```
#include <chrono>
#include <map>
#include <memory>
#include <string>
#include <diagnostic_msgs/msg/diagnostic_array.hpp>
#include <geometry_msgs/msg/twist.hpp>
#include <nav_msgs/msg/odometry.hpp>
#include <rclcpp/rclcpp.hpp>
#include <rclcpp_lifecycle/lifecycle_node.hpp>
#include <std_msgs/msg/bool.hpp>
```

Include dependency graph for safety_monitor.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class star_safety_monitor::SafetyMonitor

Namespaces

- namespace star_safety_monitor

Enumerations

- enum class star_safety_monitor::SeverityLevel { star_safety_monitor::OK = 0 , star_safety_monitor::WARN = 1 , star_safety_monitor::ERROR = 2 , star_safety_monitor::STALE = 3 }

9.16 safety_monitor.hpp

Go to the documentation of this file.

```
00001 // Copyright 2026 STAR Team
00002 // SPDX-License-Identifier: MIT
00003 //
00004 // Permission is hereby granted, free of charge, to any person obtaining a copy
00005 // of this software and associated documentation files (the "Software"), to deal
00006 // in the Software without restriction, including without limitation the rights
00007 // to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00008 // copies of the Software, and to permit persons to whom the Software is
00009 // furnished to do so, subject to the following conditions:
00010 //
00011 // The above copyright notice and this permission notice shall be included in
00012 // all copies or substantial portions of the Software.
00013 //
00014 // THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00015 // IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00016 // FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00017 // AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00018 // LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00019 // OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
00020 // SOFTWARE.
00021
00022 #ifndef STAR_SAFETY_MONITOR__SAFETY_MONITOR_HPP_
00023 #define STAR_SAFETY_MONITOR__SAFETY_MONITOR_HPP_
00024
00025 #include <chrono>
00026 #include <map>
00027 #include <memory>
00028 #include <string>
00029
00030 #include <diagnostic_msgs/msg/diagnostic_array.hpp>
00031 #include <geometry_msgs/msg/twist.hpp>
00032 #include <nav_msgs/msg/odometry.hpp>
00033 #include <rclcpp/rclcpp.hpp>
```

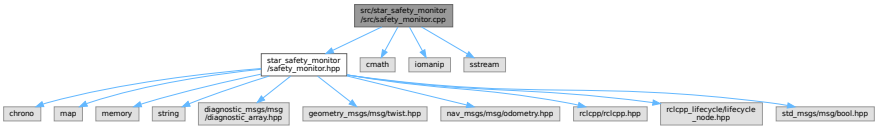
```
00034 #include <rclcpp_lifecycle/lifecycle_node.hpp>
00035 #include <std_msgs/msg/bool.hpp>
00036
00037 namespace star_safety_monitor
00038 {
00039
00040 // Enum for safety severity levels
00041 enum class SeverityLevel { OK = 0, WARN = 1, ERROR = 2, STALE = 3 };
00042
00043 class SafetyMonitor : public rclcpp_lifecycle::LifecycleNode {
00044 public:
00045   explicit SafetyMonitor(
00046     const rclcpp::NodeOptions & options = rclcpp::NodeOptions());
00047   ~SafetyMonitor();
00048
00049   // Lifecycle callbacks
00050   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00051   on_configure(const rclcpp_lifecycle::State & previous_state) override;
00052
00053   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00054   on_activate(const rclcpp_lifecycle::State & previous_state) override;
00055
00056   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00057   on_deactivate(const rclcpp_lifecycle::State & previous_state) override;
00058
00059   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00060   on_cleanup(const rclcpp_lifecycle::State & previous_state) override;
00061
00062   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00063   on_shutdown(const rclcpp_lifecycle::State & previous_state) override;
00064
00065 private:
00066   // Subscription callbacks
00067   void odometry_callback(const nav_msgs::msg::Odometry::SharedPtr msg);
00068   void diagnostics_callback(
00069     const diagnostic_msgs::msg::DiagnosticArray::SharedPtr msg);
00070   void cmd_vel_callback(const geometry_msgs::msg::Twist::SharedPtr msg);
00071
00072   // Timer callback for monitoring and publishing diagnostics
00073   void monitoring_timer_callback();
00074
00075   // Monitoring and safety check methods
00076   void check_heartbeat_health();
00077   void check_velocity_limits();
00078   void check_motor_stall();
00079   void check_diagnostic_health();
00080   void update_overall_state();
00081   void publish_diagnostics();
00082   double compute_linear_magnitude(const geometry_msgs::msg::Twist & twist) const;
00083
00084   // Helper methods
00085   void add_diagnostic_status(
00086     diagnostic_msgs::msg::DiagnosticStatus & status,
00087     const std::string & name, SeverityLevel level,
00088     const std::string & message);
00089
00090   // Publishers and Subscribers
00091   rclcpp::Subscription<nav_msgs::msg::Odometry>::SharedPtr odom_sub_;
00092   rclcpp::Subscription<diagnostic_msgs::msg::DiagnosticArray>::SharedPtr
00093     diag_sub_;
00094   rclcpp::Subscription<geometry_msgs::msg::Twist>::SharedPtr cmd_vel_sub_;
00095
00096   rclcpp_lifecycle::LifecyclePublisher<
00097     diagnostic_msgs::msg::DiagnosticArray>::SharedPtr diagnostics_pub_;
00098   rclcpp_lifecycle::LifecyclePublisher<std_msgs::msg::Bool>::SharedPtr
00099     emergency_stop_pub_;
00100
00101   // Timer
00102   rclcpp::TimerBase::SharedPtr monitoring_timer_;
00103
00104   // Safety parameters (declared in on_configure)
00105   int heartbeat_timeout_ms_;
00106   double max_linear_velocity_;
00107   double max_angular_velocity_;
00108   double publish_rate_;
00109   bool enable_auto_estop_;
00110   double estop_recovery_delay_;
00111   double stall_detection_threshold_;
00112   int stall_samples_required_;
00113   int cmd_vel_timeout_ms_;
00114
00115   // State tracking
00116   std::chrono::system_clock::time_point last_diagnostics_time_;
00117   std::map<std::string, std::chrono::system_clock::time_point> heartbeat_times_;
00118   double current_linear_velocity_{0.0};
00119   double current_angular_velocity_{0.0};
00120   bool velocity_exceeded_{false};
```

```
00121   bool heartbeat_timeout_triggered_{false};
00122   bool emergency_stop_active_{false};
00123   std::chrono::system_clock::time_point estop_trigger_time_;
00124
00125   // Motor stall tracking
00126   geometry_msgs::msg::Twist last_cmd_vel_{};
00127   std::chrono::system_clock::time_point last_cmd_vel_time_;
00128   int stall_detection_count_{0};
00129   bool motor_stall_detected_{false};
00130
00131   SeverityLevel overall_severity_{SeverityLevel::OK};
00132 };
00133
00134 }  // namespace star_safety_monitor
00135
00136 #endif  // STAR_SAFETY_MONITOR__SAFETY_MONITOR_HPP_
```

9.17 src/star_safety_monitor/src/safety_monitor.cpp File Reference

```
#include "star_safety_monitor/safety_monitor.hpp"
#include <cmath>
#include <iomanip>
#include <sstream>
```

Include dependency graph for safety_monitor.cpp:



Namespaces

- namespace star_safety_monitor

9.18 safety_monitor.cpp

Go to the documentation of this file.

```
00001 // Copyright 2026 STAR Team
00002 // SPDX-License-Identifier: MIT
00003 //
00004 // Permission is hereby granted, free of charge, to any person obtaining a copy
00005 // of this software and associated documentation files (the "Software"), to deal
00006 // in the Software without restriction, including without limitation the rights
00007 // to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00008 // copies of the Software, and to permit persons to whom the Software is
00009 // furnished to do so, subject to the following conditions:
00010 //
00011 // The above copyright notice and this permission notice shall be included in all
00012 // copies or substantial portions of the Software.
00013 //
00014 // THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00015 // IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00016 // FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00017 // AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00018 // LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00019 // OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
00020 // SOFTWARE.
00021
00022 #include "star_safety_monitor/safety_monitor.hpp"
00023 #include <cmath>
00024 #include <iomanip>
00025 #include <sstream>
```

```
00026
00027 namespace star_safety_monitor
00028 {
00029
00030 SafetyMonitor::SafetyMonitor(const rclcpp::NodeOptions & options)
00031 : rclcpp_lifecycle::LifecycleNode("safety_monitor", options)
00032 {
00033   RCLCPP_INFO(get_logger(), "SafetyMonitor constructor called");
00034 }
00035
00036 SafetyMonitor::~SafetyMonitor()
00037 {
00038   RCLCPP_INFO(get_logger(), "SafetyMonitor destructor called");
00039 }
00040
00041 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00042 SafetyMonitor::on_configure(const rclcpp_lifecycle::State & previous_state)
00043 {
00044   (void)previous_state;
00045   RCLCPP_INFO(get_logger(), "Configuring SafetyMonitor");
00046
00047   try {
00048     // Declare and get parameters
00049     declare_parameter("heartbeat_timeout_ms", 500);
00050     declare_parameter("max_linear_velocity", 1.0);
00051     declare_parameter("max_angular_velocity", 2.0);
00052     declare_parameter("publish_rate", 10.0);
00053     declare_parameter("enable_auto_estop", true);
00054     declare_parameter("estop_recovery_delay", 5.0);
00055     declare_parameter("stall_detection_threshold", 0.05);
00056     declare_parameter("stall_samples_required", 5);
00057     declare_parameter("cmd_vel_timeout_ms", 500);
00058
00059     heartbeat_timeout_ms_ = get_parameter("heartbeat_timeout_ms").as_int();
00060     max_linear_velocity_ = get_parameter("max_linear_velocity").as_double();
00061     max_angular_velocity_ = get_parameter("max_angular_velocity").as_double();
00062     publish_rate_ = get_parameter("publish_rate").as_double();
00063     enable_auto_estop_ = get_parameter("enable_auto_estop").as_bool();
00064     estop_recovery_delay_ = get_parameter("estop_recovery_delay").as_double();
00065     stall_detection_threshold_ = get_parameter("stall_detection_threshold").as_double();
00066     stall_samples_required_ = get_parameter("stall_samples_required").as_int();
00067     cmd_vel_timeout_ms_ = get_parameter("cmd_vel_timeout_ms").as_int();
00068
00069     RCLCPP_INFO(get_logger(), "Configuration loaded:");
00070     RCLCPP_INFO(get_logger(), "  Heartbeat timeout: %d ms", heartbeat_timeout_ms_);
00071     RCLCPP_INFO(get_logger(), "  Max linear velocity: %.2f m/s", max_linear_velocity_);
00072     RCLCPP_INFO(get_logger(), "  Max angular velocity: %.2f rad/s", max_angular_velocity_);
00073
00074     // Create lifecycle publishers
00075     diagnostics_pub_ =
00076       create_publisher<diagnostic_msgs::msg::DiagnosticArray>("/diagnostics", 10);
00077     emergency_stop_pub_ = create_publisher<std_msgs::msg::Bool>("/emergency_stop", 10);
00078
00079     // Create subscribers
00080     odom_sub_ = create_subscription<nav_msgs::msg::Odometry>(
00081       "/odom", 10,
00082       std::bind(&SafetyMonitor::odometry_callback, this, std::placeholders::_1));
00083
00084     diag_sub_ = create_subscription<diagnostic_msgs::msg::DiagnosticArray>(
00085       "/diagnostics", 10,
00086       std::bind(&SafetyMonitor::diagnostics_callback, this, std::placeholders::_1));
00087
00088     cmd_vel_sub_ = create_subscription<geometry_msgs::msg::Twist>(
00089       "/cmd_vel", 10,
00090       std::bind(&SafetyMonitor::cmd_vel_callback, this, std::placeholders::_1));
00091
00092     // Initialize state
00093     last_diagnostics_time_ = std::chrono::system_clock::now();
00094     last_cmd_vel_time_ = std::chrono::system_clock::now();
00095     heartbeat_times_.clear();
00096     current_linear_velocity_ = 0.0;
00097     current_angular_velocity_ = 0.0;
00098     velocity_exceeded_ = false;
00099     heartbeat_timeout_triggered_ = false;
00100     emergency_stop_active_ = false;
00101     motor_stall_detected_ = false;
00102     stall_detection_count_ = 0;
00103     overall_severity_ = SeverityLevel::OK;
00104
00105     RCLCPP_INFO(get_logger(), "SafetyMonitor configured successfully");
00106     return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::SUCCESS;
00107   } catch (const std::exception & e) {
00108     RCLCPP_ERROR(get_logger(), "Configuration failed: %s", e.what());
00109     return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::FAILURE;
00110   }
00111 }
00112
```

```
00113 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00114 SafetyMonitor::on_activate(const rclcpp_lifecycle::State & previous_state)
00115 {
00116   (void)previous_state;
00117   RCLCPP_INFO(get_logger(), "Activating SafetyMonitor");
00118
00119   try {
00120     // Activate publishers
00121     diagnostics_pub_->on_activate();
00122     emergency_stop_pub_->on_activate();
00123
00124     // Create monitoring timer (10 Hz by default)
00125     auto timer_period = std::chrono::milliseconds(
00126       static_cast<int>(1000.0 / publish_rate_));
00127     monitoring_timer_ =
00128       create_wall_timer(timer_period,
00129         std::bind(&SafetyMonitor::monitoring_timer_callback, this));
00130
00131     RCLCPP_INFO(get_logger(), "SafetyMonitor activated successfully");
00132     return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::SUCCESS;
00133   } catch (const std::exception & e) {
00134     RCLCPP_ERROR(get_logger(), "Activation failed: %s", e.what());
00135     return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::FAILURE;
00136   }
00137 }
00138
00139 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00140 SafetyMonitor::on_deactivate(const rclcpp_lifecycle::State & previous_state)
00141 {
00142   (void)previous_state;
00143   RCLCPP_INFO(get_logger(), "Deactivating SafetyMonitor");
00144
00145   try {
00146     // Cancel timer
00147     if (monitoring_timer_) {
00148       monitoring_timer_->cancel();
00149       monitoring_timer_.reset();
00150     }
00151
00152     // Deactivate publishers
00153     diagnostics_pub_->on_deactivate();
00154     emergency_stop_pub_->on_deactivate();
00155
00156     RCLCPP_INFO(get_logger(), "SafetyMonitor deactivated successfully");
00157     return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::SUCCESS;
00158   } catch (const std::exception & e) {
00159     RCLCPP_ERROR(get_logger(), "Deactivation failed: %s", e.what());
00160     return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::FAILURE;
00161   }
00162 }
00163
00164 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00165 SafetyMonitor::on_cleanup(const rclcpp_lifecycle::State & previous_state)
00166 {
00167   (void)previous_state;
00168   RCLCPP_INFO(get_logger(), "Cleaning up SafetyMonitor");
00169
00170   try {
00171     // Reset publishers and subscribers
00172     diagnostics_pub_.reset();
00173     emergency_stop_pub_.reset();
00174     odom_sub_.reset();
00175     diag_sub_.reset();
00176     cmd_vel_sub_.reset();
00177
00178     // Clear state
00179     heartbeat_times_.clear();
00180
00181     RCLCPP_INFO(get_logger(), "SafetyMonitor cleaned up successfully");
00182     return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::SUCCESS;
00183   } catch (const std::exception & e) {
00184     RCLCPP_ERROR(get_logger(), "Cleanup failed: %s", e.what());
00185     return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::FAILURE;
00186   }
00187 }
00188
00189 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00190 SafetyMonitor::on_shutdown(const rclcpp_lifecycle::State & previous_state)
00191 {
00192   (void)previous_state;
00193   RCLCPP_INFO(get_logger(), "Shutting down SafetyMonitor");
00194
00195   // Ensure emergency stop is triggered on shutdown if active
00196   if (emergency_stop_active_) {
00197     auto msg = std_msgs::msg::Bool();
00198     msg.data = true;
00199     emergency_stop_pub_->publish(msg);
```

```
00200   }
00201
00202   return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::SUCCESS;
00203 }
00204
00205 void SafetyMonitor::odometry_callback(const nav_msgs::msg::Odometry::SharedPtr msg)
00206 {
00207   // Extract current velocities
00208   current_linear_velocity_ = std::sqrt(
00209     msg->twist.twist.linear.x * msg->twist.twist.linear.x +
00210     msg->twist.twist.linear.y * msg->twist.twist.linear.y +
00211     msg->twist.twist.linear.z * msg->twist.twist.linear.z);
00212
00213   current_angular_velocity_ = std::sqrt(
00214     msg->twist.twist.angular.x * msg->twist.twist.angular.x +
00215     msg->twist.twist.angular.y * msg->twist.twist.angular.y +
00216     msg->twist.twist.angular.z * msg->twist.twist.angular.z);
00217 }
00218
00219 void SafetyMonitor::diagnostics_callback(
00220   const diagnostic_msgs::msg::DiagnosticArray::SharedPtr msg)
00221 {
00222   last_diagnostics_time_ = std::chrono::system_clock::now();
00223
00224   // Update heartbeat times for known nodes
00225   for (const auto & status : msg->status) {
00226     heartbeat_times_[status.name] = std::chrono::system_clock::now();
00227   }
00228 }
00229
00230 void SafetyMonitor::cmd_vel_callback(
00231   const geometry_msgs::msg::Twist::SharedPtr msg)
00232 {
00233   last_cmd_vel_ = *msg;
00234   last_cmd_vel_time_ = std::chrono::system_clock::now();
00235 }
00236
00237 void SafetyMonitor::monitoring_timer_callback()
00238 {
00239   // Perform safety checks
00240   check_heartbeat_health();
00241   check_velocity_limits();
00242   check_motor_stall();
00243   check_diagnostic_health();
00244   update_overall_state();
00245   publish_diagnostics();
00246 }
00247
00248 void SafetyMonitor::check_heartbeat_health()
00249 {
00250   auto now = std::chrono::system_clock::now();
00251   auto timeout_duration = std::chrono::milliseconds(heartbeat_timeout_ms_);
00252
00253   // Check for stale diagnostics
00254   auto time_since_diag = now - last_diagnostics_time_;
00255   if (time_since_diag > timeout_duration) {
00256     if (!heartbeat_timeout_triggered_) {
00257       RCLCPP_WARN(get_logger(), "Heartbeat timeout detected!");
00258       heartbeat_timeout_triggered_ = true;
00259       if (enable_auto_estop_) {
00260         emergency_stop_active_ = true;
00261         estop_trigger_time_ = now;
00262       }
00263     }
00264   } else {
00265     // Reset timeout if heartbeat recovered
00266     if (heartbeat_timeout_triggered_) {
00267       auto estop_duration = now - estop_trigger_time_;
00268       if (estop_duration > std::chrono::duration<double>(estop_recovery_delay_)) {
00269         RCLCPP_INFO(get_logger(), "Heartbeat recovered");
00270         heartbeat_timeout_triggered_ = false;
00271         emergency_stop_active_ = false;
00272       }
00273     }
00274   }
00275 }
00276
00277 void SafetyMonitor::check_velocity_limits()
00278 {
00279   velocity_exceeded_ = false;
00280
00281   if (current_linear_velocity_ > max_linear_velocity_) {
00282     RCLCPP_WARN(get_logger(),
00283       "Linear velocity limit exceeded: %.2f > %.2f m/s",
00284       current_linear_velocity_, max_linear_velocity_);
00285     velocity_exceeded_ = true;
00286   }
```

```
00287
00288   if (current_angular_velocity_ > max_angular_velocity_) {
00289     RCLCPP_WARN(get_logger(),
00290       "Angular velocity limit exceeded: %.2f > %.2f rad/s",
00291       current_angular_velocity_, max_angular_velocity_);
00292     velocity_exceeded_ = true;
00293   }
00294 }
00295
00296 void SafetyMonitor::check_motor_stall()
00297 {
00298   motor_stall_detected_ = false;
00299
00300   // Stall detection: command velocity present but actual velocity is near zero
00301   double cmd_linear = compute_linear_magnitude(last_cmd_vel_);
00302
00303   // Check if a command was issued recently
00304   auto now = std::chrono::system_clock::now();
00305   auto cmd_age = now - last_cmd_vel_time_;
00306
00307   if (cmd_age < std::chrono::milliseconds(cmd_vel_timeout_ms_) &&
00308     cmd_linear > stall_detection_threshold_)
00309   {
00310     // We have a recent command to move
00311     if (current_linear_velocity_ < stall_detection_threshold_) {
00312       // But we're not actually moving
00313       stall_detection_count_++;
00314
00315       if (stall_detection_count_ >= stall_samples_required_) {
00316         RCLCPP_WARN(get_logger(),
00317           "Motor stall detected: cmd=%.2f m/s, actual=%.2f m/s",
00318           cmd_linear, current_linear_velocity_);
00319         motor_stall_detected_ = true;
00320         if (enable_auto_estop_) {
00321           emergency_stop_active_ = true;
00322           estop_trigger_time_ = now;
00323         }
00324       }
00325     } else {
00326       // Motor is responding, reset counter
00327       stall_detection_count_ = 0;
00328     }
00329   } else {
00330     // No recent command, reset counter
00331     stall_detection_count_ = 0;
00332   }
00333 }
00334
00335 void SafetyMonitor::check_diagnostic_health()
00336 {
00337   // Additional diagnostic checks can be added here
00338   // For now, this is a placeholder for future expansion
00339 }
00340
00341 void SafetyMonitor::update_overall_state()
00342 {
00343   // Determine overall severity level
00344   overall_severity_ = SeverityLevel::OK;
00345
00346   if (heartbeat_timeout_triggered_) {
00347     overall_severity_ = SeverityLevel::ERROR;
00348   } else if (motor_stall_detected_) {
00349     overall_severity_ = SeverityLevel::WARN;
00350   } else if (velocity_exceeded_) {
00351     overall_severity_ = SeverityLevel::WARN;
00352   }
00353
00354   // Publish emergency stop signal if needed
00355   if (emergency_stop_active_) {
00356     auto msg = std_msgs::msg::Bool();
00357     msg.data = true;
00358     emergency_stop_pub_->publish(msg);
00359   }
00360 }
00361
00362 void SafetyMonitor::publish_diagnostics()
00363 {
00364   auto diag_array = diagnostic_msgs::msg::DiagnosticArray();
00365   diag_array.header.stamp = now();
00366
00367   // System health status
00368   diagnostic_msgs::msg::DiagnosticStatus system_status;
00369   system_status.name = "safety_monitor: System Health";
00370   system_status.hardware_id = "safety_monitor";
00371
00372   if (overall_severity_ == SeverityLevel::OK) {
00373     system_status.level = diagnostic_msgs::msg::DiagnosticStatus::OK;
```

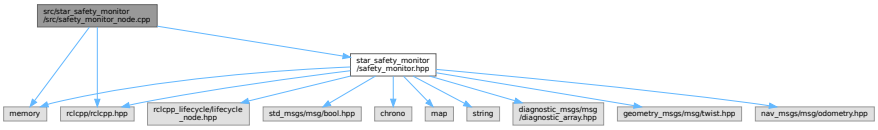
```
00374     system_status.message = "All systems nominal";
00375   } else if (overall_severity_ == SeverityLevel::WARN) {
00376     system_status.level = diagnostic_msgs::msg::DiagnosticStatus::WARN;
00377     system_status.message = "Warning conditions detected";
00378   } else if (overall_severity_ == SeverityLevel::ERROR) {
00379     system_status.level = diagnostic_msgs::msg::DiagnosticStatus::ERROR;
00380     system_status.message = "Critical error - emergency stop triggered";
00381   }
00382
00383   // Add velocity data
00384   diagnostic_msgs::msg::KeyValue kv;
00385   kv.key = "Linear Velocity (m/s)";
00386   kv.value = std::to_string(current_linear_velocity_);
00387   system_status.values.push_back(kv);
00388
00389   kv.key = "Angular Velocity (rad/s)";
00390   kv.value = std::to_string(current_angular_velocity_);
00391   system_status.values.push_back(kv);
00392
00393   kv.key = "Velocity Limit Exceeded";
00394   kv.value = velocity_exceeded_ ? "true" : "false";
00395   system_status.values.push_back(kv);
00396
00397   kv.key = "Motor Stall Detected";
00398   kv.value = motor_stall_detected_ ? "true" : "false";
00399   system_status.values.push_back(kv);
00400
00401   kv.key = "Heartbeat Timeout";
00402   kv.value = heartbeat_timeout_triggered_ ? "true" : "false";
00403   system_status.values.push_back(kv);
00404
00405   kv.key = "Emergency Stop Active";
00406   kv.value = emergency_stop_active_ ? "true" : "false";
00407   system_status.values.push_back(kv);
00408
00409   // Add heartbeat information for each tracked node
00410   diagnostic_msgs::msg::DiagnosticStatus heartbeat_status;
00411   heartbeat_status.name = "safety_monitor: Heartbeat Status";
00412   heartbeat_status.hardware_id = "safety_monitor";
00413   heartbeat_status.level = heartbeat_timeout_triggered_ ?
00414     diagnostic_msgs::msg::DiagnosticStatus::ERROR :
00415     diagnostic_msgs::msg::DiagnosticStatus::OK;
00416
00417   auto now_time = std::chrono::system_clock::now();
00418   for (const auto & [node_name, last_time] : heartbeat_times_) {
00419     auto duration_ms = std::chrono::duration_cast<std::chrono::milliseconds>(
00420       now_time - last_time).count();
00421
00422     kv.key = node_name;
00423     kv.value = std::to_string(duration_ms) + " ms";
00424     heartbeat_status.values.push_back(kv);
00425   }
00426
00427   diag_array.status.push_back(system_status);
00428   diag_array.status.push_back(heartbeat_status);
00429
00430   // Motor status
00431   diagnostic_msgs::msg::DiagnosticStatus motor_status;
00432   motor_status.name = "safety_monitor: Motor Status";
00433   motor_status.hardware_id = "safety_monitor";
00434   if (motor_stall_detected_) {
00435     motor_status.level = diagnostic_msgs::msg::DiagnosticStatus::WARN;
00436     motor_status.message = "Motor stall detected";
00437   } else {
00438     motor_status.level = diagnostic_msgs::msg::DiagnosticStatus::OK;
00439     motor_status.message = "Motor nominal";
00440   }
00441
00442   kv.key = "Stall Detected";
00443   kv.value = motor_stall_detected_ ? "true" : "false";
00444   motor_status.values.push_back(kv);
00445
00446   kv.key = "Command Velocity (m/s)";
00447   double cmd_linear = compute_linear_magnitude(last_cmd_vel_);
00448   kv.value = std::to_string(cmd_linear);
00449   motor_status.values.push_back(kv);
00450
00451   kv.key = "Actual Velocity (m/s)";
00452   kv.value = std::to_string(current_linear_velocity_);
00453   motor_status.values.push_back(kv);
00454
00455   kv.key = "Stall Detection Count";
00456   kv.value = std::to_string(stall_detection_count_);
00457   motor_status.values.push_back(kv);
00458
00459   diag_array.status.push_back(motor_status);
00460
```

```
00461   diagnostics_pub_->publish(diag_array);
00462 }
00463
00464 double SafetyMonitor::compute_linear_magnitude(const geometry_msgs::msg::Twist & twist) const
00465 {
00466   return std::sqrt(
00467     twist.linear.x * twist.linear.x +
00468     twist.linear.y * twist.linear.y +
00469     twist.linear.z * twist.linear.z);
00470 }
00471
00472 }  // namespace star_safety_monitor
```

9.19 src/star_safety_monitor/src/safety_monitor_node.cpp File Reference

```
#include <memory>
#include <rclcpp/rclcpp.hpp>
#include "star_safety_monitor/safety_monitor.hpp"
```

Include dependency graph for safety_monitor_node.cpp:



Functions

- int main (int argc, char **argv)

9.19.1 Function Documentation

9.19.1.1 main()

```
int main (
            int argc,
            char ** argv)
```

Definition at line 26 of file safety_monitor_node.cpp.

9.20 safety_monitor_node.cpp

Go to the documentation of this file.

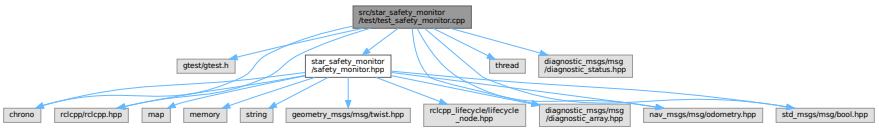
```
00001 // Copyright 2026 STAR Team
00002 // SPDX-License-Identifier: MIT
00003 //
00004 // Permission is hereby granted, free of charge, to any person obtaining a copy
00005 // of this software and associated documentation files (the "Software"), to deal
00006 // in the Software without restriction, including without limitation the rights
00007 // to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00008 // copies of the Software, and to permit persons to whom the Software is
00009 // furnished to do so, subject to the following conditions:
```

```
00010 //
00011 // The above copyright notice and this permission notice shall be included in all
00012 // copies or substantial portions of the Software.
00013 //
00014 // THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00015 // IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00016 // FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00017 // AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00018 // LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00019 // OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
00020 // SOFTWARE.
00021
00022 #include <memory>
00023 #include <rclcpp/rclcpp.hpp>
00024 #include "star_safety_monitor/safety_monitor.hpp"
00025
00026 int main(int argc, char ** argv)
00027 {
00028   rclcpp::init(argc, argv);
00029
00030   auto node = std::make_shared<star_safety_monitor::SafetyMonitor>();
00031
00032   rclcpp::spin(node->get_node_base_interface());
00033
00034   rclcpp::shutdown();
00035   return 0;
00036 }
```

9.21 src/star_safety_monitor/test/test_safety_monitor.cpp File Reference

```
#include <gtest/gtest.h>
#include <chrono>
#include <thread>
#include <rclcpp/rclcpp.hpp>
#include "star_safety_monitor/safety_monitor.hpp"
#include <nav_msgs/msg/odometry.hpp>
#include <diagnostic_msgs/msg/diagnostic_array.hpp>
#include <diagnostic_msgs/msg/diagnostic_status.hpp>
#include <std_msgs/msg/bool.hpp>
```

Include dependency graph for test_safety_monitor.cpp:



Classes

- class SafetyMonitorTest

Functions

- TEST_F (SafetyMonitorTest, NodeConstruction)
- TEST_F (SafetyMonitorTest, LifecycleConfiguration)
- TEST_F (SafetyMonitorTest, LifecycleActivation)
- TEST_F (SafetyMonitorTest, LifecycleDeactivation)
- TEST_F (SafetyMonitorTest, ParameterLoading)

- TEST_F (SafetyMonitorTest, DiagnosticsPublication)
- TEST_F (SafetyMonitorTest, OdometrySubscription)
- TEST_F (SafetyMonitorTest, EmergencyStopTrigger)
- TEST_F (SafetyMonitorTest, DiagnosticPublishing)
- int main (int argc, char ∗∗argv)

9.21.1 Function Documentation

9.21.1.1 main()

```
int main (
            int argc,
            char ** argv)
```

Definition at line 204 of file test_safety_monitor.cpp.

9.21.1.2 TEST_F() [1/9]

```
TEST_F (
            SafetyMonitorTest ,
            DiagnosticPublishing )
```

Definition at line 198 of file test_safety_monitor.cpp.

9.21.1.3 TEST_F() [2/9]

```
TEST_F (
            SafetyMonitorTest ,
            DiagnosticsPublication )
```

Definition at line 123 of file test_safety_monitor.cpp.

9.21.1.4 TEST_F() [3/9]

```
TEST_F (
            SafetyMonitorTest ,
            EmergencyStopTrigger )
```

Definition at line 192 of file test_safety_monitor.cpp.

9.21.1.5 TEST_F() [4/9]

```
TEST_F (
            SafetyMonitorTest ,
            LifecycleActivation )
```

Definition at line 74 of file test_safety_monitor.cpp.

9.21.1.6 TEST_F() [5/9]

```
TEST_F (
            SafetyMonitorTest ,
            LifecycleConfiguration )
```

Definition at line 64 of file test_safety_monitor.cpp.

9.21.1.7 TEST_F() [6/9]

```
TEST_F (
            SafetyMonitorTest ,
            LifecycleDeactivation )
```

Definition at line 88 of file test_safety_monitor.cpp.

9.21.1.8 TEST_F() [7/9]

```
TEST_F (
            SafetyMonitorTest ,
            NodeConstruction )
```

Definition at line 57 of file test_safety_monitor.cpp.

9.21.1.9 TEST_F() [8/9]

```
TEST_F (
            SafetyMonitorTest ,
            OdometrySubscription )
```

Definition at line 154 of file test_safety_monitor.cpp.

9.21.1.10 TEST_F() [9/9]

```
TEST_F (
            SafetyMonitorTest ,
            ParameterLoading )
```

Definition at line 104 of file test_safety_monitor.cpp.

9.22 test_safety_monitor.cpp

Go to the documentation of this file.

```
00001 // Copyright 2026 STAR Team
00002 // SPDX-License-Identifier: MIT
00003 //
00004 // Permission is hereby granted, free of charge, to any person obtaining a copy
00005 // of this software and associated documentation files (the "Software"), to deal
00006 // in the Software without restriction, including without limitation the rights
00007 // to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00008 // copies of the Software, and to permit persons to whom the Software is
00009 // furnished to do so, subject to the following conditions:
00010 //
00011 // The above copyright notice and this permission notice shall be included in all
00012 // copies or substantial portions of the Software.
00013 //
00014 // THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00015 // IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00016 // FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00017 // AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00018 // LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00019 // OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
00020 // SOFTWARE.
00021
00022 #include <gtest/gtest.h>
00023 #include <chrono>
00024 #include <thread>
00025 #include <rclcpp/rclcpp.hpp>
00026 #include "star_safety_monitor/safety_monitor.hpp"
00027 #include <nav_msgs/msg/odometry.hpp>
00028 #include <diagnostic_msgs/msg/diagnostic_array.hpp>
00029 #include <diagnostic_msgs/msg/diagnostic_status.hpp>
00030 #include <std_msgs/msg/bool.hpp>
00031
00032 namespace
00033 {
00034 constexpr auto LIFECYCLE_TRANSITION_DELAY = std::chrono::milliseconds(100);
00035 constexpr auto SPIN_TIMEOUT = std::chrono::milliseconds(100);
00036 constexpr double MESSAGE_WAIT_TIMEOUT_S = 2.0;
00037 }
00038
00039 class SafetyMonitorTest : public ::testing::Test
00040 {
00041 protected:
00042   void SetUp() override
00043   {
00044     if (!rclcpp::ok()) {
00045       rclcpp::init(0, nullptr);
00046     }
00047   }
00048
00049   void TearDown() override
00050   {
00051     if (rclcpp::ok()) {
00052       rclcpp::shutdown();
00053     }
00054   }
00055 };
00056
00057 TEST_F(SafetyMonitorTest, NodeConstruction)
00058 {
00059   auto node = std::make_shared<star_safety_monitor::SafetyMonitor>();
00060   EXPECT_NE(node, nullptr);
00061   EXPECT_EQ(node->get_name(), std::string("safety_monitor"));
00062 }
00063
00064 TEST_F(SafetyMonitorTest, LifecycleConfiguration)
00065 {
00066   auto node = std::make_shared<star_safety_monitor::SafetyMonitor>();
00067
00068   // Test on_configure transition
00069   node->configure();
00070   std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00071   EXPECT_EQ(node->get_current_state().label(), std::string("inactive"));
00072 }
00073
00074 TEST_F(SafetyMonitorTest, LifecycleActivation)
00075 {
00076   auto node = std::make_shared<star_safety_monitor::SafetyMonitor>();
00077
00078   // Configure
00079   node->configure();
00080   std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00081
00082   // Activate
```

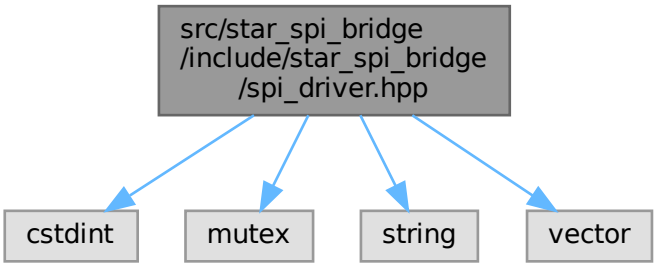
```
00083   node->activate();
00084   std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00085   EXPECT_EQ(node->get_current_state().label(), std::string("active"));
00086 }
00087
00088 TEST_F(SafetyMonitorTest, LifecycleDeactivation)
00089 {
00090   auto node = std::make_shared<star_safety_monitor::SafetyMonitor>();
00091
00092   // Configure and activate
00093   node->configure();
00094   std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00095   node->activate();
00096   std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00097
00098   // Deactivate
00099   node->deactivate();
00100   std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00101   EXPECT_EQ(node->get_current_state().label(), std::string("inactive"));
00102 }
00103
00104 TEST_F(SafetyMonitorTest, ParameterLoading)
00105 {
00106   rclcpp::NodeOptions options;
00107   options.parameter_overrides({
00108     {"heartbeat_timeout_ms", 500},
00109     {"max_linear_velocity", 1.5},
00110     {"max_angular_velocity", 2.5},
00111     {"publish_rate", 20.0},
00112   });
00113
00114   auto node = std::make_shared<star_safety_monitor::SafetyMonitor>(options);
00115   node->configure();
00116   std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00117
00118   // Verify parameters were loaded
00119   auto hb_timeout = node->get_parameter("heartbeat_timeout_ms");
00120   EXPECT_EQ(hb_timeout.as_int(), 500);
00121 }
00122
00123 TEST_F(SafetyMonitorTest, DiagnosticsPublication)
00124 {
00125   auto node = std::make_shared<star_safety_monitor::SafetyMonitor>();
00126   node->configure();
00127   std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00128   node->activate();
00129   std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00130
00131   // Create a test subscriber to receive diagnostics
00132   std::atomic<int> diag_count{0};
00133   auto test_node = rclcpp::Node::make_shared("test_node");
00134   auto diag_sub = test_node->create_subscription<
00135     diagnostic_msgs::msg::DiagnosticArray>(
00136     "/diagnostics", 10,
00137     [&diag_count](const diagnostic_msgs::msg::DiagnosticArray::SharedPtr) {
00138       diag_count++;
00139     });
00140
00141   // Wait for a few diagnostic messages
00142   auto executor = rclcpp::executors::SingleThreadedExecutor();
00143   executor.add_node(test_node->get_node_base_interface());
00144   executor.add_node(node->get_node_base_interface());
00145
00146   rclcpp::Time start_time = node->now();
00147   while (diag_count < 2 && (node->now() - start_time).seconds() < MESSAGE_WAIT_TIMEOUT_S) {
00148     executor.spin_some(SPIN_TIMEOUT);
00149   }
00150
00151   EXPECT_GT(diag_count, 0);
00152 }
00153
00154 TEST_F(SafetyMonitorTest, OdometrySubscription)
00155 {
00156   auto node = std::make_shared<star_safety_monitor::SafetyMonitor>();
00157   node->configure();
00158   std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00159   node->activate();
00160   std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00161
00162   // Create a test publisher for odometry
00163   auto test_node = rclcpp::Node::make_shared("test_odom_pub");
00164   auto odom_pub = test_node->create_publisher<nav_msgs::msg::Odometry>("/odom", 10);
00165
00166   // Create and publish odometry message
00167   auto odom_msg = nav_msgs::msg::Odometry();
00168   odom_msg.header.stamp = node->now();
00169   odom_msg.twist.twist.linear.x = 0.5;
```

```
00170   odom_msg.twist.twist.linear.y = 0.0;
00171   odom_msg.twist.twist.linear.z = 0.0;
00172   odom_msg.twist.twist.angular.x = 0.0;
00173   odom_msg.twist.twist.angular.y = 0.0;
00174   odom_msg.twist.twist.angular.z = 0.1;
00175
00176   auto executor = rclcpp::executors::SingleThreadedExecutor();
00177   executor.add_node(node->get_node_base_interface());
00178   executor.add_node(test_node->get_node_base_interface());
00179
00180   // Publish and spin
00181   odom_pub->publish(odom_msg);
00182   executor.spin_some(SPIN_TIMEOUT);
00183
00184   // Give the node time to process
00185   std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00186   executor.spin_some(SPIN_TIMEOUT);
00187
00188   // Test passes if no exceptions are thrown
00189   EXPECT_TRUE(true);
00190 }
00191
00192 TEST_F(SafetyMonitorTest, EmergencyStopTrigger)
00193 {
00194   // TODO(locked-in): Test E-Stop triggering logic
00195   GTEST_SKIP() << "Emergency stop tests not yet implemented";
00196 }
00197
00198 TEST_F(SafetyMonitorTest, DiagnosticPublishing)
00199 {
00200   // TODO(locked-in): Test diagnostic message generation
00201   GTEST_SKIP() << "Diagnostic publishing tests not yet implemented";
00202 }
00203
00204 int main(int argc, char ** argv)
00205 {
00206   ::testing::InitGoogleTest(&argc, argv);
00207   return RUN_ALL_TESTS();
00208 }
```

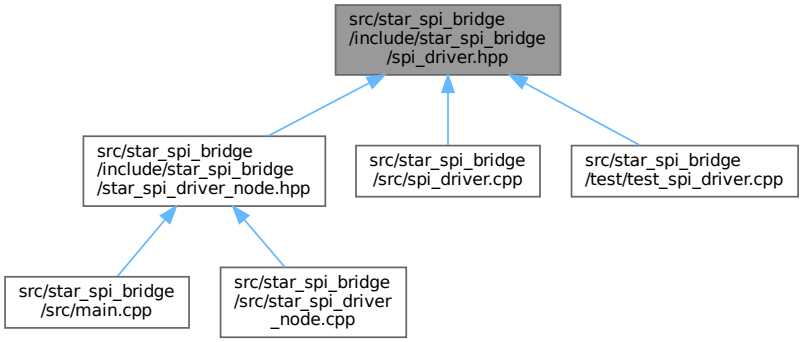
9.23 src/star_spi_bridge/include/star_spi_bridge/spi_driver.hpp File Reference

```
#include <cstdint>
#include <mutex>
#include <string>
#include <vector>
```

Include dependency graph for spi_driver.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class star_spi_bridge::SpiDriver

  SPI driver for framed communication with the RX72N peripheral MCU.

Namespaces

- namespace star_spi_bridge

Enumerations

- enum class star_spi_bridge::FrameType : uint8_t { star_spi_bridge::VelocityCommand = 0x01 , star_spi_bridge::TelemetryData = 0x02 }

  Frame type identifiers for SPI protocol messages.

9.24 spi_driver.hpp

Go to the documentation of this file.

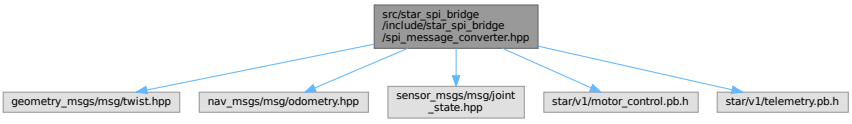
```
00001 // Copyright 2026 Locked Inc.
00002
00003 #ifndef STAR_SPI_BRIDGE__SPI_DRIVER_HPP_
00004 #define STAR_SPI_BRIDGE__SPI_DRIVER_HPP_
00005
00006 #include <cstdint>
00007 #include <mutex>
00008 #include <string>
00009 #include <vector>
00010
00011 namespace star_spi_bridge
00012 {
00013
00027 enum class FrameType : uint8_t
00028 {
00029   VelocityCommand = 0x01,
00030   TelemetryData = 0x02
00031 };
00032
00058 class SpiDriver {
00059 public:
00060   // -- Frame-structure constants ----------------------------------------
```

```
00062   static constexpr size_t k_header_size = 8;
00063
00065   static constexpr size_t k_crc_size = 4;
00066
00068   static constexpr size_t k_max_payload_size = 1024;
00069
00071   static constexpr uint16_t k_sync_word = 0x55AA;
00072
00082   explicit SpiDriver(
00083     const std::string & device_path,
00084     uint32_t speed_hz = 10000000);
00085
00087   virtual ~SpiDriver();
00088
00104   bool initialize();
00105
00111   void close_device();
00112
00127   bool transfer(
00128     const std::vector<uint8_t> & tx_data,
00129     std::vector<uint8_t> & rx_data);
00130
00152   static void encode_frame(
00153     uint16_t seq, FrameType type, uint8_t flags,
00154     const std::vector<uint8_t> & payload,
00155     std::vector<uint8_t> & out_frame);
00156
00178   static bool decode_frame(
00179     const std::vector<uint8_t> & frame, uint16_t & seq,
00180     FrameType & type, uint8_t & flags,
00181     std::vector<uint8_t> & payload);
00182
00194   static uint32_t calculate_crc32(const std::vector<uint8_t> & data);
00195
00196 private:
00197   std::string device_path_;
00198   uint32_t speed_hz_;
00199   int spi_fd_;
00200
00201   static const uint32_t k_crc32_polynomial = 0x04C11DB7;
00202   static constexpr uint8_t k_bits_per_word = 8;
00203   // [SYNC: 0x55AA (2B, LE) - wire: 0xAA, 0x55]
00204   // [SEQ: sequence number (2B, LE)]
00205   // [LEN: payload length (2B, LE)]
00206   // [TYPE: frame type (1B)]
00207   // [FLAGS: control flags (1B)]
00208   // Total header = 2+2+2+1+1 = 8 bytes.
00209
00210   static uint32_t crc32_table_[256];
00211   static std::once_flag crc32_table_init_flag_;
00212   static void init_crc32_table();
00213 };
00214
00215 }  // namespace star_spi_bridge
00216
00217 #endif  // STAR_SPI_BRIDGE__SPI_DRIVER_HPP_
```

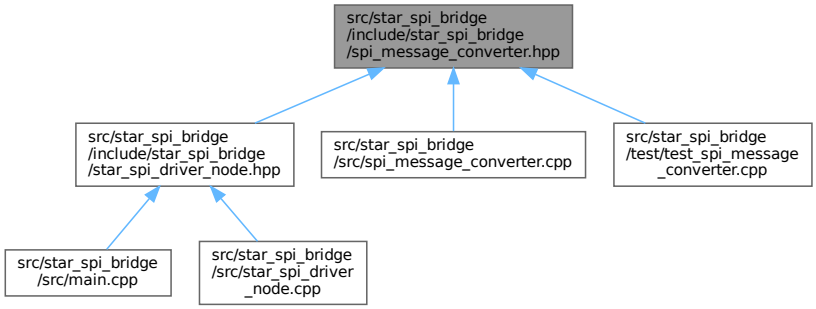
9.25 src/star_spi_bridge/include/star_spi_bridge/spi_message_converter.hpp File Reference

```
#include <geometry_msgs/msg/twist.hpp>
#include <nav_msgs/msg/odometry.hpp>
#include <sensor_msgs/msg/joint_state.hpp>
#include "star/v1/motor_control.pb.h"
#include "star/v1/telemetry.pb.h"
```

Include dependency graph for spi_message_converter.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class star_spi_bridge::SpiMessageConverter
- struct star_spi_bridge::SpiMessageConverter::Parameters

Namespaces

- namespace star_spi_bridge

9.26 spi_message_converter.hpp

Go to the documentation of this file.

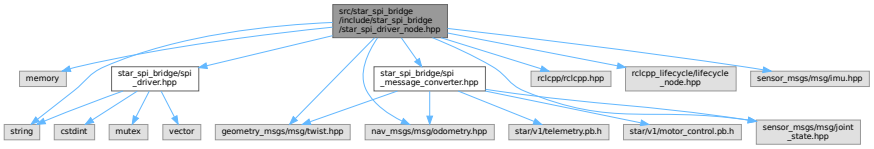
```
00001 // Copyright 2026 Locked Inc.
00002
00003 #ifndef STAR_SPI_BRIDGE__SPI_MESSAGE_CONVERTER_HPP_
00004 #define STAR_SPI_BRIDGE__SPI_MESSAGE_CONVERTER_HPP_
00005
00006 #include <geometry_msgs/msg/twist.hpp>
00007 #include <nav_msgs/msg/odometry.hpp>
00008 #include <sensor_msgs/msg/joint_state.hpp>
00009
00010 #include "star/v1/motor_control.pb.h"
00011 #include "star/v1/telemetry.pb.h"
00012
00013 namespace star_spi_bridge
00014 {
00015
00016 class SpiMessageConverter {
00017 public:
00018   struct Parameters
00019   {
00020     double wheel_base;
00021     double wheel_radius;
00022     int32_t ticks_per_rev;
00023   };
00024
00025   explicit SpiMessageConverter(const Parameters & params);
00026
00027   // ROS2 -> Protobuf
00028   bool twist_to_velocity_command(
00029     const geometry_msgs::msg::Twist & twist,
00030     star::v1::VelocityCommand & command);
00031
00032   // Protobuf -> ROS2
00033   void telemetry_to_odometry(
00034     const star::v1::TelemetryData & telemetry,
00035     nav_msgs::msg::Odometry & odom);
00036   void telemetry_to_joint_state(
```

```
00037     const star::v1::TelemetryData & telemetry,
00038     sensor_msgs::msg::JointState & joint_state);
00039
00040 private:
00041   Parameters params_;
00042
00043   // Odometry state
00044   double x_ = 0.0;
00045   double y_ = 0.0;
00046   double theta_ = 0.0;
00047
00048   // Encoder state for delta calculation
00049   int64_t prev_ticks_fl_ = 0;
00050   int64_t prev_ticks_fr_ = 0;
00051   int64_t prev_ticks_bl_ = 0;
00052   int64_t prev_ticks_br_ = 0;
00053   bool first_odom_msg_ = true;
00054
00055   static double normalize_angle(double angle);
00056 };
00057
00058 }  // namespace star_spi_bridge
00059
00060 #endif  // STAR_SPI_BRIDGE__SPI_MESSAGE_CONVERTER_HPP_
```

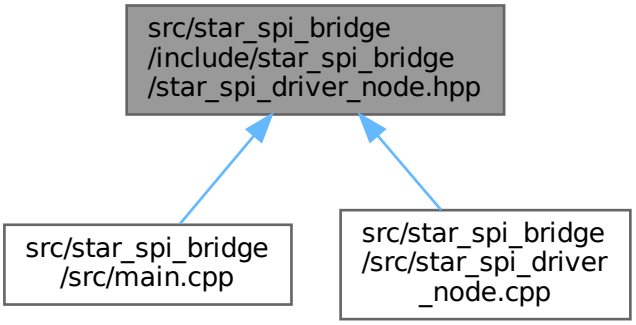
9.27 src/star_spi_bridge/include/star_spi_bridge/star_spi_driver_↩ node.hpp File Reference

```
#include <memory>
#include <string>
#include <geometry_msgs/msg/twist.hpp>
#include <nav_msgs/msg/odometry.hpp>
#include <rclcpp/rclcpp.hpp>
#include <rclcpp_lifecycle/lifecycle_node.hpp>
#include <sensor_msgs/msg/imu.hpp>
#include <sensor_msgs/msg/joint_state.hpp>
#include "star_spi_bridge/spi_driver.hpp"
#include "star_spi_bridge/spi_message_converter.hpp"
```

Include dependency graph for star_spi_driver_node.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class star_spi_bridge::StarSpiDriverNode

Namespaces

- namespace star_spi_bridge

9.28 star_spi_driver_node.hpp

Go to the documentation of this file.

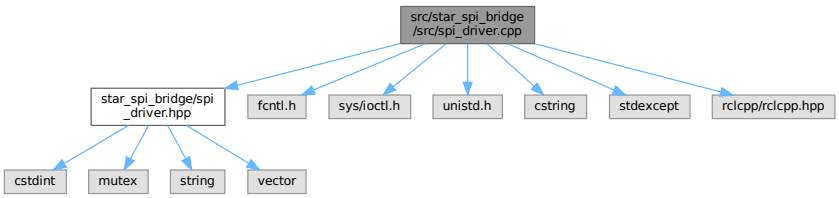
```
00001 // Copyright 2026 Locked Inc.
00002
00003 #ifndef STAR_SPI_BRIDGE__STAR_SPI_DRIVER_NODE_HPP_
00004 #define STAR_SPI_BRIDGE__STAR_SPI_DRIVER_NODE_HPP_
00005
00006 #include <memory>
00007 #include <string>
00008
00009 #include <geometry_msgs/msg/twist.hpp>
00010 #include <nav_msgs/msg/odometry.hpp>
00011 #include <rclcpp/rclcpp.hpp>
00012 #include <rclcpp_lifecycle/lifecycle_node.hpp>
00013 #include <sensor_msgs/msg/imu.hpp>
00014 #include <sensor_msgs/msg/joint_state.hpp>
00015
00016 #include "star_spi_bridge/spi_driver.hpp"
00017 #include "star_spi_bridge/spi_message_converter.hpp"
00018
00019 namespace star_spi_bridge
00020 {
00021
00022 class StarSpiDriverNode : public rclcpp_lifecycle::LifecycleNode {
00023 public:
00024   explicit StarSpiDriverNode(
00025     const rclcpp::NodeOptions & options = rclcpp::NodeOptions());
00026   ~StarSpiDriverNode() override;
00027
00028   // Lifecycle transitions
00029   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00030   on_configure(const rclcpp_lifecycle::State & prev_state) override;
00031   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00032   on_activate(const rclcpp_lifecycle::State & prev_state) override;
```

```
00033   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00034   on_deactivate(const rclcpp_lifecycle::State & prev_state) override;
00035   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00036   on_cleanup(const rclcpp_lifecycle::State & prev_state) override;
00037   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00038   on_shutdown(const rclcpp_lifecycle::State & prev_state) override;
00039
00040 private:
00041   // Callbacks
00042   void cmd_vel_callback(const geometry_msgs::msg::Twist::SharedPtr msg);
00043   void timer_callback();  // 100 Hz loop
00044
00045   // Components
00046   std::unique_ptr<SpiDriver> spi_driver_;
00047   std::unique_ptr<SpiMessageConverter> converter_;
00048
00049   // ROS handles
00050   rclcpp::Subscription<geometry_msgs::msg::Twist>::SharedPtr cmd_vel_sub_;
00051   rclcpp_lifecycle::LifecyclePublisher<nav_msgs::msg::Odometry>::SharedPtr
00052     odom_pub_;
00053   rclcpp_lifecycle::LifecyclePublisher<sensor_msgs::msg::JointState>::SharedPtr
00054     joint_state_pub_;
00055   rclcpp_lifecycle::LifecyclePublisher<sensor_msgs::msg::Imu>::SharedPtr
00056     imu_pub_;
00057   rclcpp::TimerBase::SharedPtr timer_;
00058
00059   // State
00060   geometry_msgs::msg::Twist current_cmd_vel_;
00061   rclcpp::Time last_cmd_vel_time_;
00062   uint16_t tx_seq_ = 0;
00063
00064   // Parameters
00065   std::string spi_device_path_;
00066   int spi_speed_hz_;
00067   int cmd_vel_timeout_ms_;
00068 };
00069
00070 }  // namespace star_spi_bridge
00071
00072 #endif  // STAR_SPI_BRIDGE__STAR_SPI_DRIVER_NODE_HPP_
```

9.29 src/star_spi_bridge/src/spi_driver.cpp File Reference

```
#include "star_spi_bridge/spi_driver.hpp"
#include <fcntl.h>
#include <sys/ioctl.h>
#include <unistd.h>
#include <cstring>
#include <stdexcept>
#include <rclcpp/rclcpp.hpp>
```

Include dependency graph for spi_driver.cpp:



Classes

- struct spi_ioc_transfer

Namespaces

- namespace star_spi_bridge

Macros

- #define SPI_IOC_MESSAGE(N)

Variables

- constexpr uint8_t SPI_MODE_0 = 0
- constexpr int SPI_IOC_WR_MODE = 0
- constexpr int SPI_IOC_WR_BITS_PER_WORD = 0
- constexpr int SPI_IOC_WR_MAX_SPEED_HZ = 0

9.29.1 Macro Definition Documentation

9.29.1.1 SPI_IOC_MESSAGE

```
#define SPI_IOC_MESSAGE(
            N)
```

Value:

```
0
```

Definition at line 22 of file spi_driver.cpp.

Referenced by star_spi_bridge::SpiDriver::transfer().

9.29.2 Variable Documentation

9.29.2.1 SPI_IOC_WR_BITS_PER_WORD

```
int SPI_IOC_WR_BITS_PER_WORD = 0  [constexpr]
```

Definition at line 25 of file spi_driver.cpp.

Referenced by star_spi_bridge::SpiDriver::initialize().

9.29.2.2 SPI_IOC_WR_MAX_SPEED_HZ

```
int SPI_IOC_WR_MAX_SPEED_HZ = 0  [constexpr]
```

Definition at line 26 of file spi_driver.cpp.

Referenced by star_spi_bridge::SpiDriver::initialize().

9.29.2.3 SPI_IOC_WR_MODE

```
int SPI_IOC_WR_MODE = 0  [constexpr]
```

Definition at line 24 of file spi_driver.cpp.

Referenced by star_spi_bridge::SpiDriver::initialize().

9.29.2.4 SPI_MODE_0

```
uint8_t SPI_MODE_0 = 0  [constexpr]
```

Definition at line 23 of file spi_driver.cpp.

Referenced by star_spi_bridge::SpiDriver::initialize().

9.30 spi_driver.cpp

Go to the documentation of this file.

```
00001 // Copyright 2026 Locked Inc.
00002
00003 #include "star_spi_bridge/spi_driver.hpp"
00004
00005 #include <fcntl.h>
00006 #include <sys/ioctl.h>
00007 #include <unistd.h>
00008
00009 #include <cstring>
00010 #include <stdexcept>
00011
00012 #include <rclcpp/rclcpp.hpp>
00013
00014 #ifdef __linux__
00015 #include <linux/spi/spidev.h>
00016 #else
00017 // Mock spidev headers if on macOS/non-Linux to allow compilation (for
00018 // development/analysis only)
00019 // Ideally this code runs on Linux/RPi5. On macOS, these headers likely
00020 // don't exist or are different.
00021 #ifndef SPI_IOC_MESSAGE
00022 #define SPI_IOC_MESSAGE(N) 0
00023 constexpr uint8_t SPI_MODE_0 = 0;
00024 constexpr int SPI_IOC_WR_MODE = 0;
00025 constexpr int SPI_IOC_WR_BITS_PER_WORD = 0;
00026 constexpr int SPI_IOC_WR_MAX_SPEED_HZ = 0;
00027
00028 struct spi_ioc_transfer
00029 {
00030   uint64_t tx_buf_;
00031   uint64_t rx_buf_;
00032   uint32_t len_;
00033   uint32_t speed_hz_;
00034   uint16_t delay_usecs_;
00035   uint8_t bits_per_word_;
00036   uint8_t cs_change_;
00037   uint32_t pad_;
00038 };
00039 #endif
00040 #endif
00041
00042 namespace star_spi_bridge
00043 {
00044
00045 namespace
00046 {
00048 auto logger()
00049 {
00050   return rclcpp::get_logger("star_spi_bridge.spi_driver");
00051 }
00052
```

```
00053 inline void set_spi_xfer_tx_buf(spi_ioc_transfer & xfer, uint64_t value)
00054 {
00055 #ifdef __linux__
00056   xfer.tx_buf = value;
00057 #else
00058   xfer.tx_buf_ = value;
00059 #endif
00060 }
00061
00062 inline void set_spi_xfer_rx_buf(spi_ioc_transfer & xfer, uint64_t value)
00063 {
00064 #ifdef __linux__
00065   xfer.rx_buf = value;
00066 #else
00067   xfer.rx_buf_ = value;
00068 #endif
00069 }
00070
00071 inline void set_spi_xfer_len(spi_ioc_transfer & xfer, uint32_t value)
00072 {
00073 #ifdef __linux__
00074   xfer.len = value;
00075 #else
00076   xfer.len_ = value;
00077 #endif
00078 }
00079
00080 inline void set_spi_xfer_speed_hz(spi_ioc_transfer & xfer, uint32_t value)
00081 {
00082 #ifdef __linux__
00083   xfer.speed_hz = value;
00084 #else
00085   xfer.speed_hz_ = value;
00086 #endif
00087 }
00088
00089 inline void set_spi_xfer_bits_per_word(spi_ioc_transfer & xfer, uint8_t value)
00090 {
00091 #ifdef __linux__
00092   xfer.bits_per_word = value;
00093 #else
00094   xfer.bits_per_word_ = value;
00095 #endif
00096 }
00097 }  // namespace
00098
00099 uint32_t SpiDriver::crc32_table_[256];
00100 std::once_flag SpiDriver::crc32_table_init_flag_;
00101
00102 SpiDriver::SpiDriver(const std::string & device_path, uint32_t speed_hz)
00103 : device_path_(device_path), speed_hz_(speed_hz), spi_fd_(-1)
00104 {
00105   std::call_once(crc32_table_init_flag_, init_crc32_table);
00106 }
00107
00108 SpiDriver::~SpiDriver()
00109 {
00110   close_device();
00111 }
00112
00113 bool SpiDriver::initialize()
00114 {
00115 #ifndef __linux__
00116   RCLCPP_ERROR(logger(), "SPI driver is only supported on Linux platforms");
00117   return false;
00118 #endif
00119   // Open SPI device
00120   spi_fd_ = open(device_path_.c_str(), O_RDWR);
00121   if (spi_fd_ < 0) {
00122     RCLCPP_ERROR(logger(), "Failed to open SPI device: %s (%s)", device_path_.c_str(),
00123       strerror(errno));
00124     return false;
00125   }
00126
00127   // Configure SPI mode (Mode 0: CPOL=0, CPHA=0)
00128   uint8_t mode = SPI_MODE_0;
00129   if (ioctl(spi_fd_, SPI_IOC_WR_MODE, &mode) < 0) {
00130     RCLCPP_ERROR(logger(), "Failed to set SPI mode");
00131     close_device();
00132     return false;
00133   }
00134
00135   // Configure bits per word (8 bits)
00136   uint8_t bits = k_bits_per_word;
00137   if (ioctl(spi_fd_, SPI_IOC_WR_BITS_PER_WORD, &bits) < 0) {
00138     RCLCPP_ERROR(logger(), "Failed to set SPI bits per word");
00139     close_device();
```

```
00140     return false;
00141   }
00142
00143   // Configure max speed
00144   if (ioctl(spi_fd_, SPI_IOC_WR_MAX_SPEED_HZ, &speed_hz_) < 0) {
00145     RCLCPP_ERROR(logger(), "Failed to set SPI max speed");
00146     close_device();
00147     return false;
00148   }
00149
00150   return true;
00151 }
00152
00153 void SpiDriver::close_device()
00154 {
00155   if (spi_fd_ >= 0) {
00156     close(spi_fd_);
00157     spi_fd_ = -1;
00158   }
00159 }
00160
00161 bool SpiDriver::transfer(const std::vector<uint8_t> & tx_data, std::vector<uint8_t> & rx_data)
00162 {
00163 #ifndef __linux__
00164   RCLCPP_ERROR(logger(), "SPI driver is only supported on Linux platforms");
00165   return false;
00166 #endif
00167   if (spi_fd_ < 0) {
00168     return false;
00169   }
00170
00171   if (rx_data.size() != tx_data.size()) {
00172     rx_data.resize(tx_data.size());
00173   }
00174
00175   struct spi_ioc_transfer xfer;
00176   std::memset(&xfer, 0, sizeof(xfer));
00177
00178   set_spi_xfer_tx_buf(xfer, static_cast<uint64_t>(reinterpret_cast<uintptr_t>(tx_data.data())));
00179   set_spi_xfer_rx_buf(xfer, static_cast<uint64_t>(reinterpret_cast<uintptr_t>(rx_data.data())));
00180   set_spi_xfer_len(xfer, static_cast<uint32_t>(tx_data.size()));
00181   set_spi_xfer_speed_hz(xfer, speed_hz_);
00182   set_spi_xfer_bits_per_word(xfer, k_bits_per_word);
00183
00184   int ret = ioctl(spi_fd_, SPI_IOC_MESSAGE(1), &xfer);
00185   if (ret < 0) {
00186     RCLCPP_ERROR(logger(), "SPI transfer failed: %s", strerror(errno));
00187     return false;
00188   }
00189
00190   return true;
00191 }
00192
00193 void SpiDriver::encode_frame(
00194   uint16_t seq,
00195   FrameType type,
00196   uint8_t flags,
00197   const std::vector<uint8_t> & payload,
00198   std::vector<uint8_t> & out_frame)
00199 {
00200   if (payload.size() > k_max_payload_size) {
00201     throw std::invalid_argument("payload exceeds maximum size of 1024 bytes");
00202   }
00203
00204   // [SYNC(2)][SEQ(2)][LEN(2)][TYPE(1)][FLAGS(1)][PAYLOAD(N)][CRC(4)]
00205   size_t frame_size = k_header_size + payload.size() + k_crc_size;
00206   out_frame.clear();
00207   out_frame.reserve(frame_size);
00208
00209   // SYNC: 0x55AA (Little Endian)
00210   // Note: Header fields (SYNC, SEQ, LEN) use little-endian byte order
00211   // Wire format: [0xAA, 0x55] for 0x55AA
00212   out_frame.push_back(k_sync_word & 0xFF);         // LSB first (0xAA)
00213   out_frame.push_back((k_sync_word >> 8) & 0xFF);  // MSB second (0x55)
00214
00215   // SEQ (Little Endian)
00216   out_frame.push_back(seq & 0xFF);         // LSB first
00217   out_frame.push_back((seq >> 8) & 0xFF);  // MSB second
00218
00219   // LEN (Little Endian)
00220   uint16_t len = static_cast<uint16_t>(payload.size());
00221   out_frame.push_back(len & 0xFF);         // LSB first
00222   out_frame.push_back((len >> 8) & 0xFF);  // MSB second
00223
00224   // TYPE
00225   out_frame.push_back(static_cast<uint8_t>(type));
00226
```

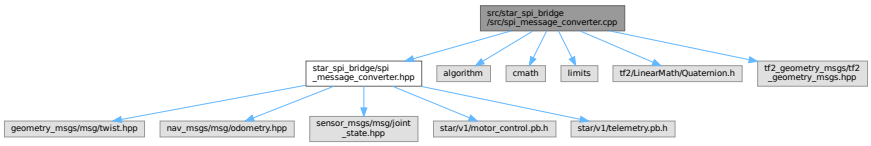
```
00227   // FLAGS
00228   out_frame.push_back(flags);
00229
00230   // PAYLOAD
00231   out_frame.insert(out_frame.end(), payload.begin(), payload.end());
00232
00233   // CRC-32 (IEEE 802.3) - Calculated over Header + Payload
00234   uint32_t crc = calculate_crc32(out_frame);
00235
00236   // Append CRC (Little Endian as per spec "CRC-32: IEEE 802.3 (4B, LE)")
00237   out_frame.push_back(crc & 0xFF);
00238   out_frame.push_back((crc » 8) & 0xFF);
00239   out_frame.push_back((crc » 16) & 0xFF);
00240   out_frame.push_back((crc » 24) & 0xFF);
00241 }
00242
00243 bool SpiDriver::decode_frame(
00244   const std::vector<uint8_t> & frame,
00245   uint16_t & seq,
00246   FrameType & type,
00247   uint8_t & flags,
00248   std::vector<uint8_t> & payload)
00249 {
00250   if (frame.size() < k_header_size + k_crc_size) {
00251     return false;
00252   }
00253
00254   // Check SYNC (Little Endian: [0xAA, 0x55] for 0x55AA)
00255   if (frame[0] != (k_sync_word & 0xFF) || frame[1] != ((k_sync_word » 8) & 0xFF)) {
00256     return false;
00257   }
00258
00259   // Extract Length (Little Endian: LSB first)
00260   uint16_t len = frame[4] | (static_cast<uint16_t>(frame[5]) « 8);
00261
00262   // Validate Frame Size
00263   // Header(8) + Payload(len) + CRC(4)
00264   if (frame.size() != k_header_size + len + k_crc_size) {
00265     return false;
00266   }
00267
00268   // Verify CRC
00269   // Calculate CRC over Header + Payload (bytes 0 to end-4)
00270   std::vector<uint8_t> data_to_check(frame.begin(), frame.end() - k_crc_size);
00271   uint32_t calculated_crc = calculate_crc32(data_to_check);
00272
00273   uint32_t received_crc = static_cast<uint32_t>(frame[frame.size() - 4]) |
00274     (static_cast<uint32_t>(frame[frame.size() - 3]) « 8) |
00275     (static_cast<uint32_t>(frame[frame.size() - 2]) « 16) |
00276     (static_cast<uint32_t>(frame[frame.size() - 1]) « 24);
00277
00278   if (calculated_crc != received_crc) {
00279     return false;
00280   }
00281
00282   // Extract fields (Little Endian: LSB first)
00283   seq = frame[2] | (static_cast<uint16_t>(frame[3]) « 8);
00284   type = static_cast<FrameType>(frame[6]);
00285   flags = frame[7];
00286
00287   payload.assign(frame.begin() + k_header_size, frame.end() - k_crc_size);
00288
00289   return true;
00290 }
00291
00292 void SpiDriver::init_crc32_table()
00293 {
00294   // Generate CRC-32 lookup table using IEEE 802.3 polynomial (reflected form)
00295   // Polynomial 0xEDB88320 is the bit-reversed form of 0x04C11DB7
00296   // This produces standard CRC-32 values (e.g., "123456789" -> 0xCBF43926)
00297   constexpr uint32_t polynomial = 0xEDB88320;
00298   for (uint32_t i = 0; i < 256; i++) {
00299     uint32_t crc = i;
00300     for (uint32_t j = 0; j < 8; j++) {
00301       if (crc & 1) {
00302         crc = (crc » 1) ^ polynomial;
00303       } else {
00304         crc »= 1;
00305       }
00306     }
00307     crc32_table_[i] = crc;
00308   }
00309 }
00310
00311 uint32_t SpiDriver::calculate_crc32(const std::vector<uint8_t> & data)
00312 {
00313   std::call_once(crc32_table_init_flag_, init_crc32_table);
```

```
00314   uint32_t crc = 0xFFFFFFFF;
00315   for (uint8_t byte : data) {
00316     uint8_t lookup_index = (crc ^ byte) & 0xFF;
00317     crc = (crc » 8) ^ crc32_table_[lookup_index];
00318   }
00319   return crc ^ 0xFFFFFFFF;
00320 }
00321
00322 }  // namespace star_spi_bridge
```

9.31 src/star_spi_bridge/src/spi_message_converter.cpp File Reference

```
#include "star_spi_bridge/spi_message_converter.hpp"
#include <algorithm>
#include <cmath>
#include <limits>
#include <tf2/LinearMath/Quaternion.h>
#include <tf2_geometry_msgs/tf2_geometry_msgs.hpp>
```

Include dependency graph for spi_message_converter.cpp:



Namespaces

- namespace star_spi_bridge

9.32 spi_message_converter.cpp

Go to the documentation of this file.

```
00001 // Copyright 2026 Locked Inc.
00002
00003 #include "star_spi_bridge/spi_message_converter.hpp"
00004
00005 #include <algorithm>
00006 #include <cmath>
00007 #include <limits>
00008 #include <tf2/LinearMath/Quaternion.h>  // NOLINT(build/include_order)
00009 #include <tf2_geometry_msgs/tf2_geometry_msgs.hpp>
00010
00011 namespace star_spi_bridge
00012 {
00013
00014 SpiMessageConverter::SpiMessageConverter(const Parameters & params)
00015 : params_(params) {}
00016
00017 bool SpiMessageConverter::twist_to_velocity_command(
00018   const geometry_msgs::msg::Twist & twist,
00019   star::v1::VelocityCommand & command)
00020 {
00021   if (!std::isfinite(twist.linear.x) || !std::isfinite(twist.angular.z)) {
00022     return false;
00023   }
00024
00025   double linear_x = twist.linear.x;
00026   double angular_z = twist.angular.z;
00027
00028   // Differential drive kinematics
```

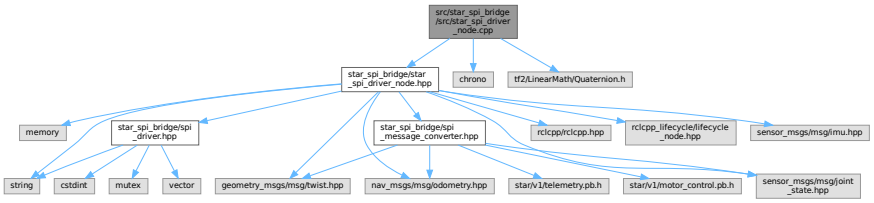
```
00029   // v_r = v + (w * L / 2)
00030   // v_l = v - (w * L / 2)
00031   double right_vel = linear_x + (angular_z * params_.wheel_base / 2.0);
00032   double left_vel = linear_x - (angular_z * params_.wheel_base / 2.0);
00033
00034   // Clamp to hardware limits (e.g. +/- 2.0 m/s as per spec)
00035   // Although the firmware should also handle this, good to be safe.
00036   // The spec says: "Velocity Limits: +/-2.0 m/s per wheel"
00037   const double k_max_vel = 2.0;
00038   right_vel = std::clamp(right_vel, -k_max_vel, k_max_vel);
00039   left_vel = std::clamp(left_vel, -k_max_vel, k_max_vel);
00040
00041   command.set_front_left_velocity_mps(static_cast<float>(left_vel));
00042   command.set_back_left_velocity_mps(static_cast<float>(left_vel));
00043   command.set_front_right_velocity_mps(static_cast<float>(right_vel));
00044   command.set_back_right_velocity_mps(static_cast<float>(right_vel));
00045
00046   return true;
00047 }
00048
00049 void SpiMessageConverter::telemetry_to_odometry(
00050   const star::v1::TelemetryData & telemetry,
00051   nav_msgs::msg::Odometry & odom)
00052 {
00053   // Extract encoder data from telemetry
00054   const auto & enc_fl = telemetry.encoder_front_left();
00055   const auto & enc_fr = telemetry.encoder_front_right();
00056   const auto & enc_bl = telemetry.encoder_back_left();
00057   const auto & enc_br = telemetry.encoder_back_right();
00058
00059   // Encoder ticks are cumulative; we compute deltas between messages
00060   int64_t ticks_fl = enc_fl.ticks();
00061   int64_t ticks_fr = enc_fr.ticks();
00062   int64_t ticks_bl = enc_bl.ticks();
00063   int64_t ticks_br = enc_br.ticks();
00064
00065   if (first_odom_msg_) {
00066     prev_ticks_fl_ = ticks_fl;
00067     prev_ticks_fr_ = ticks_fr;
00068     prev_ticks_bl_ = ticks_bl;
00069     prev_ticks_br_ = ticks_br;
00070     first_odom_msg_ = false;
00071     return;  // No movement to report on first message
00072   }
00073
00074   // Calculate deltas
00075   int64_t delta_fl = ticks_fl - prev_ticks_fl_;
00076   int64_t delta_fr = ticks_fr - prev_ticks_fr_;
00077   int64_t delta_bl = ticks_bl - prev_ticks_bl_;
00078   int64_t delta_br = ticks_br - prev_ticks_br_;
00079
00080   // Update previous
00081   prev_ticks_fl_ = ticks_fl;
00082   prev_ticks_fr_ = ticks_fr;
00083   prev_ticks_bl_ = ticks_bl;
00084   prev_ticks_br_ = ticks_br;
00085
00086   // Convert to distance
00087   double m_per_tick = (2.0 * M_PI * params_.wheel_radius) / params_.ticks_per_rev;
00088
00089   double dist_fl = delta_fl * m_per_tick;
00090   double dist_fr = delta_fr * m_per_tick;
00091   double dist_bl = delta_bl * m_per_tick;
00092   double dist_br = delta_br * m_per_tick;
00093
00094   double dist_left = (dist_fl + dist_bl) / 2.0;
00095   double dist_right = (dist_fr + dist_br) / 2.0;
00096
00097   double dist_center = (dist_right + dist_left) / 2.0;
00098   double delta_theta = (dist_right - dist_left) / params_.wheel_base;
00099
00100   // Update pose
00101   double avg_theta = theta_ + delta_theta / 2.0;
00102   x_ += dist_center * std::cos(avg_theta);
00103   y_ += dist_center * std::sin(avg_theta);
00104   theta_ = normalize_angle(theta_ + delta_theta);
00105
00106   // Populate Odometry message
00107   // Note: Header timestamp should be set by the caller (node)
00108   odom.header.frame_id = "odom";
00109   odom.child_frame_id = "base_link";
00110
00111   odom.pose.pose.position.x = x_;
00112   odom.pose.pose.position.y = y_;
00113   odom.pose.pose.position.z = 0.0;
00114
00115   tf2::Quaternion q;
```

```
00116   q.setRPY(0, 0, theta_);
00117   odom.pose.pose.orientation.x = q.x();
00118   odom.pose.pose.orientation.y = q.y();
00119   odom.pose.pose.orientation.z = q.z();
00120   odom.pose.pose.orientation.w = q.w();
00121
00122   // Populate twist from encoder-reported velocities when available
00123   double v_fl = enc_fl.velocity_mps();
00124   double v_fr = enc_fr.velocity_mps();
00125   double v_bl = enc_bl.velocity_mps();
00126   double v_br = enc_br.velocity_mps();
00127
00128   bool has_velocity_data = (v_fl != 0.0 || v_fr != 0.0 || v_bl != 0.0 || v_br != 0.0);
00129   if (has_velocity_data) {
00130     double v_left = (v_fl + v_bl) / 2.0;
00131     double v_right = (v_fr + v_br) / 2.0;
00132
00133     double v_linear = (v_right + v_left) / 2.0;
00134     double v_angular = (v_right - v_left) / params_.wheel_base;
00135
00136     odom.twist.twist.linear.x = v_linear;
00137     odom.twist.twist.angular.z = v_angular;
00138   }
00139
00140   // Pose covariance -- row-major 6x6 (x, y, z, roll, pitch, yaw)
00141   // Zero covariance makes robot_localization unstable; use realistic dead-reckoning values.
00142   odom.pose.covariance[0] = 0.1;     // x
00143   odom.pose.covariance[7] = 0.1;     // y
00144   odom.pose.covariance[14] = 1e6;    // z (not observed)
00145   odom.pose.covariance[21] = 1e6;    // roll (not observed)
00146   odom.pose.covariance[28] = 1e6;    // pitch (not observed)
00147   odom.pose.covariance[35] = 0.1;    // yaw
00148
00149   // Twist covariance -- same layout
00150   odom.twist.covariance[0] = 0.05;   // vx
00151   odom.twist.covariance[7] = 1e6;    // vy (not observed -- diff drive constraint)
00152   odom.twist.covariance[14] = 1e6;   // vz
00153   odom.twist.covariance[21] = 1e6;   // roll rate
00154   odom.twist.covariance[28] = 1e6;   // pitch rate
00155   odom.twist.covariance[35] = 0.05;  // angular z (yaw rate)
00156 }
00157
00158 void SpiMessageConverter::telemetry_to_joint_state(
00159   const star::v1::TelemetryData & telemetry,
00160   sensor_msgs::msg::JointState & joint_state)
00161 {
00162   // Extract encoder data from telemetry
00163   const auto & enc_fl = telemetry.encoder_front_left();
00164   const auto & enc_fr = telemetry.encoder_front_right();
00165   const auto & enc_bl = telemetry.encoder_back_left();
00166   const auto & enc_br = telemetry.encoder_back_right();
00167
00168   joint_state.name = {
00169     "front_left_wheel", "front_right_wheel", "back_left_wheel", "back_right_wheel"};
00170
00171   // Position (radians)
00172   double rad_per_tick = (2.0 * M_PI) / params_.ticks_per_rev;
00173   joint_state.position = {static_cast<double>(enc_fl.ticks()) * rad_per_tick,
00174     static_cast<double>(enc_fr.ticks()) * rad_per_tick,
00175     static_cast<double>(enc_bl.ticks()) * rad_per_tick,
00176     static_cast<double>(enc_br.ticks()) * rad_per_tick};
00177
00178   // Velocity (rad/s)
00179   // velocity_mps = radius * angular_velocity_rad_s
00180   // angular_velocity = velocity_mps / radius
00181   double inv_radius = 1.0 / params_.wheel_radius;
00182   joint_state.velocity = {enc_fl.velocity_mps() * inv_radius,
00183     enc_fr.velocity_mps() * inv_radius,
00184     enc_bl.velocity_mps() * inv_radius,
00185     enc_br.velocity_mps() * inv_radius};
00186 }
00187
00188 double SpiMessageConverter::normalize_angle(double angle)
00189 {
00190   while (angle > M_PI) {
00191     angle -= 2.0 * M_PI;
00192   }
00193   while (angle < -M_PI) {
00194     angle += 2.0 * M_PI;
00195   }
00196   return angle;
00197 }
00198
00199 }  // namespace star_spi_bridge
```

9.33 src/star_spi_bridge/src/star_spi_driver_node.cpp File Reference

```
#include "star_spi_bridge/star_spi_driver_node.hpp"
#include <chrono>
#include <tf2/LinearMath/Quaternion.h>
```

Include dependency graph for star_spi_driver_node.cpp:



Namespaces

- namespace star_spi_bridge

Variables

- static constexpr int star_spi_bridge::kLogThrottleMs = 1000
- static constexpr double star_spi_bridge::kImuOrientationVar = 0.01
- static constexpr double star_spi_bridge::kImuAngularVelocityVar = 0.001
- static constexpr double star_spi_bridge::kImuLinearAccelVar = 0.01

9.34 star_spi_driver_node.cpp

Go to the documentation of this file.

```
00001 // Copyright 2026 Locked Inc.
00002
00003 #include "star_spi_bridge/star_spi_driver_node.hpp"
00004
00005 #include <chrono>
00006
00007 #include <tf2/LinearMath/Quaternion.h>  // NOLINT(build/include_order)
00008
00009 using namespace std::chrono_literals;
00010
00011 namespace star_spi_bridge
00012 {
00013 static constexpr int kLogThrottleMs = 1000;
00014
00015 // IMU sensor noise model (variance = sigma^2, all diagonal).
00016 // sigma_orientation ~5.7 deg, sigma_angular_vel ~0.032 rad/s, sigma_accel ~0.1 m/s^2
00017 static constexpr double kImuOrientationVar = 0.01;      // rad^2
00018 static constexpr double kImuAngularVelocityVar = 0.001; // (rad/s)^2
00019 static constexpr double kImuLinearAccelVar = 0.01;      // (m/s^2)^2
00020
00021 StarSpiDriverNode::StarSpiDriverNode(const rclcpp::NodeOptions & options)
00022 : rclcpp_lifecycle::LifecycleNode("star_spi_driver", options)
00023 {
00024   // Declare parameters
00025   declare_parameter("spi_device_path", "/dev/spidev0.0");
00026   declare_parameter("spi_speed_hz", 10000000);  // 10 MHz
00027   declare_parameter("cmd_vel_timeout_ms", 500);
00028   declare_parameter("wheel_base", 0.150);
00029   declare_parameter("wheel_radius", 0.0325);
00030   declare_parameter("ticks_per_rev", 11599);
00031 }
```

```
00032
00033 StarSpiDriverNode::~StarSpiDriverNode()
00034 {
00035   // Ensure cleanup
00036   if (spi_driver_) {
00037     spi_driver_->close_device();
00038   }
00039 }
00040
00041 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00042 StarSpiDriverNode::on_configure(const rclcpp_lifecycle::State &)
00043 {
00044   RCLCPP_INFO(get_logger(), "Configuring StarSpiDriverNode...");
00045
00046   // Load parameters
00047   spi_device_path_ = get_parameter("spi_device_path").as_string();
00048   spi_speed_hz_ = get_parameter("spi_speed_hz").as_int();
00049   cmd_vel_timeout_ms_ = get_parameter("cmd_vel_timeout_ms").as_int();
00050
00051   SpiMessageConverter::Parameters converter_params;
00052   converter_params.wheel_base = get_parameter("wheel_base").as_double();
00053   converter_params.wheel_radius = get_parameter("wheel_radius").as_double();
00054   converter_params.ticks_per_rev = get_parameter("ticks_per_rev").as_int();
00055
00056   // Initialize components
00057   spi_driver_ = std::make_unique<SpiDriver>(spi_device_path_, spi_speed_hz_);
00058   converter_ = std::make_unique<SpiMessageConverter>(converter_params);
00059
00060   if (!spi_driver_->initialize()) {
00061     RCLCPP_ERROR(get_logger(), "Failed to initialize SPI driver on %s",
00062                  spi_device_path_.c_str());
00063     return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
00064            CallbackReturn::FAILURE;
00065   }
00066
00067   // Create publishers
00068   odom_pub_ = create_publisher<nav_msgs::msg::Odometry>("odom/unfiltered", 10);
00069   joint_state_pub_ =
00070     create_publisher<sensor_msgs::msg::JointState>("joint_states", 10);
00071   imu_pub_ = create_publisher<sensor_msgs::msg::Imu>("imu/data", 10);
00072   // Create subscription
00073   cmd_vel_sub_ = create_subscription<geometry_msgs::msg::Twist>(
00074       "cmd_vel", 10,
00075       std::bind(&StarSpiDriverNode::cmd_vel_callback, this,
00076                 std::placeholders::_1));
00077
00078   // Initialize state
00079   last_cmd_vel_time_ = get_clock()->now();
00080
00081   return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
00082          CallbackReturn::SUCCESS;
00083 }
00084
00085 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00086 StarSpiDriverNode::on_activate(const rclcpp_lifecycle::State &)
00087 {
00088   RCLCPP_INFO(get_logger(), "Activating StarSpiDriverNode...");
00089
00090   odom_pub_->on_activate();
00091   joint_state_pub_->on_activate();
00092   imu_pub_->on_activate();
00093
00094   // Start 100 Hz timer (10ms)
00095   timer_ = create_wall_timer(
00096       10ms, std::bind(&StarSpiDriverNode::timer_callback, this));
00097
00098   return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
00099          CallbackReturn::SUCCESS;
00100 }
00101
00102 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00103 StarSpiDriverNode::on_deactivate(const rclcpp_lifecycle::State &)
00104 {
00105   RCLCPP_INFO(get_logger(), "Deactivating StarSpiDriverNode...");
00106
00107   // Stop timer
00108   timer_.reset();
00109
00110   // Send zero velocity for safety
00111   // Construct zero command
00112   star::v1::VelocityCommand cmd;
00113   cmd.set_front_left_velocity_mps(0.0f);
00114   cmd.set_front_right_velocity_mps(0.0f);
00115   cmd.set_back_left_velocity_mps(0.0f);
00116   cmd.set_back_right_velocity_mps(0.0f);
00117
00118   // TODO(safety): Send final zero-velocity frame before deactivation
```

```
00119
00120   odom_pub_->on_deactivate();
00121   joint_state_pub_->on_deactivate();
00122   imu_pub_->on_deactivate();
00123
00124   return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
00125          CallbackReturn::SUCCESS;
00126 }
00127
00128 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00129 StarSpiDriverNode::on_cleanup(const rclcpp_lifecycle::State &)
00130 {
00131   RCLCPP_INFO(get_logger(), "Cleaning up StarSpiDriverNode...");
00132
00133   spi_driver_->close_device();
00134   spi_driver_.reset();
00135   converter_.reset();
00136
00137   odom_pub_.reset();
00138   joint_state_pub_.reset();
00139   imu_pub_.reset();
00140   cmd_vel_sub_.reset();
00141
00142   return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
00143          CallbackReturn::SUCCESS;
00144 }
00145
00146 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00147 StarSpiDriverNode::on_shutdown(const rclcpp_lifecycle::State & prev_state)
00148 {
00149   RCLCPP_INFO(get_logger(), "Shutting down StarSpiDriverNode...");
00150   return on_cleanup(prev_state);
00151 }
00152
00153 void StarSpiDriverNode::cmd_vel_callback(
00154   const geometry_msgs::msg::Twist::SharedPtr msg)
00155 {
00156   current_cmd_vel_ = *msg;
00157   last_cmd_vel_time_ = get_clock()->now();
00158 }
00159
00160 void StarSpiDriverNode::timer_callback()
00161 {
00162   // Check for command velocity timeout (watchdog safety)
00163   auto now = get_clock()->now();
00164   double time_since_cmd = (now - last_cmd_vel_time_).seconds();
00165
00166   geometry_msgs::msg::Twist cmd_vel_to_send;
00167   if (time_since_cmd > (cmd_vel_timeout_ms_ / 1000.0)) {
00168     // Timeout expired: send zero velocity for safety
00169   } else {
00170     cmd_vel_to_send = current_cmd_vel_;
00171   }
00172
00173   // Convert Twist to protobuf VelocityCommand
00174   star::v1::VelocityCommand velocity_cmd;
00175
00176   if (!converter_->twist_to_velocity_command(cmd_vel_to_send, velocity_cmd)) {
00177     RCLCPP_WARN(get_logger(),
00178                 "Failed to convert Twist to VelocityCommand (NaN/Inf?)");
00179     // Send zero if conversion fails
00180     velocity_cmd.set_front_left_velocity_mps(0.0f);
00181     velocity_cmd.set_front_right_velocity_mps(0.0f);
00182     velocity_cmd.set_back_left_velocity_mps(0.0f);
00183     velocity_cmd.set_back_right_velocity_mps(0.0f);
00184   }
00185
00186   // Encode protobuf payload into SPI frame
00187   std::vector<uint8_t> payload(velocity_cmd.ByteSizeLong());
00188   velocity_cmd.SerializeToArray(payload.data(), payload.size());
00189
00190   FrameType frame_type = FrameType::VelocityCommand;
00191   uint8_t flags = 0x00;
00192
00193   std::vector<uint8_t> tx_frame;
00194   SpiDriver::encode_frame(tx_seq_++, frame_type, flags, payload, tx_frame);
00195
00196   // Perform full-duplex SPI transfer
00197   std::vector<uint8_t> rx_frame;
00198   if (!spi_driver_->transfer(tx_frame, rx_frame)) {
00199     RCLCPP_ERROR_THROTTLE(get_logger(), *get_clock(), kLogThrottleMs,
00200                           "SPI Transfer Failed");
00201     return;
00202   }
00203
00204   // Decode received frame
00205
```

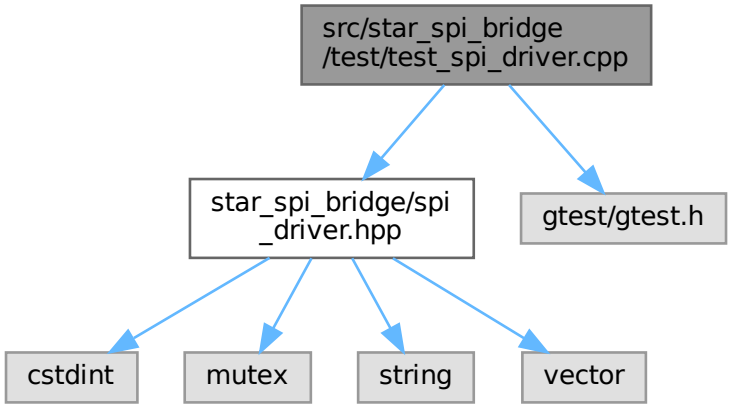
```
00206   uint16_t rx_seq_;
00207
00208   FrameType rx_type_;
00209
00210   uint8_t rx_flags_;
00211
00212   std::vector<uint8_t> rx_payload_;
00213
00214   if (!SpiDriver::decode_frame(rx_frame, rx_seq_, rx_type_, rx_flags_,
00215                                rx_payload_))
00216   {
00217     RCLCPP_WARN_THROTTLE(
00218         get_logger(), *get_clock(), kLogThrottleMs,
00219         "SPI Frame Decode Failed (CRC mismatch or garbage)");
00220
00221     return;
00222   }
00223
00224   // Parse telemetry and publish ROS2 messages
00225   if (rx_type_ == FrameType::TelemetryData) {
00226     star::v1::TelemetryData telemetry;
00227     if (telemetry.ParseFromArray(rx_payload_.data(), rx_payload_.size())) {
00228       // Check emergency stop flag from motor controller
00229       if (telemetry.emergency_stop()) {
00230         RCLCPP_ERROR_THROTTLE(get_logger(), *get_clock(), kLogThrottleMs,
00231                               "RX72N EMERGENCY STOP ACTIVE");
00232       }
00233
00234       // Publish Odometry
00235       nav_msgs::msg::Odometry odom;
00236       odom.header.stamp = now;
00237       converter_->telemetry_to_odometry(telemetry, odom);
00238       odom_pub_->publish(odom);
00239
00240       // Publish Joint States
00241       sensor_msgs::msg::JointState joint_state;
00242       joint_state.header.stamp = now;
00243       converter_->telemetry_to_joint_state(telemetry, joint_state);
00244       joint_state_pub_->publish(joint_state);
00245
00246       // Publish IMU data
00247       const auto & imu_data = telemetry.imu();
00248       // Skip if firmware has not populated IMU fields (proto3 default = all zeros).
00249       // A functioning IMU at rest reads ~9.81 m/s^2 on Z; zero indicates no data.
00250       const bool imu_populated = (imu_data.accel_z_mps2() != 0.0 ||
00251         imu_data.gyro_z_rad_per_s() != 0.0);
00252       if (!imu_populated) {
00253         return;  // wait for real IMU data
00254       }
00255
00256       sensor_msgs::msg::Imu imu_msg;
00257       imu_msg.header.stamp = now;
00258       imu_msg.header.frame_id = "imu_link";
00259
00260       tf2::Quaternion q;
00261       q.setRPY(imu_data.roll_rad(), imu_data.pitch_rad(), imu_data.yaw_rad());
00262       imu_msg.orientation.x = q.x();
00263       imu_msg.orientation.y = q.y();
00264       imu_msg.orientation.z = q.z();
00265       imu_msg.orientation.w = q.w();
00266       // Orientation covariance (3x3 row-major): sigma ~5.7 deg (rad^2)
00267       imu_msg.orientation_covariance = {
00268         kImuOrientationVar, 0.0, 0.0,
00269         0.0, kImuOrientationVar, 0.0,
00270         0.0, 0.0, kImuOrientationVar};
00271
00272       imu_msg.angular_velocity.x = imu_data.gyro_x_rad_per_s();
00273       imu_msg.angular_velocity.y = imu_data.gyro_y_rad_per_s();
00274       imu_msg.angular_velocity.z = imu_data.gyro_z_rad_per_s();
00275       // Angular velocity covariance (3x3 row-major): sigma ~0.032 rad/s
00276       imu_msg.angular_velocity_covariance = {
00277         kImuAngularVelocityVar, 0.0, 0.0,
00278         0.0, kImuAngularVelocityVar, 0.0,
00279         0.0, 0.0, kImuAngularVelocityVar};
00280
00281       imu_msg.linear_acceleration.x = imu_data.accel_x_mps2();
00282       imu_msg.linear_acceleration.y = imu_data.accel_y_mps2();
00283       imu_msg.linear_acceleration.z = imu_data.accel_z_mps2();
00284       // Linear acceleration covariance (3x3 row-major): sigma ~0.1 m/s^2
00285       imu_msg.linear_acceleration_covariance = {
00286         kImuLinearAccelVar, 0.0, 0.0,
00287         0.0, kImuLinearAccelVar, 0.0,
00288         0.0, 0.0, kImuLinearAccelVar};
00289
00290       imu_pub_->publish(imu_msg);
00291
00292     } else {
```

```
00293       RCLCPP_WARN(get_logger(), "Failed to parse TelemetryData protobuf");
00294     }
00295   }
00296 }
00297
00298 }  // namespace star_spi_bridge
```

9.35 src/star_spi_bridge/test/test_spi_driver.cpp File Reference

```
#include "star_spi_bridge/spi_driver.hpp"
#include <gtest/gtest.h>
```

Include dependency graph for test_spi_driver.cpp:



Classes

- class SpiDriverTest
- class SpiDriver

  SPI driver for framed communication with the RX72N peripheral MCU.

Enumerations

- enum class FrameType

  Frame type identifiers for SPI protocol messages.

Functions

- TEST_F (SpiDriverTest, CRC32Calculation)
- TEST_F (SpiDriverTest, FrameEncoding)
- TEST_F (SpiDriverTest, FrameDecoding)
- TEST_F (SpiDriverTest, EncodeDecodeRoundTrip)
- TEST_F (SpiDriverTest, DecodeRejectsCRCCorruption)
- TEST_F (SpiDriverTest, EncodeRejectsOversizedPayload)

9.35.1 Enumeration Type Documentation

9.35.1.1 FrameType

```
enum class star_spi_bridge::FrameType : uint8_t [strong]
```

Frame type identifiers for SPI protocol messages.

Each frame carries a one-byte type field at offset 6 in the header. Values match the RX72N firmware rx_frame_↩ type_t enum so both sides agree on the semantics of every message.

See also

SpiDriver::encode_frame Frame construction

SpiDriver::decode_frame Frame parsing

Since

Version 1.0.0

Definition at line 27 of file spi_driver.hpp.

9.35.2 Function Documentation

9.35.2.1 TEST_F() [1/6]

```
TEST_F (
            SpiDriverTest ,
            CRC32Calculation )
```

Definition at line 19 of file test_spi_driver.cpp.

References SpiDriver::calculate_crc32().

Here is the call graph for this function:

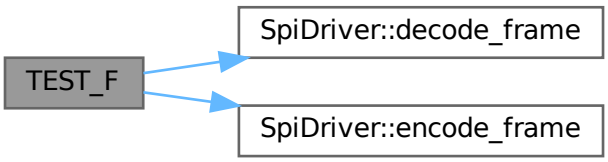
9.35.2.2 TEST_F() [2/6]

```
TEST_F (
            SpiDriverTest ,
            DecodeRejectsCRCCorruption )
```

Definition at line 89 of file test_spi_driver.cpp.

References SpiDriver::decode_frame(), and SpiDriver::encode_frame().

Here is the call graph for this function:



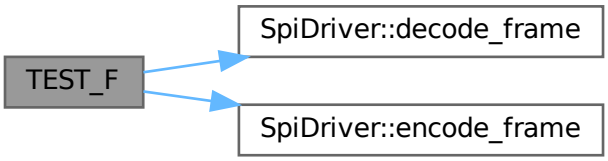
9.35.2.3 TEST_F() [3/6]

```
TEST_F (
            SpiDriverTest ,
            EncodeDecodeRoundTrip )
```

Definition at line 71 of file test_spi_driver.cpp.

References SpiDriver::decode_frame(), and SpiDriver::encode_frame().

Here is the call graph for this function:



9.35.2.4 TEST_F() [4/6]

```
TEST_F (
            SpiDriverTest ,
            EncodeRejectsOversizedPayload )
```

Definition at line 106 of file test_spi_driver.cpp.

References SpiDriver::encode_frame(), and SpiDriver::k_max_payload_size.

Here is the call graph for this function:

9.35.2.5 TEST_F() [5/6]

```
TEST_F (
            SpiDriverTest ,
            FrameDecoding )
```

Definition at line 54 of file test_spi_driver.cpp.

References SpiDriver::decode_frame().

Here is the call graph for this function:

9.35.2.6 TEST_F() [6/6]

```
TEST_F (
            SpiDriverTest ,
            FrameEncoding )
```

Definition at line 28 of file test_spi_driver.cpp.

References SpiDriver::encode_frame().

Here is the call graph for this function:

9.36 test_spi_driver.cpp

Go to the documentation of this file.

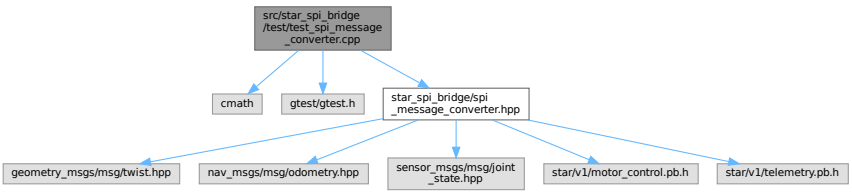
```
00001 // Copyright 2026 Locked Inc.
00002
00003 #include "star_spi_bridge/spi_driver.hpp"
00004
00005 #include <gtest/gtest.h>  // NOLINT
00006
00007 using star_spi_bridge::FrameType;
00008 using star_spi_bridge::SpiDriver;
00009
00010 class SpiDriverTest : public ::testing::Test
00011 {
00012 protected:
00013   void SetUp() override
00014   {
00015     // Setup code
00016   }
00017 };
00018
00019 TEST_F(SpiDriverTest, CRC32Calculation)
00020 {
00021   // Test vector: "123456789" -> 0xCBF43926
00022   std::string test_str = "123456789";
00023   std::vector<uint8_t> data(test_str.begin(), test_str.end());
00024
00025   EXPECT_EQ(SpiDriver::calculate_crc32(data), 0xCBF43926);
00026 }
00027
00028 TEST_F(SpiDriverTest, FrameEncoding)
00029 {
00030   std::vector<uint8_t> payload = {0x01, 0x02, 0x03};
00031   std::vector<uint8_t> frame;
00032   SpiDriver::encode_frame(100, FrameType::VelocityCommand, 0x00, payload, frame);
00033
00034   // Header (8) + Payload (3) + CRC (4) = 15 bytes
00035   EXPECT_EQ(frame.size(), 15);
00036
00037   // Check SYNC word (Little Endian: [0xAA, 0x55] for 0x55AA)
00038   EXPECT_EQ(frame[0], 0xAA);  // LSB first
00039   EXPECT_EQ(frame[1], 0x55);  // MSB second
00040
00041   // Check SEQ field (Little Endian: seq=100=0x0064 -> [0x64, 0x00])
00042   EXPECT_EQ(frame[2], 0x64);  // SEQ LSB
00043   EXPECT_EQ(frame[3], 0x00);  // SEQ MSB
00044
00045   // Check LEN field (Little Endian: len=3=0x0003 -> [0x03, 0x00])
```

```
00046   EXPECT_EQ(frame[4], 0x03);  // LEN LSB
00047   EXPECT_EQ(frame[5], 0x00);  // LEN MSB
00048
00049   // Check TYPE and FLAGS
00050   EXPECT_EQ(frame[6], 0x01);  // TYPE: VelocityCommand
00051   EXPECT_EQ(frame[7], 0x00);  // FLAGS: none
00052 }
00053
00054 TEST_F(SpiDriverTest, FrameDecoding)
00055 {
00056   // Construct a valid frame manually
00057   // SYNC(55AA) SEQ(0001) LEN(0001) TYPE(01) FLAGS(00) PAYLOAD(A5) CRC(...)
00058   // CRC for header+payload needs to be correct for decode to succeed
00059
00060   // We can't easily test decode without a working encode or pre-calculated frame
00061   // For Red Phase, we just assert that decode fails on garbage
00062   std::vector<uint8_t> garbage = {0x00, 0x00, 0x00};
00063   uint16_t seq = 0;
00064   FrameType type = FrameType::TelemetryData;
00065   uint8_t flags = 0x00;
00066   std::vector<uint8_t> payload = {};
00067
00068   EXPECT_FALSE(SpiDriver::decode_frame(garbage, seq, type, flags, payload));
00069 }
00070
00071 TEST_F(SpiDriverTest, EncodeDecodeRoundTrip)
00072 {
00073   std::vector<uint8_t> payload_in = {0xDE, 0xAD, 0xBE, 0xEF};
00074   std::vector<uint8_t> frame;
00075   SpiDriver::encode_frame(42, FrameType::TelemetryData, 0x03, payload_in, frame);
00076
00077   uint16_t seq = 0;
00078   FrameType type = FrameType::VelocityCommand;
00079   uint8_t flags = 0;
00080   std::vector<uint8_t> payload_out;
00081
00082   ASSERT_TRUE(SpiDriver::decode_frame(frame, seq, type, flags, payload_out));
00083   EXPECT_EQ(seq, 42);
00084   EXPECT_EQ(type, FrameType::TelemetryData);
00085   EXPECT_EQ(flags, 0x03);
00086   EXPECT_EQ(payload_out, payload_in);
00087 }
00088
00089 TEST_F(SpiDriverTest, DecodeRejectsCRCCorruption)
00090 {
00091   std::vector<uint8_t> payload = {0x01};
00092   std::vector<uint8_t> frame;
00093   SpiDriver::encode_frame(1, FrameType::VelocityCommand, 0x00, payload, frame);
00094
00095   // Corrupt the last byte (CRC)
00096   frame.back() ^= 0xFF;
00097
00098   uint16_t seq = 0;
00099   FrameType type = FrameType::TelemetryData;
00100   uint8_t flags = 0;
00101   std::vector<uint8_t> decoded_payload;
00102
00103   EXPECT_FALSE(SpiDriver::decode_frame(frame, seq, type, flags, decoded_payload));
00104 }
00105
00106 TEST_F(SpiDriverTest, EncodeRejectsOversizedPayload)
00107 {
00108   std::vector<uint8_t> oversized(SpiDriver::k_max_payload_size + 1, 0xAA);
00109   std::vector<uint8_t> frame;
00110
00111   EXPECT_THROW(
00112     SpiDriver::encode_frame(0, FrameType::VelocityCommand, 0x00, oversized, frame),
00113     std::invalid_argument);
00114 }
```

9.37 src/star_spi_bridge/test/test_spi_message_converter.cpp File Reference

```
#include <cmath>
#include <gtest/gtest.h>
#include "star_spi_bridge/spi_message_converter.hpp"
```

Include dependency graph for test_spi_message_converter.cpp:



Classes

- class SpiMessageConverterTest
- class SpiMessageConverter

Functions

- TEST_F (SpiMessageConverterTest, TwistToVelocity_Forward)
- TEST_F (SpiMessageConverterTest, TwistToVelocity_RotateLeft)
- TEST_F (SpiMessageConverterTest, TwistToVelocity_NaN)
- TEST_F (SpiMessageConverterTest, TelemetryToOdometry_FirstMessage)
- TEST_F (SpiMessageConverterTest, TelemetryToOdometry_ForwardMovement)
- TEST_F (SpiMessageConverterTest, TelemetryToOdometry_WithVelocity)
- TEST_F (SpiMessageConverterTest, TelemetryToJointState_Names)
- TEST_F (SpiMessageConverterTest, TelemetryToJointState_Position)
- TEST_F (SpiMessageConverterTest, TelemetryToJointState_Velocity)
- TEST_F (SpiMessageConverterTest, RoundTrip_TwistToVelocityToJointState)

9.37.1 Function Documentation

9.37.1.1 TEST_F() [1/10]

```
TEST_F (
            SpiMessageConverterTest ,
            RoundTrip_TwistToVelocityToJointState )
```

Definition at line 203 of file test_spi_message_converter.cpp.

9.37.1.2 TEST_F() [2/10]

```
TEST_F (
            SpiMessageConverterTest ,
            TelemetryToJointState_Names )
```

Definition at line 140 of file test_spi_message_converter.cpp.

9.37.1.3 TEST_F() [3/10]

```
TEST_F (
            SpiMessageConverterTest ,
            TelemetryToJointState_Position )
```

Definition at line 158 of file test_spi_message_converter.cpp.

9.37.1.4 TEST_F() [4/10]

```
TEST_F (
            SpiMessageConverterTest ,
            TelemetryToJointState_Velocity )
```

Definition at line 178 of file test_spi_message_converter.cpp.

9.37.1.5 TEST_F() [5/10]

```
TEST_F (
            SpiMessageConverterTest ,
            TelemetryToOdometry_FirstMessage )
```

Definition at line 59 of file test_spi_message_converter.cpp.

9.37.1.6 TEST_F() [6/10]

```
TEST_F (
            SpiMessageConverterTest ,
            TelemetryToOdometry_ForwardMovement )
```

Definition at line 75 of file test_spi_message_converter.cpp.

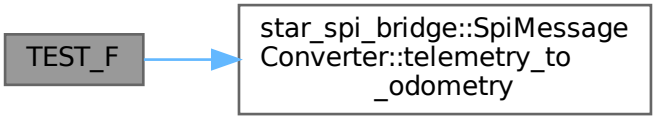
9.37.1.7 TEST_F() [7/10]

```
TEST_F (
            SpiMessageConverterTest ,
            TelemetryToOdometry_WithVelocity )
```

Definition at line 104 of file test_spi_message_converter.cpp.

References star_spi_bridge::SpiMessageConverter::telemetry_to_odometry().

Here is the call graph for this function:



9.37.1.8 TEST_F() [8/10]

```
TEST_F (
              SpiMessageConverterTest ,
              TwistToVelocity_Forward )
```

Definition at line 18 of file test_spi_message_converter.cpp.

9.37.1.9 TEST_F() [9/10]

```
TEST_F (
              SpiMessageConverterTest ,
              TwistToVelocity_NaN )
```

Definition at line 46 of file test_spi_message_converter.cpp.

9.37.1.10 TEST_F() [10/10]

```
TEST_F (
              SpiMessageConverterTest ,
              TwistToVelocity_RotateLeft )
```

Definition at line 31 of file test_spi_message_converter.cpp.

9.38 test_spi_message_converter.cpp

Go to the documentation of this file.

```
00001 // Copyright 2026 Locked Inc.
00002
00003 #include <cmath>
00004
00005 #include <gtest/gtest.h>  // NOLINT
00006
00007 #include "star_spi_bridge/spi_message_converter.hpp"
00008
00009 using star_spi_bridge::SpiMessageConverter;
00010
00011 class SpiMessageConverterTest : public ::testing::Test
00012 {
00013 protected:
00014   SpiMessageConverter::Parameters params{0.150, 0.0325, 11599};
00015   SpiMessageConverter converter{params};
00016 };
00017
00018 TEST_F(SpiMessageConverterTest, TwistToVelocity_Forward)
00019 {
00020   geometry_msgs::msg::Twist twist;
00021   twist.linear.x = 1.0;
00022   twist.angular.z = 0.0;
00023
00024   star::v1::VelocityCommand cmd;
00025   EXPECT_TRUE(converter.twist_to_velocity_command(twist, cmd));
00026
00027   EXPECT_NEAR(cmd.front_left_velocity_mps(), 1.0, 0.001);
00028   EXPECT_NEAR(cmd.front_right_velocity_mps(), 1.0, 0.001);
00029 }
00030
00031 TEST_F(SpiMessageConverterTest, TwistToVelocity_RotateLeft)
00032 {
00033   geometry_msgs::msg::Twist twist;
00034   twist.linear.x = 0.0;
00035   twist.angular.z = 2.0;  // 2 rad/s
00036
00037   star::v1::VelocityCommand cmd;
```

```
00038   EXPECT_TRUE(converter.twist_to_velocity_command(twist, cmd));
00039
00040   // v_right = 0 + 2 * (0.15/2) = 0.15
00041   // v_left = 0 - 2 * (0.15/2) = -0.15
00042   EXPECT_NEAR(cmd.front_right_velocity_mps(), 0.15, 0.001);
00043   EXPECT_NEAR(cmd.front_left_velocity_mps(), -0.15, 0.001);
00044 }
00045
00046 TEST_F(SpiMessageConverterTest, TwistToVelocity_NaN)
00047 {
00048   geometry_msgs::msg::Twist twist;
00049   twist.linear.x = std::numeric_limits<double>::quiet_NaN();
00050
00051   star::v1::VelocityCommand cmd;
00052   EXPECT_FALSE(converter.twist_to_velocity_command(twist, cmd));
00053 }
00054
00055 // ============================================================================
00056 // Tests for telemetry_to_odometry
00057 // ============================================================================
00058
00059 TEST_F(SpiMessageConverterTest, TelemetryToOdometry_FirstMessage)
00060 {
00061   star::v1::TelemetryData telemetry;
00062   telemetry.mutable_encoder_front_left()->set_ticks(100);
00063   telemetry.mutable_encoder_front_right()->set_ticks(100);
00064   telemetry.mutable_encoder_back_left()->set_ticks(100);
00065   telemetry.mutable_encoder_back_right()->set_ticks(100);
00066
00067   nav_msgs::msg::Odometry odom;
00068   converter.telemetry_to_odometry(telemetry, odom);
00069
00070   // First message should initialize state but not update pose
00071   EXPECT_NEAR(odom.pose.pose.position.x, 0.0, 0.001);
00072   EXPECT_NEAR(odom.pose.pose.position.y, 0.0, 0.001);
00073 }
00074
00075 TEST_F(SpiMessageConverterTest, TelemetryToOdometry_ForwardMovement)
00076 {
00077   // First message to initialize state
00078   star::v1::TelemetryData telemetry1;
00079   telemetry1.mutable_encoder_front_left()->set_ticks(0);
00080   telemetry1.mutable_encoder_front_right()->set_ticks(0);
00081   telemetry1.mutable_encoder_back_left()->set_ticks(0);
00082   telemetry1.mutable_encoder_back_right()->set_ticks(0);
00083
00084   nav_msgs::msg::Odometry odom1;
00085   converter.telemetry_to_odometry(telemetry1, odom1);
00086
00087   // Second message with forward movement (same ticks on both sides = straight)
00088   star::v1::TelemetryData telemetry2;
00089   int64_t ticks = 11599;  // One full wheel revolution
00090   telemetry2.mutable_encoder_front_left()->set_ticks(ticks);
00091   telemetry2.mutable_encoder_front_right()->set_ticks(ticks);
00092   telemetry2.mutable_encoder_back_left()->set_ticks(ticks);
00093   telemetry2.mutable_encoder_back_right()->set_ticks(ticks);
00094
00095   nav_msgs::msg::Odometry odom2;
00096   converter.telemetry_to_odometry(telemetry2, odom2);
00097
00098   // One revolution = 2 * pi * radius = 2 * pi * 0.0325 ~ 0.204 m
00099   double expected_dist = 2.0 * M_PI * 0.0325;
00100   EXPECT_NEAR(odom2.pose.pose.position.x, expected_dist, 0.01);
00101   EXPECT_NEAR(odom2.pose.pose.position.y, 0.0, 0.001);
00102 }
00103
00104 TEST_F(SpiMessageConverterTest, TelemetryToOdometry_WithVelocity)
00105 {
00106   // Create fresh converter for this test
00107   SpiMessageConverter test_converter{params};
00108
00109   // First message
00110   star::v1::TelemetryData telemetry1;
00111   telemetry1.mutable_encoder_front_left()->set_ticks(0);
00112   telemetry1.mutable_encoder_front_right()->set_ticks(0);
00113   telemetry1.mutable_encoder_back_left()->set_ticks(0);
00114   telemetry1.mutable_encoder_back_right()->set_ticks(0);
00115   nav_msgs::msg::Odometry odom1;
00116   test_converter.telemetry_to_odometry(telemetry1, odom1);
00117
00118   // Second message with velocity data
00119   star::v1::TelemetryData telemetry2;
00120   telemetry2.mutable_encoder_front_left()->set_ticks(100);
00121   telemetry2.mutable_encoder_front_left()->set_velocity_mps(1.0);
00122   telemetry2.mutable_encoder_front_right()->set_ticks(100);
00123   telemetry2.mutable_encoder_front_right()->set_velocity_mps(1.0);
00124   telemetry2.mutable_encoder_back_left()->set_ticks(100);
```

```
00125   telemetry2.mutable_encoder_back_left()->set_velocity_mps(1.0);
00126   telemetry2.mutable_encoder_back_right()->set_ticks(100);
00127   telemetry2.mutable_encoder_back_right()->set_velocity_mps(1.0);
00128
00129   nav_msgs::msg::Odometry odom2;
00130   test_converter.telemetry_to_odometry(telemetry2, odom2);
00131
00132   EXPECT_NEAR(odom2.twist.twist.linear.x, 1.0, 0.001);
00133   EXPECT_NEAR(odom2.twist.twist.angular.z, 0.0, 0.001);
00134 }
00135
00136 // ============================================================================
00137 // Tests for telemetry_to_joint_state
00138 // ============================================================================
00139
00140 TEST_F(SpiMessageConverterTest, TelemetryToJointState_Names)
00141 {
00142   star::v1::TelemetryData telemetry;
00143   telemetry.mutable_encoder_front_left()->set_ticks(0);
00144   telemetry.mutable_encoder_front_right()->set_ticks(0);
00145   telemetry.mutable_encoder_back_left()->set_ticks(0);
00146   telemetry.mutable_encoder_back_right()->set_ticks(0);
00147
00148   sensor_msgs::msg::JointState joint_state;
00149   converter.telemetry_to_joint_state(telemetry, joint_state);
00150
00151   ASSERT_EQ(joint_state.name.size(), 4u);
00152   EXPECT_EQ(joint_state.name[0], "front_left_wheel");
00153   EXPECT_EQ(joint_state.name[1], "front_right_wheel");
00154   EXPECT_EQ(joint_state.name[2], "back_left_wheel");
00155   EXPECT_EQ(joint_state.name[3], "back_right_wheel");
00156 }
00157
00158 TEST_F(SpiMessageConverterTest, TelemetryToJointState_Position)
00159 {
00160   star::v1::TelemetryData telemetry;
00161   // One full revolution = 11599 ticks = 2*PI radians
00162   int64_t one_rev = 11599;
00163   telemetry.mutable_encoder_front_left()->set_ticks(one_rev);
00164   telemetry.mutable_encoder_front_right()->set_ticks(one_rev / 2);
00165   telemetry.mutable_encoder_back_left()->set_ticks(one_rev * 2);
00166   telemetry.mutable_encoder_back_right()->set_ticks(0);
00167
00168   sensor_msgs::msg::JointState joint_state;
00169   converter.telemetry_to_joint_state(telemetry, joint_state);
00170
00171   ASSERT_EQ(joint_state.position.size(), 4u);
00172   EXPECT_NEAR(joint_state.position[0], 2.0 * M_PI, 0.01);  // FL: 1 rev
00173   EXPECT_NEAR(joint_state.position[1], M_PI, 0.01);        // FR: 0.5 rev
00174   EXPECT_NEAR(joint_state.position[2], 4.0 * M_PI, 0.02);  // BL: 2 rev
00175   EXPECT_NEAR(joint_state.position[3], 0.0, 0.001);        // BR: 0 rev
00176 }
00177
00178 TEST_F(SpiMessageConverterTest, TelemetryToJointState_Velocity)
00179 {
00180   star::v1::TelemetryData telemetry;
00181   double velocity_mps = 1.0;  // 1 m/s linear
00182   telemetry.mutable_encoder_front_left()->set_velocity_mps(velocity_mps);
00183   telemetry.mutable_encoder_front_right()->set_velocity_mps(velocity_mps);
00184   telemetry.mutable_encoder_back_left()->set_velocity_mps(velocity_mps);
00185   telemetry.mutable_encoder_back_right()->set_velocity_mps(velocity_mps);
00186
00187   sensor_msgs::msg::JointState joint_state;
00188   converter.telemetry_to_joint_state(telemetry, joint_state);
00189
00190   // angular_velocity = linear_velocity / radius = 1.0 / 0.0325 ~ 30.77 rad/s
00191   double expected_angular = velocity_mps / 0.0325;
00192   ASSERT_EQ(joint_state.velocity.size(), 4u);
00193   EXPECT_NEAR(joint_state.velocity[0], expected_angular, 0.1);
00194   EXPECT_NEAR(joint_state.velocity[1], expected_angular, 0.1);
00195   EXPECT_NEAR(joint_state.velocity[2], expected_angular, 0.1);
00196   EXPECT_NEAR(joint_state.velocity[3], expected_angular, 0.1);
00197 }
00198
00199 // ============================================================================
00200 // Round-trip encode/decode test
00201 // ============================================================================
00202
00203 TEST_F(SpiMessageConverterTest, RoundTrip_TwistToVelocityToJointState)
00204 {
00205   // Create a twist command
00206   geometry_msgs::msg::Twist twist;
00207   twist.linear.x = 0.5;
00208   twist.angular.z = 0.2;
00209
00210   // Convert to velocity command
00211   star::v1::VelocityCommand cmd;
```

```
00212   ASSERT_TRUE(converter.twist_to_velocity_command(twist, cmd));
00213
00214   // Verify the velocity command was set correctly
00215   // v_right = 0.5 + 0.2 * (0.15/2) = 0.5 + 0.015 = 0.515
00216   // v_left  = 0.5 - 0.2 * (0.15/2) = 0.5 - 0.015 = 0.485
00217   EXPECT_NEAR(cmd.front_right_velocity_mps(), 0.515, 0.001);
00218   EXPECT_NEAR(cmd.front_left_velocity_mps(), 0.485, 0.001);
00219   EXPECT_NEAR(cmd.back_right_velocity_mps(), 0.515, 0.001);
00220   EXPECT_NEAR(cmd.back_left_velocity_mps(), 0.485, 0.001);
00221
00222   // Create telemetry with matching velocities (simulating firmware response)
00223   star::v1::TelemetryData telemetry;
00224   telemetry.mutable_encoder_front_left()->set_velocity_mps(cmd.front_left_velocity_mps());
00225   telemetry.mutable_encoder_front_right()->set_velocity_mps(cmd.front_right_velocity_mps());
00226   telemetry.mutable_encoder_back_left()->set_velocity_mps(cmd.back_left_velocity_mps());
00227   telemetry.mutable_encoder_back_right()->set_velocity_mps(cmd.back_right_velocity_mps());
00228
00229   // Convert telemetry to joint state
00230   sensor_msgs::msg::JointState joint_state;
00231   converter.telemetry_to_joint_state(telemetry, joint_state);
00232
00233   // Verify velocities converted to rad/s
00234   double inv_radius = 1.0 / 0.0325;
00235   EXPECT_NEAR(joint_state.velocity[0], cmd.front_left_velocity_mps() * inv_radius, 0.1);
00236   EXPECT_NEAR(joint_state.velocity[1], cmd.front_right_velocity_mps() * inv_radius, 0.1);
00237 }
```

Index